
Assignment 1

Atif M. Mahmud

Task 1: Familiarize Yourself with the Dataset

There are two main subtasks:

1. Load the four data files that are part of this dataset.
2. Explore each dataset and define the data types of each column. Describe the contents of each column, including the nature of the data values. Follow a similar approach as demonstrated in the lecture and tutorial. Discuss the shape of the data, value ranges, and possible categories. Convert any data into a more appropriate format if needed.

Add as many code and markdown cells as necessary below. Typically, you should perform one type of investigation per code cell with output, and then comment on the results in a markdown cell below it.

After completing the analysis, write a concise summary in the **Task 1: summary report** cell at the end. The final report should succinctly summarize the main characteristics of each data file, how they relate to each other, and any notable features.

```
# this is your first code cell, add more cells for code and markdown as needed
# imports
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from IPython.display import display
from datetime import datetime
import plotly.express as px
import seaborn as sns
```

Exploring sessions.csv

```
# TASK 1a: SESSIONS.CSV
df_sessions = pd.read_csv("data/sessions.csv")
display(df_sessions)
display (f"The shape of the dataframe is: {df_sessions.shape}")
columns = [*df_sessions.columns]
print(f"Columns are: {columns}")
print("\nGiven below are descriptive stats on the numerical columns\
```

```

n")
df_sessions.describe()

{"columns":[{"name":"index","rawType":"int64","type":"integer"},
{"name":"session_id","rawType":"str","type":"string"},
{"name":"viewer_id","rawType":"str","type":"string"},
{"name":"streamer_id","rawType":"str","type":"string"},
{"name":"stream_id","rawType":"str","type":"string"},
{"name":"started_at","rawType":"str","type":"string"},
{"name":"ended_at","rawType":"str","type":"string"},
{"name":"duration_mins","rawType":"int64","type":"integer"},
{"name":"chat_messages_sent","rawType":"int64","type":"integer"},
{"name":"bits_cheered","rawType":"int64","type":"integer"},
{"name":"followed_during","rawType":"bool","type":"boolean"},
{"name":"subscribed_during","rawType":"bool","type":"boolean"}],"ref":
"98fee5a8-2824-4fe8-9df3-e7ac85157ece","rows":
[["0","SES000001","VWR00938","STR0055","SM00860","2024-01-18
19:16:00","2024-01-18 20:28:00","72","0","0","False","False"],
["1","SES000002","VWR00804","STR0034","SM00542","2024-01-04
17:11:00","2024-01-04 19:41:00","150","11","0","False","False"],
["2","SES000003","VWR00704","STR0008","SM00124","2024-02-15
15:11:00","2024-02-15 16:53:00","102","5","0","False","False"],
["3","SES000004","VWR00926","STR0024","SM00373","2024-01-18
19:59:00","2024-01-18 20:56:00","57","3","9663","False","False"],
["4","SES000005","VWR00883","STR0008","SM00120","2024-03-19
20:08:00","2024-03-19 20:22:00","14","7","0","False","False"],
["5","SES000006","VWR00860","STR0056","SM00877","2024-01-07
00:41:00","2024-01-07 02:13:00","92","13","0","False","False"],
["6","SES000007","VWR00925","STR0052","SM00828","2024-01-29
21:41:00","2024-01-30 01:59:00","258","13","0","False","False"],
["7","SES000008","VWR00519","STR0047","SM00730","2024-03-13
14:52:00","2024-03-13 14:59:00","7","0","0","False","False"],
["8","SES000009","VWR00901","STR0045","SM00715","2024-02-17
18:42:00","2024-02-17 19:13:00","31","1","0","False","False"],
["9","SES000010","VWR00869","STR0016","SM00238","2024-02-03
18:07:00","2024-02-03 18:54:00","47","5","0","False","False"],
["10","SES000011","VWR00812","STR0033","SM00530","2024-03-17
11:00:00","2024-03-17 11:33:00","33","1","0","False","False"],
["11","SES000012","VWR00424","STR0006","SM00096","2024-03-27
12:29:00","2024-03-27 14:37:00","128","1","0","False","False"],
["12","SES000013","VWR00839","STR0042","SM00676","2024-01-24
13:53:00","2024-01-24 14:05:00","12","0","0","False","False"],
["13","SES000014","VWR00633","STR0024","SM00374","2024-03-01
21:30:00","2024-03-01 22:57:00","87","3","3873","False","False"],
["14","SES000015","VWR00925","STR0030","SM00487","2024-01-07
19:38:00","2024-01-07 20:20:00","42","0","0","False","False"],
["15","SES000016","VWR00601","STR0012","SM00182","2024-03-12
12:37:00","2024-03-12 13:03:00","26","2","0","False","False"],
["16","SES000017","VWR00928","STR0020","SM00313","2024-03-20
02:48:00","2024-03-20 03:43:00","55","7","0","False","False"],

```

```
[ "17", "SES000018", "VWR00067", "STR0061", "SM00950", "2024-01-05  
00:02:00", "2024-01-05 02:29:00", "147", "8", "0", "True", "False"],  
[ "18", "SES000019", "VWR00992", "STR0057", "SM00894", "2024-01-11  
21:31:00", "2024-01-11 23:46:00", "135", "6", "3985", "False", "False"],  
[ "19", "SES000020", "VWR00726", "STR0074", "SM01179", "2024-01-31  
13:02:00", "2024-01-31 14:07:00", "65", "4", "0", "False", "False"],  
[ "20", "SES000021", "VWR00994", "STR0051", "SM00792", "2024-02-19  
12:10:00", "2024-02-19 12:46:00", "36", "5", "4638", "False", "False"],  
[ "21", "SES000022", "VWR00846", "STR0075", "SM01182", "2024-03-20  
18:00:00", "2024-03-20 18:51:00", "51", "7", "0", "False", "False"],  
[ "22", "SES000023", "VWR00722", "STR0064", "SM01010", "2024-03-09  
12:11:00", "2024-03-09 13:17:00", "66", "4", "0", "False", "False"],  
[ "23", "SES000024", "VWR00979", "STR0070", "SM01108", "2024-02-28  
20:44:00", "2024-02-28 20:49:00", "5", "1", "0", "False", "False"],  
[ "24", "SES000025", "VWR00616", "STR0024", "SM00367", "2024-03-30  
16:15:00", "2024-03-30 16:29:00", "14", "1", "0", "False", "False"],  
[ "25", "SES000026", "VWR00387", "STR0065", "SM01021", "2024-03-11  
16:06:00", "2024-03-11 18:50:00", "164", "13", "0", "False", "False"],  
[ "26", "SES000027", "VWR00959", "STR0060", "SM00937", "2024-01-07  
20:20:00", "2024-01-07 21:57:00", "97", "0", "6251", "False", "False"],  
[ "27", "SES000028", "VWR00852", "STR0066", "SM01043", "2024-03-25  
16:41:00", "2024-03-25 16:49:00", "8", "0", "0", "False", "False"],  
[ "28", "SES000029", "VWR00682", "STR0005", "SM00080", "2024-03-21  
22:10:00", "2024-03-21 22:27:00", "17", "2", "0", "False", "False"],  
[ "29", "SES000030", "VWR00217", "STR0021", "SM00327", "2024-01-31  
14:06:00", "2024-01-31 14:56:00", "50", "2", "0", "False", "False"],  
[ "30", "SES000031", "VWR00455", "STR0054", "SM00848", "2024-01-23  
18:17:00", "2024-01-23 21:33:00", "196", "1", "0", "False", "False"],  
[ "31", "SES000032", "VWR00628", "STR0067", "SM01056", "2024-02-08  
09:05:00", "2024-02-08 11:35:00", "150", "33", "3589", "False", "False"],  
[ "32", "SES000033", "VWR00624", "STR0006", "SM00100", "2024-01-08  
21:50:00", "2024-01-08 22:13:00", "23", "5", "0", "False", "False"],  
[ "33", "SES000034", "VWR00646", "STR0071", "SM01132", "2024-02-18  
23:42:00", "2024-02-19 00:58:00", "76", "4", "412", "True", "False"],  
[ "34", "SES000035", "VWR00878", "STR0023", "SM00358", "2024-03-16  
11:47:00", "2024-03-16 11:55:00", "8", "0", "0", "False", "False"],  
[ "35", "SES000036", "VWR00928", "STR0055", "SM00864", "2024-03-10  
08:33:00", "2024-03-10 08:46:00", "13", "0", "0", "False", "False"],  
[ "36", "SES000037", "VWR00382", "STR0018", "SM00271", "2024-01-02  
21:05:00", "2024-01-02 23:08:00", "123", "0", "0", "False", "False"],  
[ "37", "SES000038", "VWR00980", "STR0060", "SM00936", "2024-02-28  
09:47:00", "2024-02-28 10:52:00", "65", "7", "0", "False", "False"],  
[ "38", "SES000039", "VWR00772", "STR0026", "SM00412", "2024-01-31  
14:11:00", "2024-01-31 15:06:00", "55", "2", "0", "False", "False"],  
[ "39", "SES000040", "VWR00890", "STR0072", "SM01137", "2024-02-20  
18:21:00", "2024-02-20 20:01:00", "100", "12", "0", "False", "False"],  
[ "40", "SES000041", "VWR00426", "STR0049", "SM00766", "2024-03-07  
14:54:00", "2024-03-07 15:31:00", "37", "7", "0", "False", "False"],  
[ "41", "SES000042", "VWR00259", "STR0024", "SM00368", "2024-02-19
```

```

13:44:00", "2024-02-19 16:10:00", "146", "3", "0", "False", "False"],
["42", "SES000043", "VWR00927", "STR0003", "SM00047", "2024-02-17
12:49:00", "2024-02-17 13:27:00", "38", "0", "0", "False", "False"],
["43", "SES000044", "VWR00969", "STR0006", "SM00098", "2024-03-14
20:36:00", "2024-03-14 20:46:00", "10", "0", "0", "False", "False"],
["44", "SES000045", "VWR00978", "STR0049", "SM00759", "2024-01-05
23:16:00", "2024-01-06 03:12:00", "236", "47", "0", "False", "False"],
["45", "SES000046", "VWR00665", "STR0004", "SM00069", "2024-03-07
08:37:00", "2024-03-07 10:09:00", "92", "15", "0", "False", "False"],
["46", "SES000047", "VWR00765", "STR0055", "SM00859", "2024-03-23
12:27:00", "2024-03-23 13:05:00", "38", "1", "0", "False", "False"],
["47", "SES000048", "VWR00865", "STR0044", "SM00695", "2024-02-09
16:56:00", "2024-02-09 17:26:00", "30", "1", "0", "False", "False"],
["48", "SES000049", "VWR00872", "STR0044", "SM00701", "2024-03-08
12:59:00", "2024-03-08 14:29:00", "90", "9", "254", "False", "False"],
["49", "SES000050", "VWR00949", "STR0030", "SM00475", "2024-01-11
15:37:00", "2024-01-11
19:24:00", "227", "7", "0", "False", "False"]], "shape":
{"columns": 11, "rows": 22825}}

```

'The shape of the dataframe is: (22825, 11)'

Columns are: ['session_id', 'viewer_id', 'streamer_id', 'stream_id', 'started_at', 'ended_at', 'duration_mins', 'chat_messages_sent', 'bits_cheered', 'followed_during', 'subscribed_during']

Given below are descriptive stats on the numerical columns

```

{"columns": [{"name": "index", "rawType": "str", "type": "string"},
{"name": "duration_mins", "rawType": "float64", "type": "float"},
{"name": "chat_messages_sent", "rawType": "float64", "type": "float"},
{"name": "bits_cheered", "rawType": "float64", "type": "float"}], "ref": "fd5
f3cee-405a-4bc7-9b63-c3c2a6f09b16", "rows":
[["count", "22825.0", "22825.0", "22825.0"],
["mean", "59.05130339539978", "6.4426286966046", "398.42532311062433"],
["std", "60.38397876511292", "46.57630913705245", "1571.5286582612994"],
["min", "0.0", "0.0", "0.0"], ["25%", "15.0", "0.0", "0.0"],
["50%", "39.0", "1.0", "0.0"], ["75%", "83.0", "5.0", "0.0"],
["max", "443.0", "1987.0", "10000.0"]], "shape": {"columns": 3, "rows": 8}}

```

Task 1a: Summarizing sessions.csv

Since every file is converted to a dataframe and is being treated like a table, I will often use the term table and file interchangeably (within reason) in this assignment.

Overview

This table contains information about individual viewing sessions.

Table shape

- **Rows:** There are 22825 rows (22824 entries, since one row is for the column headings.)
- **Columns:** There are 11 columns. They are described below.

Columns description

Column Header	Decription	Type of data	Comments
<code>session_id</code>	unique identifier for the individual session	Categorical (nominal)	N/A
<code>viewer_id</code>	unique identifer for the viewer (consistent across other files)	Cateogrical (nominal)	N/A
<code>streamer_id</code>	unique identifer for the viewer (consistent across other files)	Categorical (nominal)	N/A
<code>stream_id</code>	unique identifer for the stream session (consistent across other files)	Categorical (nominal)	N/A
<code>started_at</code>	Time and date the stream started. Format: YYYY-MM-DD HH:MM:SS	Numerical (interval)	I will create another column with this in unix timestamp to make calculations easier
<code>ended_at</code>	Time and date the stream ended. Format: YYYY-MM-DD HH:MM:SS	Numerical (interval)	I will create another column with this in unix timestamp to make calculations easier
<code>duration_mins</code>	The length of the stream in minutes.	Numerical (ratio)	N/A
<code>chat_messages_sent</code>	The number of messages set on the chat	Numerical (ratio)	N/A
<code>bits_cheered</code>	The number bit cheers sent (a twitch currency)	Numerical (ratio)	N/A
<code>followed_during</code>	Boolean representing if the viewer followed the	Categorical (boolean)	N/A

Column Header	Decription	Type of data	Comments
	streamer during the session		
subscribed_during	Boolean representing if the viewer subscribed the streamer during the session	Categorical (boolean)	N/A

Below, I will convert the `started_at` and `ended_at` time to unix timestamp and do `describe()` again to see if any weird time is there.

```
# Task 1a: Continued, creating new columns with unix_timestamp
df_sessions["started_at_unix"] =
(pd.to_datetime(df_sessions["started_at"]) - pd.Timestamp("1970-01-01")) // pd.Timedelta("1s")
started_at_unix = df_sessions.pop("started_at_unix")
df_sessions.insert(loc=5, column="started_at_unix_time",
value=started_at_unix)

df_sessions["ended_at_unix"] =
(pd.to_datetime(df_sessions["ended_at"]) - pd.Timestamp("1970-01-01"))
// pd.Timedelta("1s")
ended_at_unix = df_sessions.pop("ended_at_unix")
df_sessions.insert(loc=7, column="ended_at_unix_time",
value=ended_at_unix)
display(df_sessions)
df_sessions.describe()

{"columns":[{"name":"index","rawType":"int64","type":"integer"},
{"name":"session_id","rawType":"str","type":"string"},
{"name":"viewer_id","rawType":"str","type":"string"},
{"name":"streamer_id","rawType":"str","type":"string"},
{"name":"stream_id","rawType":"str","type":"string"},
{"name":"started_at","rawType":"str","type":"string"},
{"name":"started_at_unix_time","rawType":"int64","type":"integer"},
{"name":"ended_at","rawType":"str","type":"string"},
{"name":"ended_at_unix_time","rawType":"int64","type":"integer"},
{"name":"duration_mins","rawType":"int64","type":"integer"},
{"name":"chat_messages_sent","rawType":"int64","type":"integer"},
{"name":"bits_cheered","rawType":"int64","type":"integer"},
{"name":"followed_during","rawType":"bool","type":"boolean"},
{"name":"subscribed_during","rawType":"bool","type":"boolean"}],"ref":
"eec3bbae-e4e4-4e24-95b0-956214b93a9e","rows":
[["0","SES000001","VWR00938","STR0055","SM00860","2024-01-18
19:16:00","1705605360","2024-01-18
20:28:00","1705609680","72","0","0","False","False"],
["1","SES000002","VWR00804","STR0034","SM00542","2024-01-04
17:11:00","1704388260","2024-01-04
```

19:41:00","1704397260","150","11","0","False","False"],
["2","SES000003","VWR00704","STR0008","SM00124","2024-02-15
15:11:00","1708009860","2024-02-15
16:53:00","1708015980","102","5","0","False","False"],
["3","SES000004","VWR00926","STR0024","SM00373","2024-01-18
19:59:00","1705607940","2024-01-18
20:56:00","1705611360","57","3","9663","False","False"],
["4","SES000005","VWR00883","STR0008","SM00120","2024-03-19
20:08:00","1710878880","2024-03-19
20:22:00","1710879720","14","7","0","False","False"],
["5","SES000006","VWR00860","STR0056","SM00877","2024-01-07
00:41:00","1704588060","2024-01-07
02:13:00","1704593580","92","13","0","False","False"],
["6","SES000007","VWR00925","STR0052","SM00828","2024-01-29
21:41:00","1706564460","2024-01-30
01:59:00","1706579940","258","13","0","False","False"],
["7","SES000008","VWR00519","STR0047","SM00730","2024-03-13
14:52:00","1710341520","2024-03-13
14:59:00","1710341940","7","0","0","False","False"],
["8","SES000009","VWR00901","STR0045","SM00715","2024-02-17
18:42:00","1708195320","2024-02-17
19:13:00","1708197180","31","1","0","False","False"],
["9","SES000010","VWR00869","STR0016","SM00238","2024-02-03
18:07:00","1706983620","2024-02-03
18:54:00","1706986440","47","5","0","False","False"],
["10","SES000011","VWR00812","STR0033","SM00530","2024-03-17
11:00:00","1710673200","2024-03-17
11:33:00","1710675180","33","1","0","False","False"],
["11","SES000012","VWR00424","STR0006","SM00096","2024-03-27
12:29:00","1711542540","2024-03-27
14:37:00","1711550220","128","1","0","False","False"],
["12","SES000013","VWR00839","STR0042","SM00676","2024-01-24
13:53:00","1706104380","2024-01-24
14:05:00","1706105100","12","0","0","False","False"],
["13","SES000014","VWR00633","STR0024","SM00374","2024-03-01
21:30:00","1709328600","2024-03-01
22:57:00","1709333820","87","3","3873","False","False"],
["14","SES000015","VWR00925","STR0030","SM00487","2024-01-07
19:38:00","1704656280","2024-01-07
20:20:00","1704658800","42","0","0","False","False"],
["15","SES000016","VWR00601","STR0012","SM00182","2024-03-12
12:37:00","1710247020","2024-03-12
13:03:00","1710248580","26","2","0","False","False"],
["16","SES000017","VWR00928","STR0020","SM00313","2024-03-20
02:48:00","1710902880","2024-03-20
03:43:00","1710906180","55","7","0","False","False"],
["17","SES000018","VWR00067","STR0061","SM00950","2024-01-05
00:02:00","1704412920","2024-01-05
02:29:00","1704421740","147","8","0","True","False"],

["18", "SES000019", "VWR00992", "STR0057", "SM00894", "2024-01-11
21:31:00", "1705008660", "2024-01-11
23:46:00", "1705016760", "135", "6", "3985", "False", "False"],
["19", "SES000020", "VWR00726", "STR0074", "SM01179", "2024-01-31
13:02:00", "1706706120", "2024-01-31
14:07:00", "1706710020", "65", "4", "0", "False", "False"],
["20", "SES000021", "VWR00994", "STR0051", "SM00792", "2024-02-19
12:10:00", "1708344600", "2024-02-19
12:46:00", "1708346760", "36", "5", "4638", "False", "False"],
["21", "SES000022", "VWR00846", "STR0075", "SM01182", "2024-03-20
18:00:00", "1710957600", "2024-03-20
18:51:00", "1710960660", "51", "7", "0", "False", "False"],
["22", "SES000023", "VWR00722", "STR0064", "SM01010", "2024-03-09
12:11:00", "1709986260", "2024-03-09
13:17:00", "1709990220", "66", "4", "0", "False", "False"],
["23", "SES000024", "VWR00979", "STR0070", "SM01108", "2024-02-28
20:44:00", "1709153040", "2024-02-28
20:49:00", "1709153340", "5", "1", "0", "False", "False"],
["24", "SES000025", "VWR00616", "STR0024", "SM00367", "2024-03-30
16:15:00", "1711815300", "2024-03-30
16:29:00", "1711816140", "14", "1", "0", "False", "False"],
["25", "SES000026", "VWR00387", "STR0065", "SM01021", "2024-03-11
16:06:00", "1710173160", "2024-03-11
18:50:00", "1710183000", "164", "13", "0", "False", "False"],
["26", "SES000027", "VWR00959", "STR0060", "SM00937", "2024-01-07
20:20:00", "1704658800", "2024-01-07
21:57:00", "1704664620", "97", "0", "6251", "False", "False"],
["27", "SES000028", "VWR00852", "STR0066", "SM01043", "2024-03-25
16:41:00", "1711384860", "2024-03-25
16:49:00", "1711385340", "8", "0", "0", "False", "False"],
["28", "SES000029", "VWR00682", "STR0005", "SM00080", "2024-03-21
22:10:00", "1711059000", "2024-03-21
22:27:00", "1711060020", "17", "2", "0", "False", "False"],
["29", "SES000030", "VWR00217", "STR0021", "SM00327", "2024-01-31
14:06:00", "1706709960", "2024-01-31
14:56:00", "1706712960", "50", "2", "0", "False", "False"],
["30", "SES000031", "VWR00455", "STR0054", "SM00848", "2024-01-23
18:17:00", "1706033820", "2024-01-23
21:33:00", "1706045580", "196", "1", "0", "False", "False"],
["31", "SES000032", "VWR00628", "STR0067", "SM01056", "2024-02-08
09:05:00", "1707383100", "2024-02-08
11:35:00", "1707392100", "150", "33", "3589", "False", "False"],
["32", "SES000033", "VWR00624", "STR0006", "SM00100", "2024-01-08
21:50:00", "1704750600", "2024-01-08
22:13:00", "1704751980", "23", "5", "0", "False", "False"],
["33", "SES000034", "VWR00646", "STR0071", "SM01132", "2024-02-18
23:42:00", "1708299720", "2024-02-19
00:58:00", "1708304280", "76", "4", "412", "True", "False"],
["34", "SES000035", "VWR00878", "STR0023", "SM00358", "2024-03-16

```
11:47:00","1710589620","2024-03-16
11:55:00","1710590100","8","0","0","False","False"],
["35","SES000036","VWR00928","STR0055","SM00864","2024-03-10
08:33:00","1710059580","2024-03-10
08:46:00","1710060360","13","0","0","False","False"],
["36","SES000037","VWR00382","STR0018","SM00271","2024-01-02
21:05:00","1704229500","2024-01-02
23:08:00","1704236880","123","0","0","False","False"],
["37","SES000038","VWR00980","STR0060","SM00936","2024-02-28
09:47:00","1709113620","2024-02-28
10:52:00","1709117520","65","7","0","False","False"],
["38","SES000039","VWR00772","STR0026","SM00412","2024-01-31
14:11:00","1706710260","2024-01-31
15:06:00","1706713560","55","2","0","False","False"],
["39","SES000040","VWR00890","STR0072","SM01137","2024-02-20
18:21:00","1708453260","2024-02-20
20:01:00","1708459260","100","12","0","False","False"],
["40","SES000041","VWR00426","STR0049","SM00766","2024-03-07
14:54:00","1709823240","2024-03-07
15:31:00","1709825460","37","7","0","False","False"],
["41","SES000042","VWR00259","STR0024","SM00368","2024-02-19
13:44:00","1708350240","2024-02-19
16:10:00","1708359000","146","3","0","False","False"],
["42","SES000043","VWR00927","STR0003","SM00047","2024-02-17
12:49:00","1708174140","2024-02-17
13:27:00","1708176420","38","0","0","False","False"],
["43","SES000044","VWR00969","STR0006","SM00098","2024-03-14
20:36:00","1710448560","2024-03-14
20:46:00","1710449160","10","0","0","False","False"],
["44","SES000045","VWR00978","STR0049","SM00759","2024-01-05
23:16:00","1704496560","2024-01-06
03:12:00","1704510720","236","47","0","False","False"],
["45","SES000046","VWR00665","STR0004","SM00069","2024-03-07
08:37:00","1709800620","2024-03-07
10:09:00","1709806140","92","15","0","False","False"],
["46","SES000047","VWR00765","STR0055","SM00859","2024-03-23
12:27:00","1711196820","2024-03-23
13:05:00","1711199100","38","1","0","False","False"],
["47","SES000048","VWR00865","STR0044","SM00695","2024-02-09
16:56:00","1707497760","2024-02-09
17:26:00","1707499560","30","1","0","False","False"],
["48","SES000049","VWR00872","STR0044","SM00701","2024-03-08
12:59:00","1709902740","2024-03-08
14:29:00","1709908140","90","9","254","False","False"],
["49","SES000050","VWR00949","STR0030","SM00475","2024-01-11
15:37:00","1704987420","2024-01-11
19:24:00","1705001040","227","7","0","False","False"]], "shape":
{"columns":13,"rows":22825}}
```

```
{
  "columns": [
    {"name": "index", "rawType": "str", "type": "string"},
    {"name": "started_at_unix_time", "rawType": "float64", "type": "float"},
    {"name": "ended_at_unix_time", "rawType": "float64", "type": "float"},
    {"name": "duration_mins", "rawType": "float64", "type": "float"},
    {"name": "chat_messages_sent", "rawType": "float64", "type": "float"},
    {"name": "bits_cheered", "rawType": "float64", "type": "float"}
  ],
  "ref": "239dbf54-4bd4-4008-97bd-236698226ae8",
  "rows": [
    ["count", "22825.0", "22825.0", "22825.0", "22825.0", "22825.0"],
    ["mean", "1707894046.2361445", "1707897581.457174", "59.05130339539978", "6.4426286966046", "398.42532311062433"],
    ["std", "2247282.9710697485", "2247340.278517896", "60.38397876511292", "46.57630913705245", "1571.5286582612994"],
    ["min", "1704097200.0", "1704097440.0", "0.0", "0.0", "0.0"],
    ["25%", "1706015880.0", "1706019600.0", "15.0", "0.0", "0.0"],
    ["50%", "1707835680.0", "1707839820.0", "39.0", "1.0", "0.0"],
    ["75%", "1709842140.0", "1709846580.0", "83.0", "5.0", "0.0"],
    ["max", "1711848120.0", "1711850040.0", "443.0", "1987.0", "10000.0"]
  ],
  "shape": {"columns": 5, "rows": 8}
}
```

Exploring streamers.csv

```
# TASK 1b: STREAMERS.CSV
df_streamers = pd.read_csv("data/streamers.csv")
display(df_streamers)
display(f"The shape of the dataframe is: {df_streamers.shape}")
columns_streamers = [*df_streamers.columns] # this stringifies the
items, sep in the list, separating them with a comma
(https://www.geeksforgeeks.org/python/print-lists-in-python-4-different-ways/)
print(f"Columns are: {columns_streamers}")

print("\nGiven below are descriptive stats on the numerical columns\n")
df_streamers.describe()

{
  "columns": [
    {"name": "index", "rawType": "int64", "type": "integer"},
    {"name": "streamer_id", "rawType": "str", "type": "string"},
    {"name": "streamer_name", "rawType": "str", "type": "string"},
    {"name": "category", "rawType": "str", "type": "string"},
    {"name": "language", "rawType": "str", "type": "string"},
    {"name": "partner_status", "rawType": "bool", "type": "boolean"},
    {"name": "total_followers", "rawType": "int64", "type": "integer"},
    {"name": "avg_concurrent_viewers", "rawType": "int64", "type": "integer"}
  ],
  "ref": "8916aa22-349e-456b-ab8e-d2d723c96a22",
  "rows": [
    ["0", "STR0001", "xQc", "Just Chatting", "English", "True", "11500000", "138706"],
    ["1", "STR0002", "Pokimane", "Just Chatting", "English", "True", "25551", "71660"],
    ["2", "STR0003", "HasanAbi", "Just Chatting", "English", "True", "2600000", "34220"]
  ]
}
```

```
[ "3", "STR0004", "Amouranth", "Just  
Chatting", "English", "True", "6200000", "68607"],  
[ "4", "STR0005", "Mizkif", "Just  
Chatting", "English", "True", "2100000", "6771"],  
[ "5", "STR0006", "QTCinderella", "Just  
Chatting", "English", "True", "950000", "13948"],  
[ "6", "STR0007", "Devin_Nash", "Just Chatting", "English", "True", "-  
45130", "5114"], [ "7", "STR0008", "Ibai", "Just  
Chatting", "Spanish", "True", "-46964", "156401"],  
[ "8", "STR0009", "Auronplay", "Just  
Chatting", "Spanish", "True", "14500000", "53149"],  
[ "9", "STR0010", "Rubius", "Just  
Chatting", null, "True", "9700000", "76193"],  
[ "10", "STR0011", "ElSpreen", "Just  
Chatting", "Spanish", "True", "8200000", "55927"],  
[ "11", "STR0012", "Hafu", "Just  
Chatting", "English", "True", "1400000", "19667"],  
[ "12", "STR0013", "Fuslie", "Just Chatting", "English", "True", "-  
8902", "9540"], [ "13", "STR0014", "Valkyrae", "Just  
Chatting", "English", "True", "3700000", "46974"],  
[ "14", "STR0015", "DisguisedToast", "Just  
Chatting", "English", "True", "2400000", "18634"],  
[ "15", "STR0016", "Trainwreckstv", "Just  
Chatting", "English", "True", "2800000", "13871"],  
[ "16", "STR0017", "xChocobars", "Just  
Chatting", "English", "False", "540000", "4973"],  
[ "17", "STR0018", "shroud", "FPS", "English", "True", "10300000", "29145"],  
[ "18", "STR0019", "Tfue", "FPS", "English", "True", "11100000", "141627"],  
[ "19", "STR0020", "Nickmercs", "FPS", "English", "True", "6800000", "69439"],  
[ "20", "STR0021", "TimTheTatman", "FPS", null, "True", "7200000", "85357"],  
[ "21", "STR0022", "Aceu", "FPS", "English", "True", "1900000", "12556"],  
[ "22", "STR0023", "Teep", "FPS", "English", "False", "340000", "4970"],  
[ "23", "STR0024", "sImple", "FPS", "English", "True", "3600000", "48998"],  
[ "24", "STR0025", "GeT_RiGhT", "FPS", "English", "True", "-36135", "10785"],  
[ "25", "STR0026", "Symfuhny", "FPS", "English", "True", "3200000", "14496"],  
[ "26", "STR0027", "Cloakzy", "FPS", "English", "True", "2100000", "16941"],  
[ "27", "STR0028", "Bugha", "FPS", "English", "True", "2900000", "7451"],  
[ "28", "STR0029", "Mongraal", "FPS", "English", "True", "2600000", "10414"],  
[ "29", "STR0030", "Tyler1", "MOBA", "English", "True", "5200000", "56574"],  
[ "30", "STR0031", "Doublelift", "MOBA", null, "True", "1700000", "19859"],  
[ "31", "STR0032", "Yassuo", "MOBA", "English", "True", "2300000", "33528"],  
[ "32", "STR0033", "Faker", "MOBA", "Korean", "True", "5800000", "36167"],  
[ "33", "STR0034", "Caps", "MOBA", "English", "True", "980000", "6679"],  
[ "34", "STR0035", "Perkz", "MOBA", "English", "True", "670000", "5429"],  
[ "35", "STR0036", "Caedrel", "MOBA", "English", "True", "1200000", "5355"],  
[ "36", "STR0037", "Jankos", "MOBA", "English", "True", "780000", "2877"],  
[ "37", "STR0038", "Nightblue3", "MOBA", null, "True", "2800000", "22915"],  
[ "38", "STR0039", "Imaqtpie", "MOBA", "English", "True", "2300000", "11384"],  
[ "39", "STR0040", "Voyboy", "MOBA", "English", "False", "670000", "7174"],
```

```
[
  ["40", "STR0041", "Ludwig", "IRL", "English", "True", "4100000", "31500"],
  ["41", "STR0042", "MoistCr1TiKaL", "IRL", null, "True", "9800000", "125683"],
  ["42", "STR0043", "Asmongold", "IRL", "English", "True", "3500000", "38862"],
  ["43", "STR0044", "Sodapoppin", "IRL", "English", "True", "9100000", "55152"],
  ["44", "STR0045", "Nmplol", "IRL", "English", "True", "1600000", "20511"],
  ["45", "STR0046", "Stanz", "IRL", "English", "False", "410000", "5109"],
  ["46", "STR0047", "ConnorEatsPants", "IRL", "English", "False", "380000", "2674"],
  ["47", "STR0048", "Destiny", "IRL", null, "True", "1100000", "6323"],
  ["48", "STR0049", "Jinny", "IRL", "English", "True", "820000", "8915"],
  ["49", "STR0050", "Jerma985", "Creative", "English", "True", "1800000", "6870"]
], "shape": {"columns": 7, "rows": 76}}
```

'The shape of the dataframe is: (76, 7)'

Columns are: ['streamer_id', 'streamer_name', 'category', 'language', 'partner_status', 'total_followers', 'avg_concurrent_viewers']

Given below are descriptive stats on the numerical columns

```
{
  "columns": [
    {"name": "index", "rawType": "str", "type": "string"},
    {"name": "total_followers", "rawType": "float64", "type": "float"},
    {"name": "avg_concurrent_viewers", "rawType": "float64", "type": "float"}
  ],
  "ref": "99483147-5c9b-451b-b41b-3eelf7198ee2",
  "rows": [
    ["count", "76.0", "76.0"],
    ["mean", "3300885.763157895", "31227.986842105263"],
    ["std", "3793772.4791098055", "38307.22365246305"],
    ["min", "46964.0", "634.0"],
    ["25%", "780000.0", "6633.25"],
    ["50%", "2000000.0", "14762.5"],
    ["75%", "3875000.0", "36840.75"],
    ["max", "18700000.0", "156401.0"]
  ],
  "shape": {"columns": 2, "rows": 8}
}
```

Task 1b: Summarizing streamers.csv

Since every file is converted to a dataframe and is being treated like a table, I will often use the term table and file interchangeably (within reason) in this assignment.

Overview

This table contains information about each streamer.

Table shape

- **Rows:** There are 76 rows (75 entries, since one row is for the column headings). Thus it means we have info on 75 streamers.
- **Columns:** There are 7 columns. They are described below.

Columns description

Column Header	Description	Type of data	Comments
streamer_id	Unique identifier for the streamer.	Categorical (nominal)	N/A
streamer_name	The name/display	Categorical (nominal)	N/A

Column Header	Decription	Type of data	Comments
	name of the streamer.		
category	Type of stream/category/genre.	Categorical (nominal)	N/A
language	The language they stream in.	Categorical (nominal)	There are some missing values
partner_status	Whether they are a twitch partner.	Categorical (boolean)	N/A
total_followers	Total number of followers of this streamer.	Numerical (ratio)	Contains some negative values. Those are problematic datapoints
avg_concurrent_viewers	Average number of viewers that watch streams of this streamer at a time.	Numerical (ratio)	N/A

Exploring streams.csv

```
# TASK 1c: STREAMS.CSV
df_streams = pd.read_csv("data/streams.csv")
display(df_streams)
display (f"The shape of the dataframe is: {df_streams.shape}")
columns_streams = [*df_streams.columns]
print(f"Columns are: {columns_streams}")

print("\nGiven below are descriptive stats on the numerical columns\n")
df_streams.describe()

{"columns":[{"name":"index","rawType":"int64","type":"integer"},
{"name":"stream_id","rawType":"str","type":"string"},
{"name":"streamer_id","rawType":"str","type":"string"},
{"name":"started_at","rawType":"str","type":"string"},
{"name":"ended_at","rawType":"str","type":"string"},
{"name":"stream_duration_hrs","rawType":"float64","type":"float"},
{"name":"category","rawType":"str","type":"string"},
{"name":"peak_viewers","rawType":"float64","type":"float"},
{"name":"title_has_hype_word","rawType":"bool","type":"boolean"},
{"name":"was_raid","rawType":"bool","type":"boolean"}], "ref": "716f1638-b02b-41ec-9faa-f1301f36ba85", "rows": [[ "0", "SM00001", "STR0001", "2024-02-14 18:00:00", "2024-02-14 21:00:00", "3.0", "Just Chatting", "108989.0", "False", "True"], [ "1", "SM00002", "STR0001", "2024-01-28 18:00:00", "2024-01-28 21:00:00", "3.0", "Just Chatting", "51635.0", "True", "False"], [ "2", "SM00003", "STR0001", "2024-01-
```

31 09:00:00","2024-01-31 16:00:00","7.0","Just Chatting","212127.0","False","False"],["3","SM00004","STR0001","2024-03-03 14:00:00","2024-03-03 21:00:00","7.0","Just Chatting","186745.0","False","True"],["4","SM00005","STR0001","2024-01-24 18:00:00","2024-01-24 21:00:00","3.0","Just Chatting","87723.0","False","True"],["5","SM00006","STR0001","2024-01-09 14:00:00","2024-01-09 17:00:00","3.0","Just Chatting","74062.0","True","False"],["6","SM00007","STR0001","2024-03-05 12:00:00","2024-03-05 16:00:00","48.0","Just Chatting",null,"True","False"],["7","SM00008","STR0001","2024-03-18 08:00:00","2024-03-18 13:00:00","5.0","Just Chatting","118974.0","False","False"],["8","SM00009","STR0001","2024-01-28 17:00:00","2024-01-28 21:00:00","4.0","Just Chatting","161545.0","True","False"],["9","SM00010","STR0001","2024-01-08 19:00:00","2024-01-08 20:30:00","1.5","Just Chatting","131075.0","False","True"],["10","SM00011","STR0001","2024-01-17 12:00:00","2024-01-17 14:00:00","2.0","Just Chatting","54715.0","False","False"],["11","SM00012","STR0001","2024-02-22 19:00:00","2024-02-23 01:00:00","6.0","Just Chatting","148097.0","False","False"],["12","SM00013","STR0001","2024-02-10 13:00:00","2024-02-10 17:00:00","4.0","Just Chatting","38958.0","False","False"],["13","SM00014","STR0002","2024-02-07 15:00:00","2024-02-07 20:00:00","5.0","Just Chatting","118.0","True","True"],["14","SM00015","STR0002","2024-02-13 18:00:00","2024-02-14 01:00:00","7.0","Just Chatting","324.0","False","False"],["15","SM00016","STR0002","2024-02-18 15:00:00","2024-02-18 17:30:00","2.5","Just Chatting","220.0","False","False"],["16","SM00017","STR0002","2024-02-16 20:00:00","2024-02-16 21:30:00","1.5","Just Chatting","435.0","False","False"],["17","SM00018","STR0002","2024-01-01 21:00:00","2024-01-01 23:00:00","2.0","Just Chatting","317.0","False","False"],["18","SM00019","STR0002","2024-01-10 17:00:00","2024-01-10 19:30:00","2.5","Just Chatting",null,"False","False"],["19","SM00020","STR0002","2024-03-06 18:00:00","2024-03-06 23:00:00","5.0","Just Chatting","203.0","False","False"],["20","SM00021","STR0002","2024-03-23 22:00:00","2024-03-23 23:30:00","1.5",null,"317.0","False","True"],["21","SM00022","STR0002","2024-02-19 13:00:00","2024-02-19 18:00:00","5.0",null,null,"True","False"],["22","SM00023","STR0002","2024-01-29 11:00:00","2024-01-29 13:30:00","2.5","Just Chatting","300.0","True","False"],["23","SM00024","STR0002","2024-03-22 11:00:00","2024-03-22 18:00:00","7.0","Just Chatting","183.0","True","False"],["24","SM00025","STR0002","2024-01-05 12:00:00","2024-01-05 20:00:00","8.0","Just Chatting","463.0","False","False"],["25","SM00026","STR0002","2024-01-09 19:00:00","2024-01-10 01:00:00","6.0","Just Chatting","464.0","False","False"],["26","SM00027","STR0002","2024-02-16 15:00:00","2024-02-16 20:00:00","5.0","Just Chatting","364.0","True","False"],

```
[
  ["27", "SM00028", "STR0002", "2024-02-06 13:00:00", "2024-02-06 15:00:00", "2.0", "Just Chatting", "401.0", "False", "False"],
  ["28", "SM00029", "STR0002", "2024-01-24 08:00:00", "2024-01-24 10:30:00", "2.5", "Just Chatting", "284.0", "True", "False"],
  ["29", "SM00030", "STR0002", "2024-01-07 20:00:00", "2024-01-07 22:30:00", "2.5", null, "154.0", "True", "False"],
  ["30", "SM00031", "STR0002", "2024-02-23 18:00:00", "2024-02-23 20:30:00", "2.5", "Just Chatting", "-19.0", "False", "False"],
  ["31", "SM00032", "STR0002", "2024-03-30 19:00:00", "2024-03-31 02:00:00", "7.0", null, "294.0", "False", "False"],
  ["32", "SM00033", "STR0002", "2024-01-13 22:00:00", "2024-01-14 00:30:00", "2.5", "Just Chatting", "133.0", "False", "False"],
  ["33", "SM00034", "STR0002", "2024-01-02 18:00:00", "2024-01-02 20:00:00", "2.0", "Just Chatting", "296.0", "True", "True"],
  ["34", "SM00035", "STR0003", "2024-03-09 16:00:00", "2024-03-09 18:30:00", "2.5", "Just Chatting", "43342.0", "False", "False"],
  ["35", "SM00036", "STR0003", "2024-03-05 09:00:00", "2024-03-05 11:00:00", "2.0", "Just Chatting", "13269.0", "True", "False"],
  ["36", "SM00037", "STR0003", "2024-02-21 21:00:00", "2024-02-22 00:00:00", "3.0", "Just Chatting", "21101.0", "False", "False"],
  ["37", "SM00038", "STR0003", "2024-02-13 15:00:00", "2024-02-13 22:00:00", "7.0", "Just Chatting", "20460.0", "False", "False"],
  ["38", "SM00039", "STR0003", "2024-03-27 21:00:00", "2024-03-27 22:30:00", "1.5", "Just Chatting", "32339.0", "False", "False"],
  ["39", "SM00040", "STR0003", "2024-02-27 20:00:00", "2024-02-27 22:00:00", "2.0", "Just Chatting", "26324.0", "True", "False"],
  ["40", "SM00041", "STR0003", "2024-01-01 22:00:00", "2024-01-02 02:00:00", "4.0", "Just Chatting", "49039.0", "False", "False"],
  ["41", "SM00042", "STR0003", "2024-03-13 14:00:00", "2024-03-13 19:00:00", "5.0", "Just Chatting", "8588.0", "False", "False"],
  ["42", "SM00043", "STR0003", "2024-02-29 09:00:00", "2024-02-29 14:00:00", "5.0", "Just Chatting", "43019.0", "False", "False"],
  ["43", "SM00044", "STR0003", "2024-01-21 13:00:00", "2024-01-21 17:00:00", "4.0", "Just Chatting", "46154.0", "False", "False"],
  ["44", "SM00045", "STR0003", "2024-03-08 17:00:00", "2024-03-08 20:00:00", "3.0", "Just Chatting", "24338.0", "True", "True"],
  ["45", "SM00046", "STR0003", "2024-02-08 09:00:00", "2024-02-08 15:00:00", "6.0", "Just Chatting", "27862.0", "True", "False"],
  ["46", "SM00047", "STR0003", "2024-02-17 11:00:00", "2024-02-17 13:30:00", "2.5", "Just Chatting", null, "False", "False"],
  ["47", "SM00048", "STR0003", "2024-03-14 22:00:00", "2024-03-15 00:00:00", "2.0", "Just Chatting", "34073.0", "True", "False"],
  ["48", "SM00049", "STR0003", "2024-02-02 10:00:00", "2024-02-02 15:00:00", "72.0", "Just Chatting", "21516.0", "False", "False"],
  ["49", "SM00050", "STR0003", "2024-01-13 21:00:00", "2024-01-13 23:30:00", "2.5", "Just Chatting", null, "False", "False"]], "shape":
{"columns": 9, "rows": 1199}]
```

'The shape of the dataframe is: (1199, 9)'

```
Columns are: ['stream_id', 'streamer_id', 'started_at', 'ended_at', 'stream_duration_hrs', 'category', 'peak_viewers', 'title_has_hype_word', 'was_raid']
```

Given below are descriptive stats on the numerical columns

```
{"columns":[{"name":"index","rawType":"str","type":"string"}, {"name":"stream_duration_hrs","rawType":"float64","type":"float"}, {"name":"peak_viewers","rawType":"float64","type":"float"}],"ref":"56aace4d-7d70-467d-b10a-1bbf3b3609de","rows": [{"count","1199.0","1119.0"}, {"mean","4.62510425354462","38375.658623771225"}, {"std","7.995028419838813","50377.085405554106"}, {"min","1.5","-50.0"}, {"25%","2.5","7598.5"}, {"50%","3.0","20224.0"}, {"75%","5.0","47941.0"}, {"max","96.0","370165.0"}],"shape":{"columns":2,"rows":8}}
```

Task 1c: Summarizing streams.csv

Since every file is converted to a dataframe and is being treated like a table, I will often use the term table and file interchangeably (within reason) in this assignment.

Overview

This table contains information about each stream.

Table shape

- **Rows:** There are 1199 rows (1198 entries, since one row is for the column headings). Thus it means we have info on 1198 streams.
- **Columns:** There are 9 columns. They are described below.

Columns description

Column Header	Decription	Type of data	Comments
<code>stream_id</code>	Unique identifier for the stream.	Categorical (nominal)	N/A
<code>streamer_id</code>	The unique identifier of the streamer.	Cateogrical (nominal)	N/A
<code>started_at</code>	Time the stream started at.	Numerical (ratio)	Will convert to unix timestamp for calculation
<code>ended_at</code>	Time the stream ended	Numerical (ratio)	Will convert to unix timestamp for calculation
<code>stream_duration_hrs</code>	Duration of stream in hours	Numerical (ratio)	N/A
<code>category</code>	Category of stream	Categorical (nominal)	Has missing

Column Header	Decription	Type of data	Comments
peak_viewers	Peak viewers during this stream	Numerical (ratio)	Has negative and missing values
title_has_hype_word	Whether the title has a hype word	Categorical (boolean)	N/A
was_raided	Was it a raid from another stream?	Categorical (boolean)	N/A

Below, I wil convert the times to unix timestamp

```
# Task 1c: Continued, creating new columns with unix_timestamp
df_streams["started_at_unix"] =
(pd.to_datetime(df_streams["started_at"]) - pd.Timestamp("1970-01-01")) // pd.Timedelta("1s")
started_at_unix_streams = df_streams.pop("started_at_unix")
df_streams.insert(loc=3, column="started_at_unix_time",
value=started_at_unix_streams)

df_streams["ended_at_unix"] = (pd.to_datetime(df_streams["ended_at"])
- pd.Timestamp("1970-01-01")) // pd.Timedelta("1s")
ended_at_unix_streams = df_streams.pop("ended_at_unix")
df_streams.insert(loc=5, column="ended_at_unix_time",
value=ended_at_unix)
display(df_streams)
df_streams.describe()

{"columns":[{"name":"index","rawType":"int64","type":"integer"},
{"name":"stream_id","rawType":"str","type":"string"},
{"name":"streamer_id","rawType":"str","type":"string"},
{"name":"started_at","rawType":"str","type":"string"},
{"name":"started_at_unix_time","rawType":"int64","type":"integer"},
{"name":"ended_at","rawType":"str","type":"string"},
{"name":"ended_at_unix_time","rawType":"int64","type":"integer"},
{"name":"stream_duration_hrs","rawType":"float64","type":"float"},
{"name":"category","rawType":"str","type":"string"},
{"name":"peak_viewers","rawType":"float64","type":"float"},
{"name":"title_has_hype_word","rawType":"bool","type":"boolean"},
{"name":"was_raided","rawType":"bool","type":"boolean"}],"ref":"216b29ba-73c1-44fd-870f-f872e9d0c97b","rows":[["0","SM00001","STR0001","2024-02-14 18:00:00","1707933600","2024-02-14 21:00:00","1705609680","3.0","Just Chatting","108989.0","False","True"],["1","SM00002","STR0001","2024-01-28 18:00:00","1706464800","2024-01-28 21:00:00","1704397260","3.0","Just Chatting","51635.0","True","False"],["2","SM00003","STR0001","2024-01-31 09:00:00","1706691600","2024-01-31 12:00:00","1706691600","3.0","Just Chatting","51635.0","True","False"]]
```

16:00:00","1708015980","7.0","Just Chatting","212127.0","False","False"],["3","SM00004","STR0001","2024-03-03 14:00:00","1709474400","2024-03-03 21:00:00","1705611360","7.0","Just Chatting","186745.0","False","True"],["4","SM00005","STR0001","2024-01-24 18:00:00","1706119200","2024-01-24 21:00:00","1710879720","3.0","Just Chatting","87723.0","False","True"],["5","SM00006","STR0001","2024-01-09 14:00:00","1704808800","2024-01-09 17:00:00","1704593580","3.0","Just Chatting","74062.0","True","False"],["6","SM00007","STR0001","2024-03-05 12:00:00","1709640000","2024-03-05 16:00:00","1706579940","48.0","Just Chatting",null,"True","False"],["7","SM00008","STR0001","2024-03-18 08:00:00","1710748800","2024-03-18 13:00:00","1710341940","5.0","Just Chatting","118974.0","False","False"],["8","SM00009","STR0001","2024-01-28 17:00:00","1706461200","2024-01-28 21:00:00","1708197180","4.0","Just Chatting","161545.0","True","False"],["9","SM00010","STR0001","2024-01-08 19:00:00","1704740400","2024-01-08 20:30:00","1706986440","1.5","Just Chatting","131075.0","False","True"],["10","SM00011","STR0001","2024-01-17 12:00:00","1705492800","2024-01-17 14:00:00","1710675180","2.0","Just Chatting","54715.0","False","False"],["11","SM00012","STR0001","2024-02-22 19:00:00","1708628400","2024-02-23 01:00:00","1711550220","6.0","Just Chatting","148097.0","False","False"],["12","SM00013","STR0001","2024-02-10 13:00:00","1707570000","2024-02-10 17:00:00","1706105100","4.0","Just Chatting","38958.0","False","False"],["13","SM00014","STR0002","2024-02-07 15:00:00","1707318000","2024-02-07 20:00:00","1709333820","5.0","Just Chatting","118.0","True","True"],["14","SM00015","STR0002","2024-02-13 18:00:00","1707847200","2024-02-14 01:00:00","1704658800","7.0","Just Chatting","324.0","False","False"],["15","SM00016","STR0002","2024-02-18 15:00:00","1708268400","2024-02-18 17:30:00","1710248580","2.5","Just Chatting","220.0","False","False"],["16","SM00017","STR0002","2024-02-16 20:00:00","1708113600","2024-02-16 21:30:00","1710906180","1.5","Just Chatting","435.0","False","False"],["17","SM00018","STR0002","2024-01-01 21:00:00","1704142800","2024-01-01 23:00:00","1704421740","2.0","Just Chatting","317.0","False","False"],["18","SM00019","STR0002","2024-01-10 17:00:00","1704906000","2024-01-10 19:30:00","1705016760","2.5","Just Chatting",null,"False","False"],["19","SM00020","STR0002","2024-03-06 18:00:00","1709748000","2024-03-06 23:00:00","1706710020","5.0","Just Chatting","203.0","False","False"],["20","SM00021","STR0002","2024-03-23 22:00:00","1711231200","2024-03-23

23:30:00", "1708346760", "1.5", null, "317.0", "False", "True"],
["21", "SM00022", "STR0002", "2024-02-19 13:00:00", "1708347600", "2024-02-19 18:00:00", "1710960660", "5.0", null, null, "True", "False"],
["22", "SM00023", "STR0002", "2024-01-29 11:00:00", "1706526000", "2024-01-29 13:30:00", "1709990220", "2.5", "Just Chatting", "300.0", "True", "False"], ["23", "SM00024", "STR0002", "2024-03-22 11:00:00", "1711105200", "2024-03-22 18:00:00", "1709153340", "7.0", "Just Chatting", "183.0", "True", "False"],
["24", "SM00025", "STR0002", "2024-01-05 12:00:00", "1704456000", "2024-01-05 20:00:00", "1711816140", "8.0", "Just Chatting", "463.0", "False", "False"], ["25", "SM00026", "STR0002", "2024-01-09 19:00:00", "1704826800", "2024-01-10 01:00:00", "1710183000", "6.0", "Just Chatting", "464.0", "False", "False"],
["26", "SM00027", "STR0002", "2024-02-16 15:00:00", "1708095600", "2024-02-16 20:00:00", "1704664620", "5.0", "Just Chatting", "364.0", "True", "False"], ["27", "SM00028", "STR0002", "2024-02-06 13:00:00", "1707224400", "2024-02-06 15:00:00", "1711385340", "2.0", "Just Chatting", "401.0", "False", "False"],
["28", "SM00029", "STR0002", "2024-01-24 08:00:00", "1706083200", "2024-01-24 10:30:00", "1711060020", "2.5", "Just Chatting", "284.0", "True", "False"], ["29", "SM00030", "STR0002", "2024-01-07 20:00:00", "1704657600", "2024-01-07 22:30:00", "1706712960", "2.5", null, "154.0", "True", "False"],
["30", "SM00031", "STR0002", "2024-02-23 18:00:00", "1708711200", "2024-02-23 20:30:00", "1706045580", "2.5", "Just Chatting", "-19.0", "False", "False"], ["31", "SM00032", "STR0002", "2024-03-30 19:00:00", "1711825200", "2024-03-31 02:00:00", "1707392100", "7.0", null, "294.0", "False", "False"],
["32", "SM00033", "STR0002", "2024-01-13 22:00:00", "1705183200", "2024-01-14 00:30:00", "1704751980", "2.5", "Just Chatting", "133.0", "False", "False"], ["33", "SM00034", "STR0002", "2024-01-02 18:00:00", "1704218400", "2024-01-02 20:00:00", "1708304280", "2.0", "Just Chatting", "296.0", "True", "True"],
["34", "SM00035", "STR0003", "2024-03-09 16:00:00", "1710000000", "2024-03-09 18:30:00", "1710590100", "2.5", "Just Chatting", "43342.0", "False", "False"], ["35", "SM00036", "STR0003", "2024-03-05 09:00:00", "1709629200", "2024-03-05 11:00:00", "1710060360", "2.0", "Just Chatting", "13269.0", "True", "False"], ["36", "SM00037", "STR0003", "2024-02-21 21:00:00", "1708549200", "2024-02-22 00:00:00", "1704236880", "3.0", "Just Chatting", "21101.0", "False", "False"], ["37", "SM00038", "STR0003", "2024-02-13 15:00:00", "1707836400", "2024-02-13 22:00:00", "1709117520", "7.0", "Just Chatting", "20460.0", "False", "False"], ["38", "SM00039", "STR0003", "2024-03-27 21:00:00", "1711573200", "2024-03-27 22:30:00", "1706713560", "1.5", "Just Chatting", "32339.0", "False", "False"], ["39", "SM00040", "STR0003", "2024-02-27 20:00:00", "1709064000", "2024-02-27

```

22:00:00","1708459260","2.0","Just
Chatting","26324.0","True","False"],["40","SM00041","STR0003","2024-
01-01 22:00:00","1704146400","2024-01-02
02:00:00","1709825460","4.0","Just
Chatting","49039.0","False","False"],["41","SM00042","STR0003","2024-
03-13 14:00:00","1710338400","2024-03-13
19:00:00","1708359000","5.0","Just
Chatting","8588.0","False","False"],["42","SM00043","STR0003","2024-
02-29 09:00:00","1709197200","2024-02-29
14:00:00","1708176420","5.0","Just
Chatting","43019.0","False","False"],["43","SM00044","STR0003","2024-
01-21 13:00:00","1705842000","2024-01-21
17:00:00","1710449160","4.0","Just
Chatting","46154.0","False","False"],["44","SM00045","STR0003","2024-
03-08 17:00:00","1709917200","2024-03-08
20:00:00","1704510720","3.0","Just Chatting","24338.0","True","True"],
["45","SM00046","STR0003","2024-02-08 09:00:00","1707382800","2024-02-
08 15:00:00","1709806140","6.0","Just
Chatting","27862.0","True","False"],["46","SM00047","STR0003","2024-
02-17 11:00:00","1708167600","2024-02-17
13:30:00","1711199100","2.5","Just Chatting",null,"False","False"],
["47","SM00048","STR0003","2024-03-14 22:00:00","1710453600","2024-03-
15 00:00:00","1707499560","2.0","Just
Chatting","34073.0","True","False"],["48","SM00049","STR0003","2024-
02-02 10:00:00","1706868000","2024-02-02
15:00:00","1709908140","72.0","Just
Chatting","21516.0","False","False"],["49","SM00050","STR0003","2024-
01-13 21:00:00","1705179600","2024-01-13
23:30:00","1705001040","2.5","Just
Chatting",null,"False","False"]], "shape":{"columns":11,"rows":1199}}

{"columns":[{"name":"index","rawType":"str","type":"string"},
{"name":"started_at_unix_time","rawType":"float64","type":"float"},
{"name":"ended_at_unix_time","rawType":"float64","type":"float"},
{"name":"stream_duration_hrs","rawType":"float64","type":"float"},
{"name":"peak_viewers","rawType":"float64","type":"float"}],"ref":"5a6
83d0e-372a-4741-8b1b-bb83bec164e9","rows":
[["count","1199.0","1199.0","1199.0","1119.0"],
["mean","1707898713.9282737","1707936543.9532945","4.62510425354462","
38375.658623771225"],
["std","2230573.6973919943","2228837.029603017","7.995028419838813","5
0377.085405554106"],["min","1704096000.0","1704104400.0","1.5","-
50.0"],["25%","1706034600.0","1706031900.0","2.5","7598.5"],
["50%","1707843600.0","1708015980.0","3.0","20224.0"],
["75%","1709825400.0","1709907030.0","5.0","47941.0"],
["max","1711832400.0","1711842660.0","96.0","370165.0"]],"shape":
{"columns":4,"rows":8}}

```

Exploring viewers.csv

```
# TASK 1d: VIEWERS.CSV
df_viewers = pd.read_csv("data/viewers.csv")
display(df_viewers)
display(f"The shape of the dataframe is: {df_viewers.shape}")
columns_viewers = [*df_viewers.columns]
print(f"Columns are: {columns_viewers}")

print("\nGiven below are descriptive stats on the numerical columns\n")
df_viewers.describe()

{"columns":[{"name":"index","rawType":"int64","type":"integer"},
{"name":"viewer_id","rawType":"str","type":"string"},
{"name":"age_group","rawType":"str","type":"string"},
{"name":"country","rawType":"str","type":"string"},
{"name":"account_age_days","rawType":"int64","type":"integer"},
{"name":"subscription_tier","rawType":"str","type":"string"},
{"name":"preferred_category","rawType":"str","type":"string"}],"ref":"
962e3e24-3bf3-40c6-bdda-8b6a994e9a52","rows":[["0","VWR00001","18-
24","AR","355","free","IRL"],["1","VWR00002","25-
34","CA","1670","free","Creative"],["2","VWR00003","35-
44","US","1108","free","Just Chatting"],
["3","VWR00004","45+","null","822","tier1","FPS"],["4","VWR00005","25-
34","DE","2226","free","Just Chatting"],["5","VWR00006","18-
24","US","1766","free","Just Chatting"],["6","VWR00007","13-
17","ES","1367","tier1","Sports"],["7","VWR00008","18-
24","other","2720","free","MOBA"],
["8","VWR00009","45+","FR","33","tier2","FPS"],
["9","VWR00010","45+","GB","363","free","FPS"],["10","VWR00011","18-
24","GB","3511","tier1","Sports"],["11","VWR00012","18-
24","US","2431","tier3","FPS"],["12","VWR00013","25-
34","US","2019","free","IRL"],["13","VWR00014","18-
24","FR","83","tier1","FPS"],
["14","VWR00015","45+","null","2196","free","Just Chatting"],
["15","VWR00016","18-24","FR","1603","tier1","IRL"],
["16","VWR00017","18-24","SE","2201","tier2","Just Chatting"],
["17","VWR00018","18-24","US","178","tier1","IRL"],
["18","VWR00019","18-24","US","2023","tier2","FPS"],
["19","VWR00020","18-24","US","2121","free","IRL"],
["20","VWR00021","35-44","null","1079","free","Creative"],
["21","VWR00022","18-24","BR","227","free","IRL"],
["22","VWR00023","25-34","IT","3547","tier1","Just Chatting"],
["23","VWR00024","18-24","RU","2597","tier2","IRL"],
["24","VWR00025","18-24","MX","2280","free trial","Sports"],
["25","VWR00026","13-17","GB","2806","free","Just Chatting"],
["26","VWR00027","18-24","US","2458","free","FPS"],
["27","VWR00028","13-17","GB","516","tier1","Creative"],
["28","VWR00029","18-24","CA","1676","tier2","Just Chatting"]]
```

```
[ "29", "VWR00030", "35-44", "PL", "3246", "free", "IRL"],
[ "30", "VWR00031", "25-34", "DE", "1072", "free", "Creative"],
[ "31", "VWR00032", "45+", "US", "3549", "free", "Just Chatting"],
[ "32", "VWR00033", "18-24", null, "262", "free", "MOBA"],
[ "33", "VWR00034", "25-34", "US", "1227", "tier3", "MOBA"],
[ "34", "VWR00035", "25-34", "AR", "1297", "free", "FPS"],
[ "35", "VWR00036", "18-24", "US", "2265", "free", "Creative"],
[ "36", "VWR00037", "18-24", "US", "667", "free", "Just Chatting"],
[ "37", "VWR00038", "25-34", "US", "859", "free", "FPS"],
[ "38", "VWR00039", "35-44", "NL", "2426", "tier3", "Creative"],
[ "39", "VWR00040", "25-34", "FR", "1060", "free", "Just Chatting"],
[ "40", "VWR00041", "18-24", "MX", "3211", "free", "Creative"],
[ "41", "VWR00042", "45+", null, "3180", "free", "Just Chatting"],
[ "42", "VWR00043", "18-24", "US", "1125", "tier3", "MOBA"],
[ "43", "VWR00044", "45+", "SE", "2057", "tier3", "Just Chatting"],
[ "44", "VWR00045", "35-44", "US", "1235", "free", "IRL"],
[ "45", "VWR00046", "35-44", "US", "1278", "premium", "Creative"],
[ "46", "VWR00047", "35-44", "BR", "2456", "free", "Creative"],
[ "47", "VWR00048", "18-24", "US", "1091", "free", "Sports"],
[ "48", "VWR00049", "25-34", "US", "570", "free", "IRL"],
[ "49", "VWR00050", "35-44", "IT", "967", "tier1", "Creative"]], "shape":
{"columns": 6, "rows": 1000}}
```

'The shape of the dataframe is: (1000, 6)'

Columns are: ['viewer_id', 'age_group', 'country', 'account_age_days', 'subscription_tier', 'preferred_category']

Given below are descriptive stats on the numerical columns

```
{"columns": [{"name": "index", "rawType": "str", "type": "string"},
{"name": "account_age_days", "rawType": "float64", "type": "float"}], "ref":
"9e8b9453-e4a3-4745-9f01-affd4c9a32b9", "rows": [{"count", "1000.0"},
["mean", "1881.119"], ["std", "1036.1446562076105"], ["min", "33.0"],
["25%", "984.5"], ["50%", "1966.5"], ["75%", "2761.25"],
["max", "3645.0"]], "shape": {"columns": 1, "rows": 8}}
```

Task 1d: Summarizing viewers.csv

Since every file is converted to a dataframe and is being treated like a table, I will often use the term table and file interchangeably (within reason) in this assignment.

Overview

This table contains information about each viewer

Table shape

- **Rows:** There are 1000 rows (999 entries, since one row is for the column headings). Thus it means we have info on 999 streams.

- **Columns:** There are 6 columns. They are described below.

Columns description

Column Header	Decription	Type of data	Comments
<code>viewer_id</code>	Unique identifier for the viewer.	Categorical (nominal)	N/A
<code>age_group</code>	The age category for the viewer.	Cateogrical (nominal)	N/A
<code>country</code>	The country the viewer is from in alpha-2 format.	Categorical (nominal)	Has missing values and "other"
<code>account_age_days</code>	The age of the account in days.	Numerical (ratio)	N/A
<code>subscription_tier</code>	What tier of subscription they have	Categorical (ordinal)	Includes none, and some are in CAPS
<code>preferred_category</code>	Their preferred category of streams.	Categorical (nominal)	N/A

Task 1 Summary Report

In the cells above, I explored the four csv files: sessions, streamers, streams, and viewers. Specifically, I investigated (and displayed) the shape, the columns, and descriptive statistics on the numerical data points. I also described each column (what it shows, what type of data it is, and added comments about if there are missing or strange values). For start and end time of streams, I converted them to Unix timestamp for ease of calculations in the future. All of this can be seen above in detail.

`sessions.csv` contains data on individual viewing sessions. The viewing session is when a viewer (identified by viewer id), watches a stream (identified by a stream id) from a streamer (identiified by a streamer id). The ids for viewer, stream, streamer, are consistent across the tables and can be used to link and aggregate data. In this file, I also created a new column which has the start and end times of the session in unix timestamps.

`streamers.csv` contains information on streamers. This table has a few issues. The `language` column has some missing values, and there are negative values in the `follower_count` column. The `streamer_id` can be used to link to other tables.

`streams.csv` contains information of streams. This is different from a session because a single stream can have multiple sessions because a session is a combination of viewer x streamer x stream. The category column has some missing values and the peak viewerrs has missing and negative values. This table also contains `streamer_id` and `stream_id` which we can use to link to `sessions.csv`

`viewers.csv` contains information on viewers. The `viewer_id` can be used to link to other tables. The `country` column has missing values and "other". The `subscription_tier` includes "none" and some subscription values written in caps.

Details of the problematic data points can be found in Task 2

Task 2: Data Cleaning, Diagnostics, and Corrections

Identify and resolve issues in the dataset. These can be of different kinds, such as:

- Look for duplicates or implausible values (e.g., users with negative followers, invalid age groups). Thoroughly document your exploration of the data. Decide whether these are outliers or the result of a systemic problem. If the latter, resolve the problem here; if they are outliers, we will address those later.
- Define at least three heuristics (common-sense rules) to flag unusual entries (e.g., someone watching dozens of streams per day). Discuss whether what you found are errors or artifacts.

I will repeat the instructions here, but this is expected for each task: each code cell should focus on a specific analysis. You should run the code, and in the markdown below each code cell and its result, provide a detailed interpretation of what you have uncovered in that step. After completing the analyses, summarize the key findings in the indicated summary report cell below.

`sessions.csv`

Observing for implausible values in `started_at`

```
# Checking for min/max of SESSION_START timestamp fields for anomalies
date_start = pd.to_datetime(df_sessions["started_at"]).dt
print(f"In sessions.csv - START, earliest year was {date_start.year.min()} and latest year was {date_start.year.max()}")
print(f"In sessions.csv - START, the earliest month was {date_start.month.min()} and latest month was {date_start.month.max()}")
print(f"In sessions.csv - START, the earliest month was {date_start.month.min()} and latest month was {date_start.month.max()}")
print(f"In sessions.csv - START, the earliest day was {date_start.day.min()} and latest day was {date_start.day.max()}")
print(f"In sessions.csv - START, the earliest hour was {date_start.hour.min()} and latest hour was {date_start.hour.max()}")
print(f"In sessions.csv - START, the earliest minute was {date_start.minute.min()} and latest minute was {date_start.minute.max()}")
```

In sessions.csv - START, earliest year was 2024 and latest year was 2024

In sessions.csv - START, the earliest month was 1 and latest month was 3

```
In sessions.csv - START, the earliest month was 1 and latest month was 3
In sessions.csv - START, the earliest day was 1 and latest day was 31
In sessions.csv - START, the earliest hour was 0 and latest hour was 23
In sessions.csv - START, the earliest minute was 0 and latest minute was 59
```

Observing for implausible values in ended_at

```
# Checking for min/max of SESSION_END timestamp fields for anomalies
date_end = pd.to_datetime(df_sessions["ended_at"]).dt
print(f"In sessions.csv - END, earliest year was {date_end.year.min()} and latest year was {date_end.year.max()}")
print(f"In sessions.csv - END, the earliest month was {date_end.month.min()} and latest month was {date_end.month.max()}")
print(f"In sessions.csv - END, the earliest month was {date_end.month.min()} and latest month was {date_end.month.max()}")
print(f"In sessions.csv - END, the earliest day was {date_end.day.min()} and latest day was {date_end.day.max()}")
print(f"In sessions.csv - END, the earliest hour was {date_end.hour.min()} and latest hour was {date_end.hour.max()}")
print(f"In sessions.csv - END, the earliest minute was {date_end.minute.min()} and latest minute was {date_end.minute.max()}")

In sessions.csv - END, earliest year was 2024 and latest year was 2024
In sessions.csv - END, the earliest month was 1 and latest month was 3
In sessions.csv - END, the earliest month was 1 and latest month was 3
In sessions.csv - END, the earliest day was 1 and latest day was 31
In sessions.csv - END, the earliest hour was 0 and latest hour was 23
In sessions.csv - END, the earliest minute was 0 and latest minute was 59
```

streamers.csv

Observing for implausible values in languages

```
unique_languages = df_streamers["language"].values.unique()
print(f"The languages of the streamers are {unique_languages}")

The languages of the streamers are <StringArray>
['English', 'Spanish', nan, 'Korean', 'Portuguese']
Length: 5, dtype: str

missing_language = (df_streamers["language"].isnull()).sum()
print(f"Number of missing values in languages in df_streamers is {missing_language}")

Number of missing values in languages in df_streamers is 8
```

Observing for implausible values in partner_status

```
unique_partner_status =  
np.unique(df_streamers["partner_status"].values)  
print(f"The partner statuses of the streamers are  
{unique_partner_status}")
```

The partner statuses of the streamers are [False True]

Observing for implausible values in categories

```
unique_categories = df_streamers["category"].values.unique()  
print(f"The unique categories are {unique_categories}")
```

The unique categories are <StringArray>
['Just Chatting', 'FPS', 'MOBA', 'IRL', 'Creative', 'Sports']
Length: 6, dtype: str

Observing for implausible values in total_followers

```
unique_total_followers =  
np.unique(df_streamers["total_followers"].values)  
print(f"Unique values for total followers are  
{unique_total_followers}")
```

Unique values for total followers are [-46964 -45130 -36135 -
25551 -8902 120000 230000 290000
310000 340000 380000 410000 450000 540000 670000
720000
760000 780000 820000 890000 940000 950000 980000
1100000
1200000 1300000 1400000 1500000 1600000 1700000 1800000
1900000
2100000 2200000 2300000 2400000 2600000 2800000 2900000
3100000
3200000 3400000 3500000 3600000 3700000 3800000 4100000
4200000
5100000 5200000 5800000 6200000 6500000 6800000 7200000
8200000
9100000 9700000 9800000 10300000 11100000 11500000 12300000
14500000
18700000]

We see there are some negative values. Below, we will count how many negative fields there are.

```
negative_followers = (df_streamers["total_followers"] < 0).sum()  
print(f"Number of negative values in total followers  
{negative_followers}")
```

Number of negative values in total followers 5

Fixing the following issues by removing rows: missing language, negative followers

```
print(f"The shape of the dataframe is {df_streamers.shape}")
df_streamers.drop(df_streamers[df_streamers["language"].isnull()].index, inplace = True)
df_streamers.drop(df_streamers[df_streamers["total_followers"] < 0].index, inplace = True)
print(f"The shape of the dataframe is {df_streamers.shape}")
unique_total_followers =
np.unique(df_streamers["total_followers"].values)
print(f"Unique values for total followers are {unique_total_followers}")
```

The shape of the dataframe is (76, 7)

The shape of the dataframe is (63, 7)

Unique values for total followers are [120000 230000 310000
340000 380000 410000 450000 540000
670000 720000 760000 780000 820000 890000 940000
950000
980000 1100000 1200000 1300000 1400000 1500000 1600000
1800000
1900000 2100000 2200000 2300000 2400000 2600000 2800000
2900000
3100000 3200000 3400000 3500000 3600000 3700000 3800000
4100000
5100000 5200000 5800000 6200000 6500000 6800000 8200000
9100000
10300000 11100000 11500000 12300000 14500000 18700000]

streams.csv

Observing implausible values in was_raided

```
unique_was_raided = np.unique(df_streams["was_raided"].values)
print(f"The values for was raided was {unique_was_raided}")
```

The values for was raided was [False True]

Observing implausible values in category

```
unique_categories_streams = df_streams["category"].values.unique()
print(f"The values for categories are {unique_categories_streams}")
```

The values for categories are <StringArray>

['Just Chatting', nan, 'FPS', 'MOBA', 'IRL', 'Creative', 'Sports']
Length: 7, dtype: str

```
missing_category = (df_streams["category"].isnull()).sum()
print(f"Number of missing categories in streams is {missing_category}")
```

Number of missing categories in streams is 50

Observing implausible values in peak_viewers

```
peak_viewers_min = df_streams["peak_viewers"].min()
peak_viewers_max = df_streams["peak_viewers"].max()
peak_viewers_unique = np.unique(df_streams["peak_viewers"].values)
print(f"The minimum peak viewers is {peak_viewers_min} and maximum is {peak_viewers_max}. The unique values are {peak_viewers_unique}")
```

The minimum peak viewers is -50.0 and maximum is 370165.0. The unique values are [-5.00000e+01 -4.90000e+01 -4.50000e+01 ... 3.38648e+05 3.70165e+05 nan]

```
num_negative_peak_viewers = (df_streams["peak_viewers"] < 0).sum()
print(f"There are {num_negative_peak_viewers} negative values in peak viewers")
```

There are 40 negative values in peak viewers

```
num_missing_peak = (df_streams["peak_viewers"].isnull()).sum()
print(f"The number of missing entries in peak viewers is {num_missing_peak}")
```

The number of missing entries in peak viewers is 80

Fixing the following issues by removing rows: missing categories, negative peak viewers, missing peak viewer count

```
print(f"The shape of the dataframe is {df_streams.shape}")
df_streams.drop(df_streams[df_streams["category"].isnull()].index, inplace=True)
df_streams.drop(df_streams[df_streams["peak_viewers"] < 0].index, inplace=True)
df_streams.drop(df_streams[df_streams["peak_viewers"].isnull()].index, inplace=True)
print(f"The shape of the dataframe is {df_streams.shape}")
```

The shape of the dataframe is (1199, 11)
The shape of the dataframe is (1032, 11)

viewers.csv

Observing implausible values in country

```
unique_country_viewers = df_viewers["country"].unique()
print(f"The unique countries are {unique_country_viewers}. Below is the value counts for each.\n\n")
print(df_viewers["country"].value_counts())
```

```
The unique countries are <StringArray>
[  'AR',    'CA',    'US',      nan,    'DE',    'ES', 'other',
  'FR',
    'GB',    'SE',    'BR',    'IT',    'RU',    'MX',    'PL',
  'NL',
    'JP',    'IN',    'KR',    'AU',    'TR']
Length: 21, dtype: str. Below is the value counts for each.
```

```
country
US      264
GB       82
CA       66
DE       53
BR       51
FR       47
KR       44
MX       40
other    38
JP       33
AU       31
IT       28
NL       22
ES       21
RU       20
AR       19
SE       19
IN       18
PL       17
TR        7
Name: count, dtype: int64
```

```
missing_country_viewers = (df_viewers["country"].isnull()).sum()
print(f"There are {missing_country_viewers} missing entries int he country column")
```

There are 80 missing entries int he country column

Replace the missing values in country with other

```
df_viewers_country_fixed = df_viewers.fillna("other")
unique_country_viewers_fixed =
```

```
df_viewers_country_fixed["country"].unique()
print(f"The unique countries are {unique_country_viewers_fixed}. Below
is the value counts for each.\n\n")
```

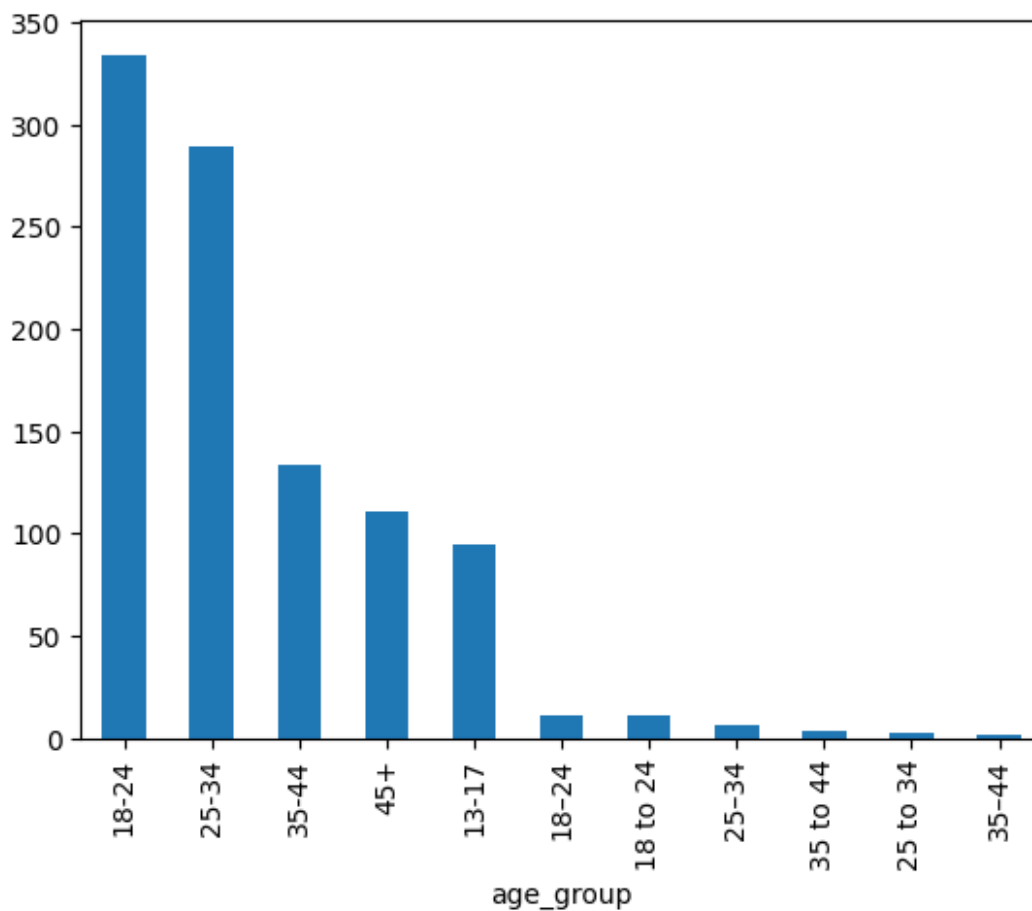
```
The unique countries are <StringArray>
[ 'AR',      'CA',      'US', 'other',      'DE',      'ES',      'FR',
'GB',
'SE',      'BR',      'IT',      'RU',      'MX',      'PL',      'NL',
'JP',
'IN',      'KR',      'AU',      'TR']
Length: 20, dtype: str. Below is the value counts for each.
```

Observing implausible values in age

```
unique_age_viewers = df_viewers_country_fixed["age_group"].unique()
print(f"The age groups are {unique_age_viewers}")
```

```
The age groups are <StringArray>
[ '18-24',      '25-34',      '35-44',      '45+',      '13-17',      '18-
24',
'25-34', '35 to 44', '18 to 24', '25 to 34',      '35-44']
Length: 11, dtype: str
```

```
df_viewers["age_group"].value_counts().plot(kind="bar")
<Axes: xlabel='age_group'>
```

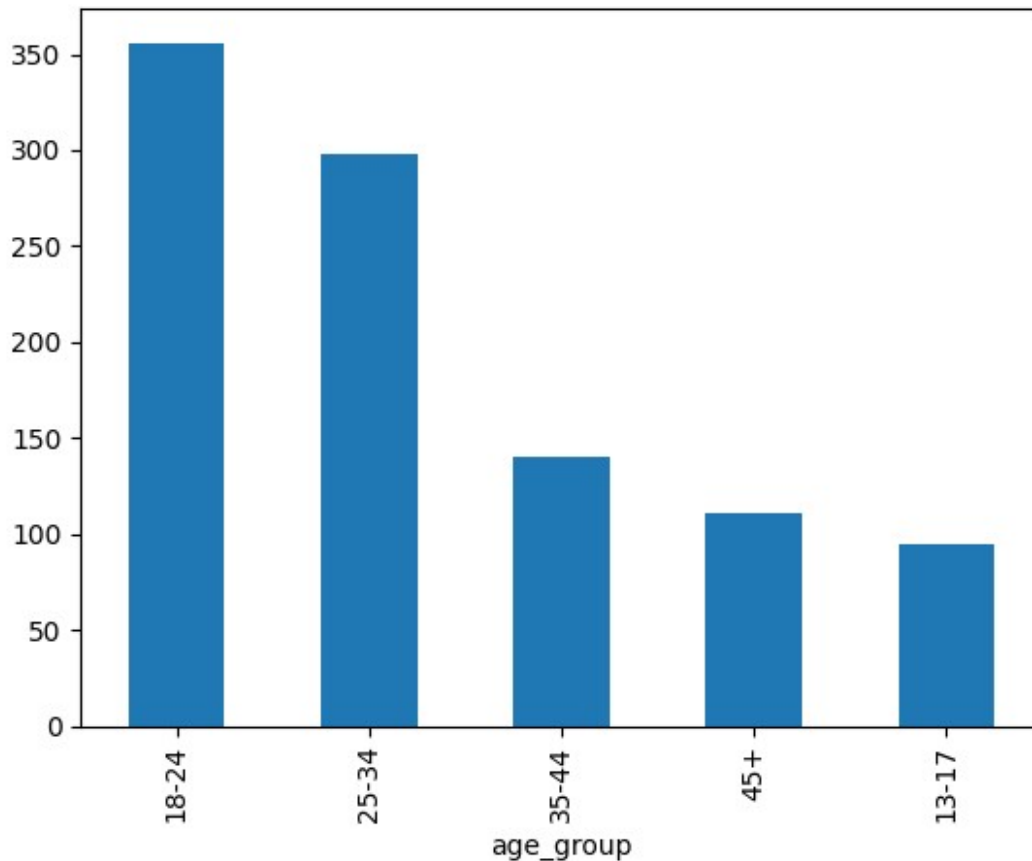


Let us fix this

We can combine some bins to fix this issue. I also posted this on Canvas.

```
# From now on, we can use this df for our analysis
df_viewers_age_fixed = df_viewers.replace({"18 to 24" : "18-24", "18-24" : "18-24", "25 to 34" : "25-34", "25-34" : "25-34", "35 to 44" : "35-44", "35-44" : "35-44" })
df_viewers_age_fixed["age_group"].value_counts().plot(kind="bar")

<Axes: xlabel='age_group'>
```



Observing implausible values in subscription

```
subscription_viewers =
df_viewers_age_fixed["subscription_tier"].value_counts()
display(subscription_viewers)

{"columns":
[{"name": "subscription_tier", "rawType": "str", "type": "string"},
{"name": "count", "rawType": "int64", "type": "integer"}], "ref": "050162dc-
f7da-4428-8afd-c8e68ce69e0e", "rows": [{"free", "578"}, {"tier1", "243"},
["tier2", "100"}, {"tier3", "49"}, {"premium", "10"}, {"none", "7"},
["TIER1", "5"}, {"free trial", "4"}, {"tier4", "4"}], "shape":
{"columns": 1, "rows": 9}}
```

According to the internet, Twitch only has these three tiers: Tier1, Tier2, and Tier3. It offers trial periods, so we will treat `free trial` as valid - I will just change it to `free_trial` for consistency. `free` isn't a thing, so it will be combined with `none`. `tier4` and `premium` don't exist, so I will just cap them to maximum, which is `tier3`.

```
df_viewers_all_fixed = df_viewers_age_fixed.replace({"free" : "none",
"TIER1" : "tier1", "free trial" : "free_trial", "tier4" : "tier3",
"premium" : "tier3"})
display(df_viewers_all_fixed["subscription_tier"].value_counts())
```

```
{
  "columns": [
    {"name": "subscription_tier", "rawType": "str", "type": "string"},
    {"name": "count", "rawType": "int64", "type": "integer"}], "ref": "07c8ccc2-4299-4093-897a-6eeb4507b2ad", "rows": [
    ["none", "585"], ["tier1", "248"], ["tier2", "100"], ["tier3", "63"], ["free_trial", "4"]], "shape": {
    "columns": 1, "rows": 5}}
}
```

Task 2 summary report

There are three ways in which I inspected the data. Firstly, I performed a rough visual inspection by looking through the dataframe. Then, depending on the type of data, I looked at the summary statistics using `df.describe()`, where I could see extreme values for the numerical variables. I also listed the unique data points for categorical data to see what categories exist and looked at min/max, count of negative/missing values. Some of these issues were already visible in part 1.

If the affected rows are little in number, I will remove them.

The issues found are listed below. All of these are errors/systemic problem, not outliers:

`sessions.csv`

- No issues found.

`streamers.csv`

- 5 negative values in the `total_followers` column. I will remove these rows. They are few enough for me to continue my analysis without them.
- 8 missing values in the `languages` column. I will remove these rows.

`streams.csv`

- 50 missing values in the `category` column. I will remove these rows.
- 80 missing values in the `peak_viewers` column. I will remove these rows.
- 40 negative values in `peak_viewers` column. I will remove these rows.

`viewers.csv`

- 80 missing missing values in `country` column. And 38 entries with `other`. Though `other` isn't necessarily an issue, just pointing it out. I replaced `missing` with `other`.
- 5 entries with `subscription = TIER1` instead of `tier1`. Furthermore, the subscription tiers here don't match the tiers twitch has. This has been discussed on Canvas and Discord. I fixed this above by combining some of the fields.
- Repetitions and duplicates in `age_group`. Like `18 to 24` and `18-24`. It is described above and in more detail in Canvas and Discord discussions. I fixed them above by combining some of the age groups.

Common sense/heuristics

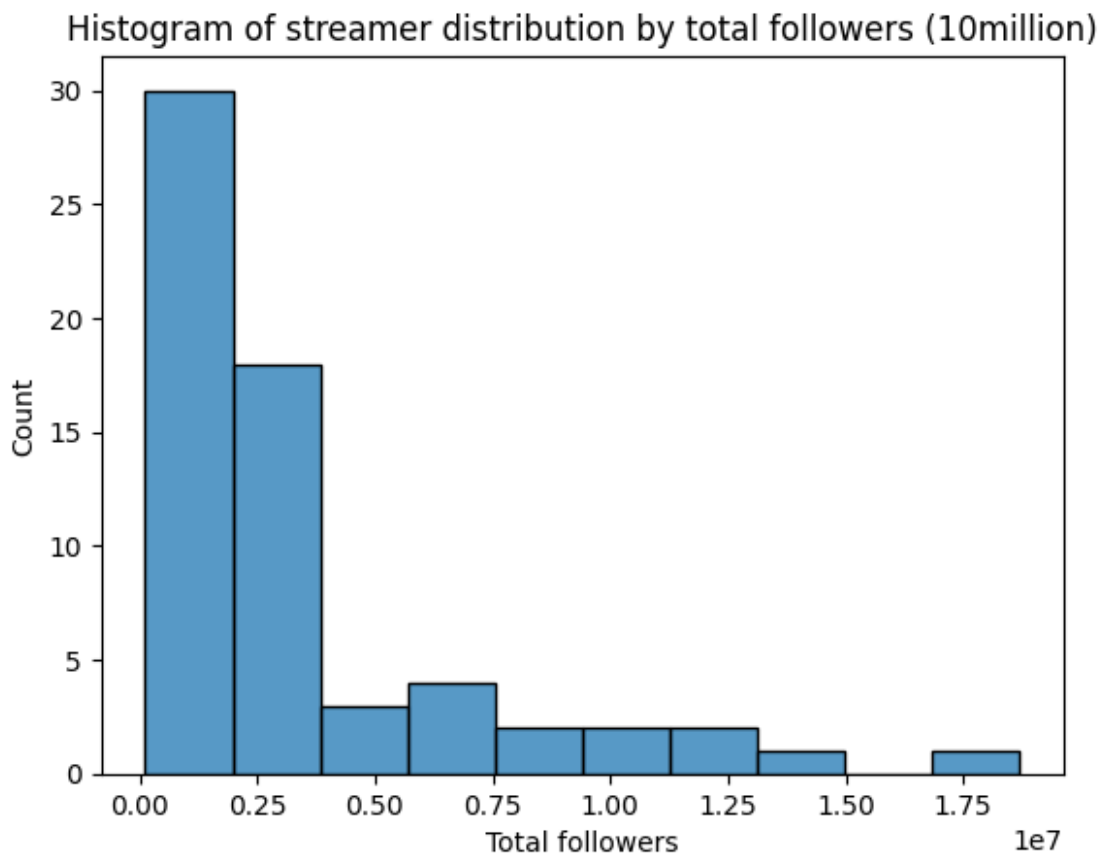
1. Stream end time being earlier than stream start time. That is theoretically not possible.
2. A streamer running two streams with overlapping times (see last section "Data Detective"). Twitch doesn't allow a streamer to have > 1 concurrent stream.
3. A viewer with an account age larger than the age of twitch - for example, 20 years. That would be impossible.

Task 3: Descriptive Statistics and Visualizations

- Create summary statistics and visualizations to understand user characteristics
- Create summary statistics for streams data overall, and summaries by age group, account tire, or peak viewers
- Visualize the distribution of streams durations, popular categories,
- Show category-specific information and compare streamers based on categories

Histogram and summary stats of total followers of streamers

```
plt.figure()
sns.histplot(df_streamers, x="total_followers", bins=10)
plt.title("Histogram of streamer distribution by total followers (10million)")
plt.xlabel("Total followers")
plt.ylabel("Count")
Text(0, 0.5, 'Count')
```



```
print("Given below is 5-number summary of total followers")
display(df_streamers["total_followers"].describe())

frequencies = df_streamers["total_followers"].value_counts(bins=10)
print("Given below is the frequency distribution of the total
followers")
display(frequencies)
```

Given below is 5-number summary of total followers

```
{"columns":[{"name":"index","rawType":"str","type":"string"},
{"name":"total_followers","rawType":"float64","type":"float"}],"ref":"
1878f00e-adeb-4fff-a36e-58a24fecf5ab","rows":[{"count","63.0"},
["mean","3400634.9206349207"],["std","3825519.51901618"],
["min","120000.0"],["25%","915000.0"],["50%","2100000.0"],
["75%","3750000.0"],["max","18700000.0"]],"shape":
{"columns":1,"rows":8}}
```

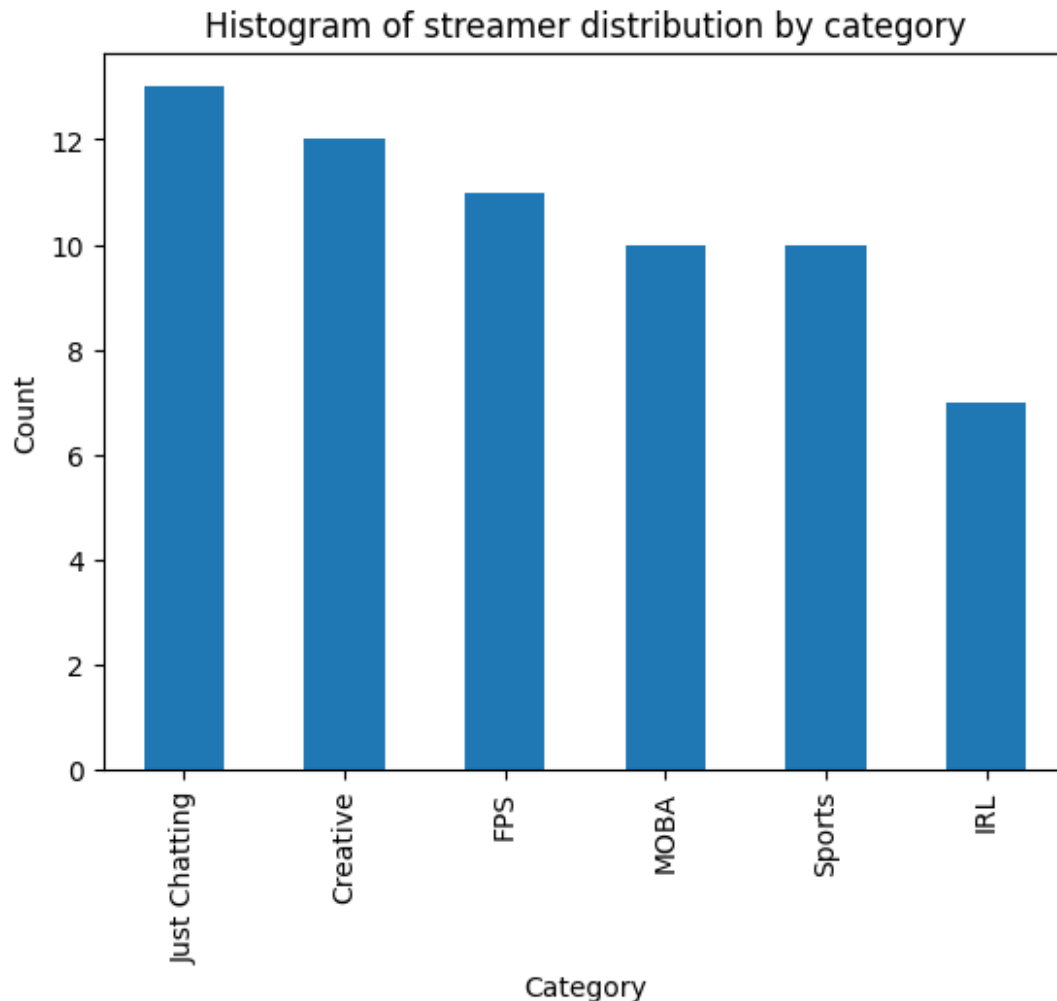
Given below is the frequency distribution of the total followers

```
{"columns":[{"name":"index","rawType":"interval[float64,
right]","type":"unknown"},
{"name":"count","rawType":"int64","type":"integer"}],"ref":"ab4b9b5e-
dd5c-449c-b8a9-d61f54764c12","rows":[{"(101419.999, 1978000.0)","30"},
["(1978000.0, 3836000.0)","18"],["(5694000.0, 7552000.0)","4"],
["(3836000.0, 5694000.0)","3"],["(7552000.0, 9410000.0)","2"],
["(9410000.0, 11268000.0)","2"],["(11268000.0, 13126000.0)","2"],
["(13126000.0, 14984000.0)","1"],["(16842000.0, 18700000.0)","1"],
["(14984000.0, 16842000.0)","0"]],"shape":{"columns":1,"rows":10}}
```

Bar chart and summary stats of category of streamers

```
plt.figure()
df_streamers["category"].value_counts().plot(kind="bar")
plt.title("Histogram of streamer distribution by category")
plt.xlabel("Category")
plt.ylabel("Count")

Text(0, 0.5, 'Count')
```



```
categories = df_streamers["category"].value_counts()
print("Given below is the distribution of the streamer by categories")
display(categories)
```

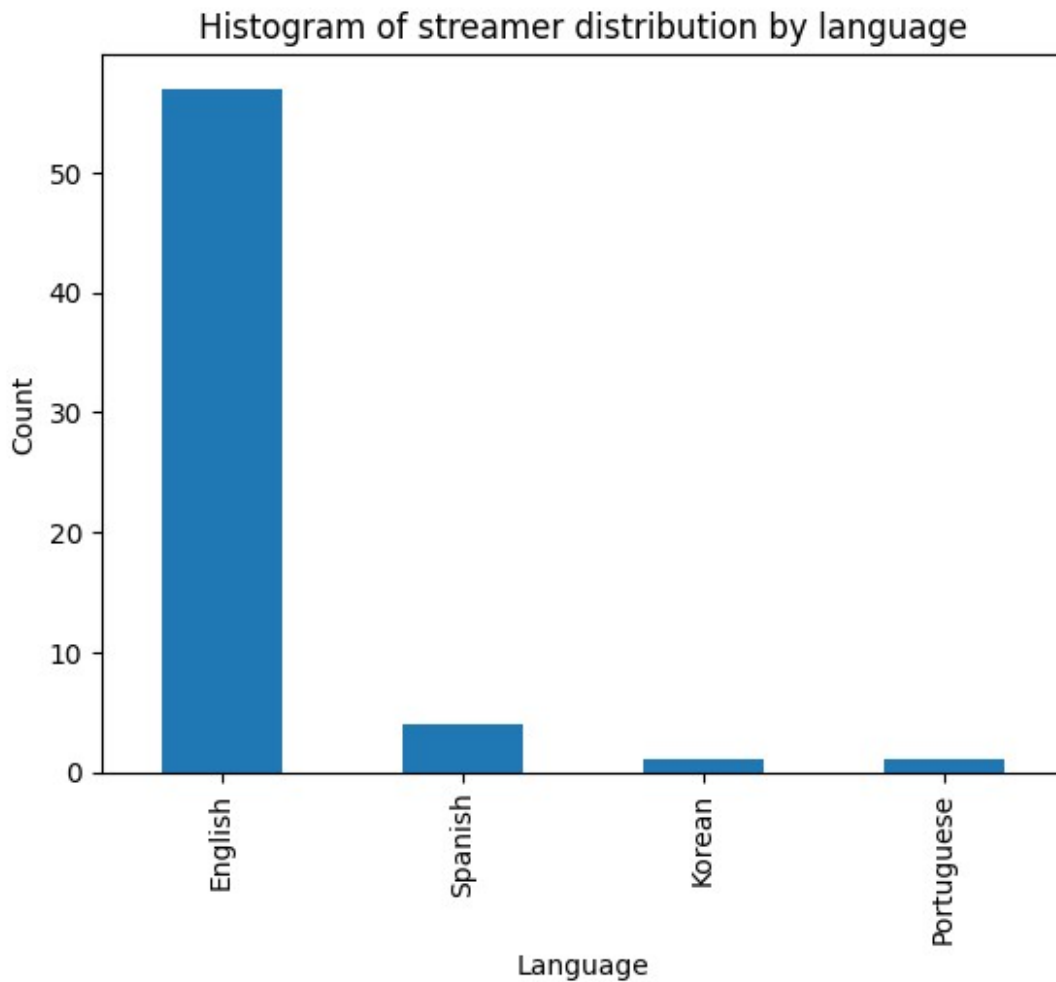
Given below is the distribution of the streamer by categories

```
{"columns":[{"name":"category","rawType":"str","type":"string"},
{"name":"count","rawType":"int64","type":"integer"}],"ref":"9db86741-0514-4f3a-96f2-dc099b25f6fd","rows":[["Just Chatting","13"],
["Creative","12"],["FPS","11"],["MOBA","10"],["Sports","10"],
["IRL","7"]],"shape":{"columns":1,"rows":6}}
```

Bar chart and summary stats of language of streamers

```
plt.figure()
df_streamers["language"].value_counts().plot(kind="bar")
plt.title("Histogram of streamer distribution by language")
plt.xlabel("Language")
plt.ylabel("Count")
```

```
Text(0, 0.5, 'Count')
```



```
languages = df_streamers["language"].value_counts()
print("Given below is the distribution of streamer language")
display(languages)
```

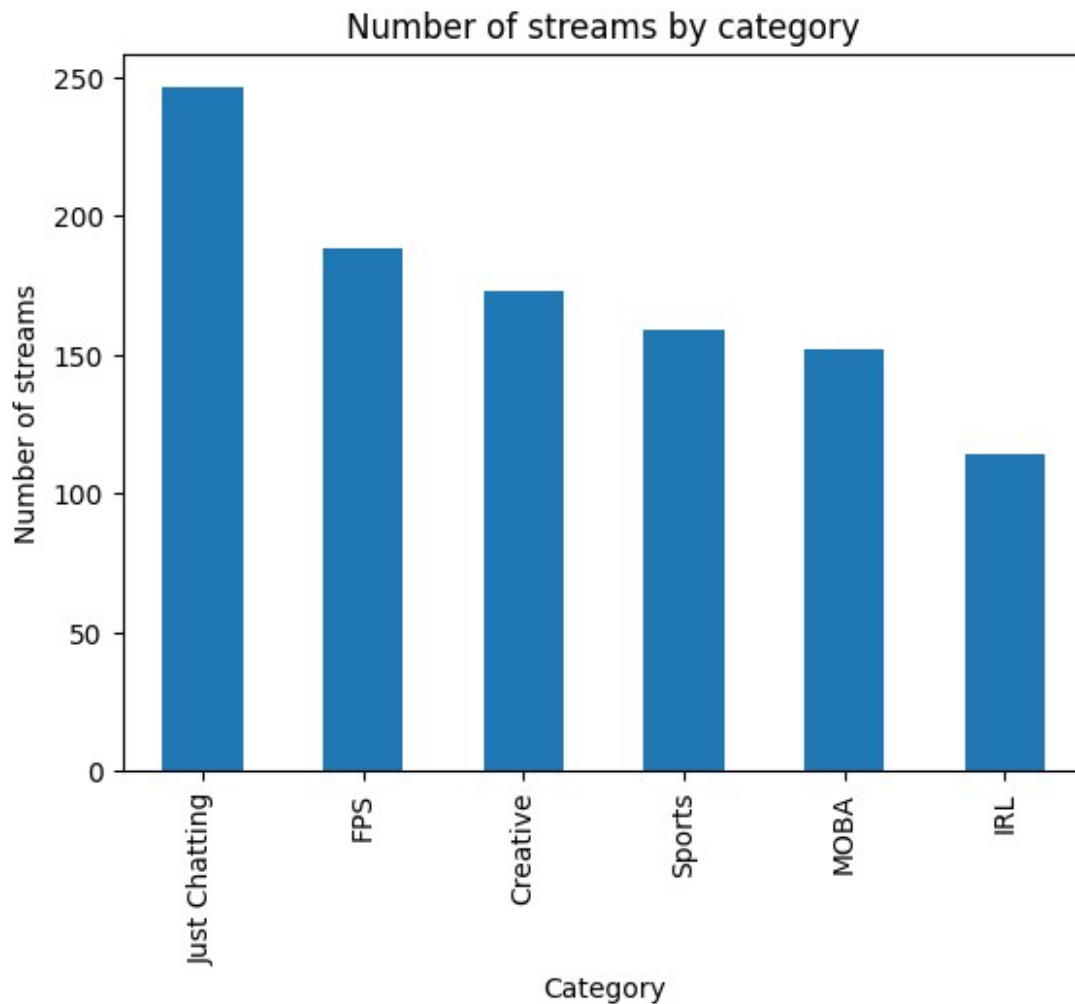
Given below is the distribution of streamer language

```
{"columns":[{"name":"language","rawType":"str","type":"string"},
{"name":"count","rawType":"int64","type":"integer"}],"ref":"6f3b39a5-
062d-4a43-a845-5c0f22594d40","rows":[["English","57"],["Spanish","4"],
["Korean","1"],["Portuguese","1"]],"shape":{"columns":1,"rows":4}}
```

Bar chart and summary stats of categories of streams

```
plt.figure()
df_streams["category"].value_counts().plot(kind="bar")
plt.title("Number of streams by category")
plt.xlabel("Category")
```

```
plt.ylabel("Number of streams")  
plt.show()
```



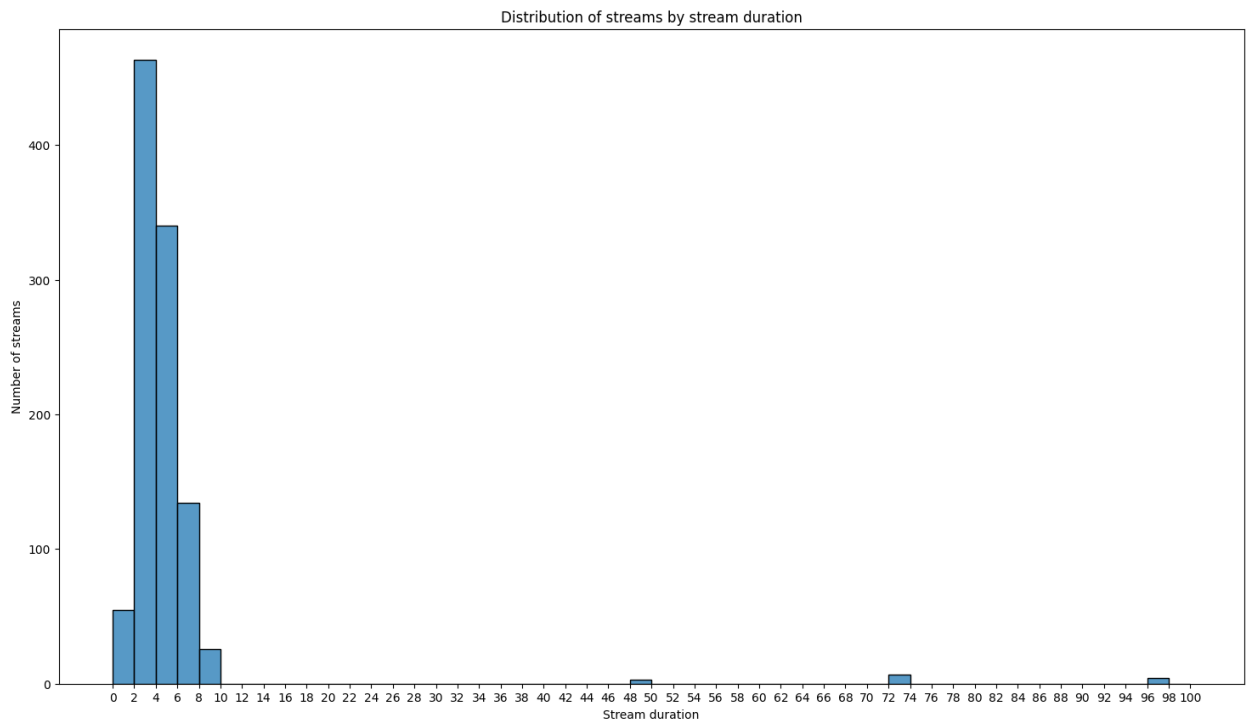
```
categories_streams = df_streams["category"].value_counts()  
print("Given below is the distribution of stream categories")  
display(categories_streams)
```

Given below is the distribution of stream categories

```
{"columns":[{"name":"category","rawType":"str","type":"string"},  
{"name":"count","rawType":"int64","type":"integer"}],"ref":"def01f3b-  
6bdd-423b-af39-edc0ad99715a","rows":[["Just Chatting","246"],  
["FPS","188"],["Creative","173"],["Sports","159"],["MOBA","152"],  
["IRL","114"]],"shape":{"columns":1,"rows":6}}
```

Histogram and summary stats of the distribution of duration of streams

```
fig, ax = plt.subplots(figsize=(18,10))
bin_edges = np.arange(0, 101, 2)
plot = sns.histplot(df_streams, x="stream_duration_hrs",
bins=bin_edges)
plot.set_xticks(bin_edges)
plt.title("Distribution of streams by stream duration")
plt.xlabel("Stream duration")
plt.ylabel("Number of streams")
plt.show()
```



```
print("Given below is 5-number summary of total stream duration")
display(df_streams["stream_duration_hrs"].describe())
```

```
distribution_duration =
df_streams["stream_duration_hrs"].value_counts(bins=bin_edges)
print("Given below is the frequency distribution of the streams
duration")
display(distribution_duration)
```

Given below is 5-number summary of total stream duration

```
{"columns":[{"name":"index","rawType":"str","type":"string"},
{"name":"stream_duration_hrs","rawType":"float64","type":"float"}],"re
f":"8ea51c26-a45f-41e5-b5f1-09a5bf9f8b42","rows":[["count","1032.0"],
["mean","4.721414728682171"],["std","8.481374397622366"]],
```

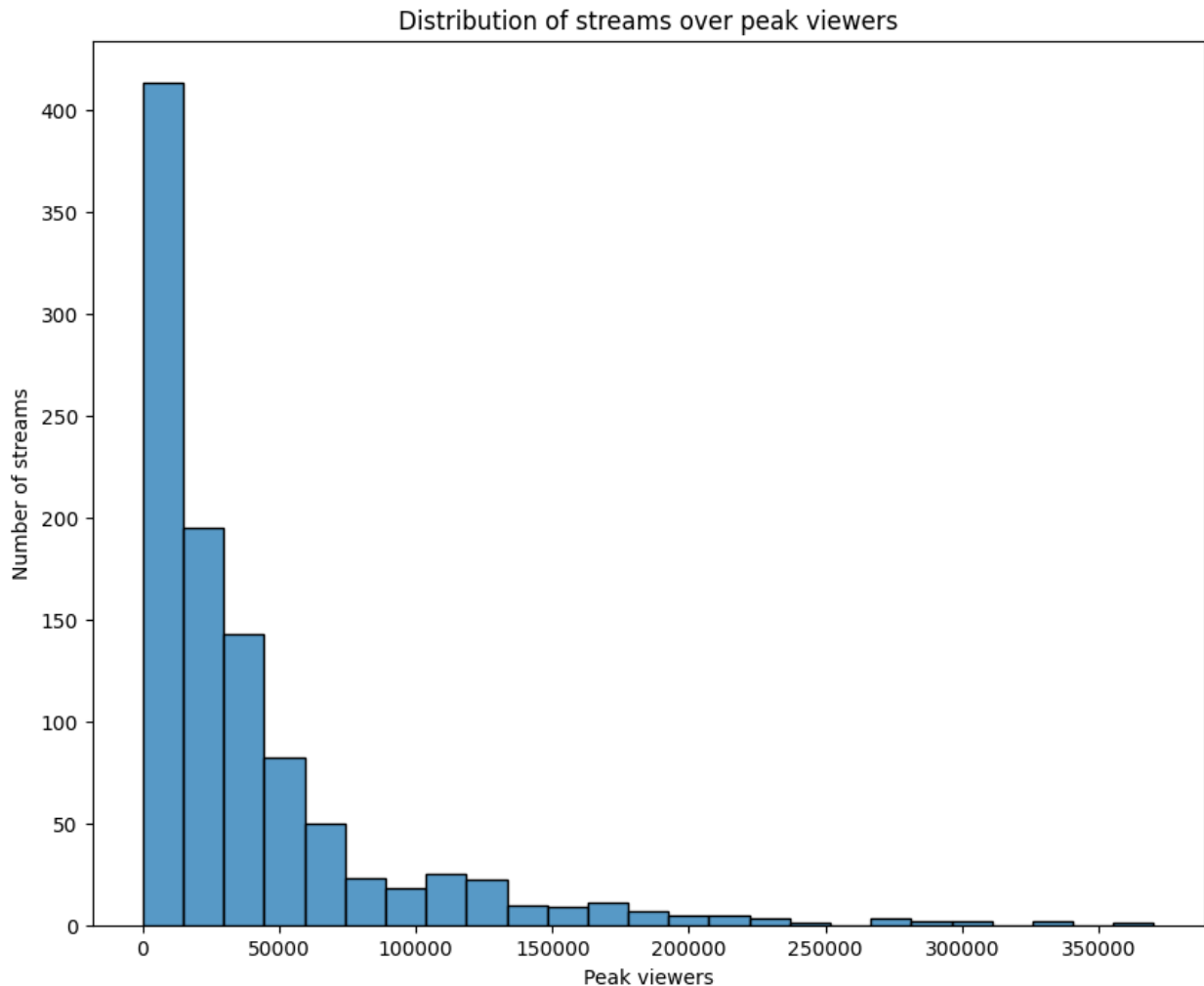
```
[ "min", "1.5"], [ "25%", "2.5"], [ "50%", "3.0"], [ "75%", "5.0"],
[ "max", "96.0"]], "shape": {"columns": 1, "rows": 8}}
```

Given below is the frequency distribution of the streams duration

```
{ "columns": [{"name": "index", "rawType": "interval[float64,
right]", "type": "unknown"},
{"name": "count", "rawType": "int64", "type": "integer"}], "ref": "6bab25aa-
bd28-49a6-9f25-11b2b8d0935a", "rows": [{"(2.0, 4.0]", "549"}, [{"(4.0,
6.0]", "228"}, [{"(-0.001, 2.0]", "168"}, [{"(6.0, 8.0]", "73"}, [{"(70.0,
72.0]", "7"}, [{"(94.0, 96.0]", "4"}, [{"(46.0, 48.0]", "3"}, [{"(8.0,
10.0]", "0"}, [{"(10.0, 12.0]", "0"}, [{"(12.0, 14.0]", "0"}, [{"(14.0,
16.0]", "0"}, [{"(16.0, 18.0]", "0"}, [{"(18.0, 20.0]", "0"}, [{"(20.0,
22.0]", "0"}, [{"(22.0, 24.0]", "0"}, [{"(24.0, 26.0]", "0"}, [{"(26.0,
28.0]", "0"}, [{"(28.0, 30.0]", "0"}, [{"(30.0, 32.0]", "0"}, [{"(32.0,
34.0]", "0"}, [{"(34.0, 36.0]", "0"}, [{"(36.0, 38.0]", "0"}, [{"(38.0,
40.0]", "0"}, [{"(40.0, 42.0]", "0"}, [{"(42.0, 44.0]", "0"}, [{"(44.0,
46.0]", "0"}, [{"(48.0, 50.0]", "0"}, [{"(50.0, 52.0]", "0"}, [{"(52.0,
54.0]", "0"}, [{"(54.0, 56.0]", "0"}, [{"(56.0, 58.0]", "0"}, [{"(58.0,
60.0]", "0"}, [{"(60.0, 62.0]", "0"}, [{"(62.0, 64.0]", "0"}, [{"(64.0,
66.0]", "0"}, [{"(66.0, 68.0]", "0"}, [{"(68.0, 70.0]", "0"}, [{"(72.0,
74.0]", "0"}, [{"(74.0, 76.0]", "0"}, [{"(76.0, 78.0]", "0"}, [{"(78.0,
80.0]", "0"}, [{"(80.0, 82.0]", "0"}, [{"(82.0, 84.0]", "0"}, [{"(84.0,
86.0]", "0"}, [{"(86.0, 88.0]", "0"}, [{"(88.0, 90.0]", "0"}, [{"(90.0,
92.0]", "0"}, [{"(92.0, 94.0]", "0"}, [{"(96.0, 98.0]", "0"}, [{"(98.0,
100.0]", "0"}]], "shape": {"columns": 1, "rows": 50}}
```

Histogram and summary stats of the distribution of peak viewers of streams

```
fig, ax = plt.subplots(figsize=(10,8))
peak_viewers_bin_edges = np.arange(0, 400000, 10000)
sns.histplot(df_streams, x="peak_viewers", bins=25)
plt.title("Distribution of streams over peak viewers")
plt.xlabel("Peak viewers")
plt.ylabel("Number of streams")
plt.show()
```



```
print("Given below is 5-number summary of peak_viewer")
display(df_streams["peak_viewers"].describe())

distribution_peak_viewers =
df_streams["peak_viewers"].value_counts(bins=peak_viewers_bin_edges)
print("Given below is the frequency distribution of the peakviewers")
display(distribution_peak_viewers)
```

Given below is 5-number summary of peak_viewer

```
{
  "columns": [
    {
      "name": "index",
      "rawType": "str",
      "type": "string"
    },
    {
      "name": "peak_viewers",
      "rawType": "float64",
      "type": "float"
    }
  ],
  "ref": "0b5905af-d356-475c-a4a8-100235c5111c",
  "rows": [
    ["count", "1032.0"],
    ["mean", "39595.27519379845"],
    ["std", "50706.7787358752"],
    ["min", "57.0"],
    ["25%", "9100.25"],
    ["50%", "21187.5"],
    ["75%", "48559.5"],
    ["max", "370165.0"]
  ],
  "shape": {
    "columns": 1,
    "rows": 8
  }
}
```

Given below is the frequency distribution of the peakviewers

```

{"columns":[{"name":"index","rawType":"interval[float64,
right]","type":"unknown"},
{"name":"count","rawType":"int64","type":"integer"}],"ref":"c24bb557-
ed31-416a-9aaf-ed8b6f5599e2","rows":[["(-0.001, 10000.0)","284"],
["(10000.0, 20000.0)","214"],["(20000.0, 30000.0)","115"],["(30000.0,
40000.0)","107"],["(40000.0, 50000.0)","58"],["(50000.0,
60000.0)","55"],["(60000.0, 70000.0)","38"],["(70000.0,
80000.0)","22"],["(100000.0, 110000.0)","22"],["(120000.0,
130000.0)","15"],["(80000.0, 90000.0)","13"],["(90000.0,
100000.0)","12"],["(110000.0, 120000.0)","12"],["(160000.0,
170000.0)","10"],["(140000.0, 150000.0)","8"],["(130000.0,
140000.0)","7"],["(150000.0, 160000.0)","6"],["(180000.0,
190000.0)","5"],["(170000.0, 180000.0)","4"],["(200000.0,
210000.0)","4"],["(230000.0, 240000.0)","4"],["(190000.0,
200000.0)","3"],["(210000.0, 220000.0)","3"],["(290000.0,
300000.0)","3"],["(270000.0, 280000.0)","2"],["(330000.0,
340000.0)","2"],["(220000.0, 230000.0)","1"],["(260000.0,
270000.0)","1"],["(280000.0, 290000.0)","1"],["(370000.0,
380000.0)","1"],["(240000.0, 250000.0)","0"],["(250000.0,
260000.0)","0"],["(300000.0, 310000.0)","0"],["(310000.0,
320000.0)","0"],["(320000.0, 330000.0)","0"],["(340000.0,
350000.0)","0"],["(350000.0, 360000.0)","0"],["(360000.0,
370000.0)","0"],["(380000.0, 390000.0)","0"]],"shape":
{"columns":1,"rows":39}}

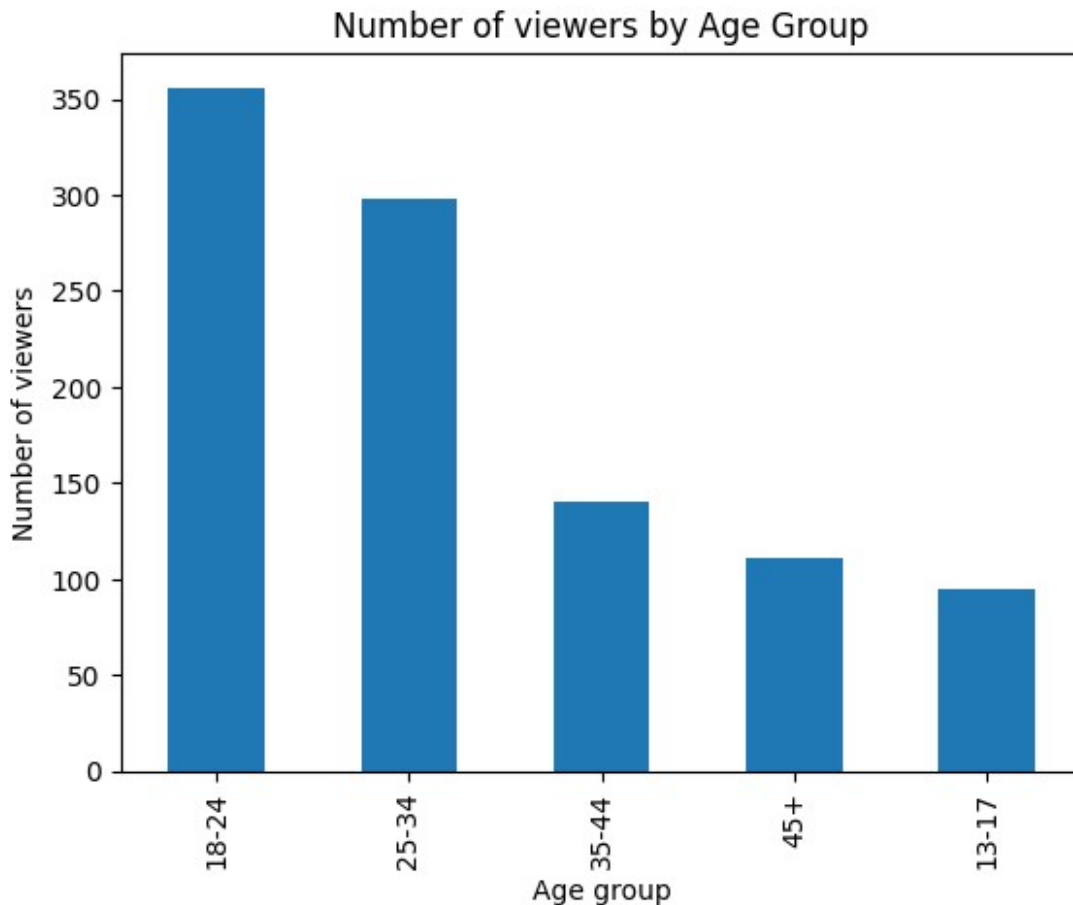
```

Bar chart and summary stats of viewers across age groups

```

plt.figure()
df_viewers_all_fixed["age_group"].value_counts().plot(kind="bar")
plt.title("Number of viewers by Age Group")
plt.xlabel("Age group")
plt.ylabel("Number of viewers")
plt.show()

```



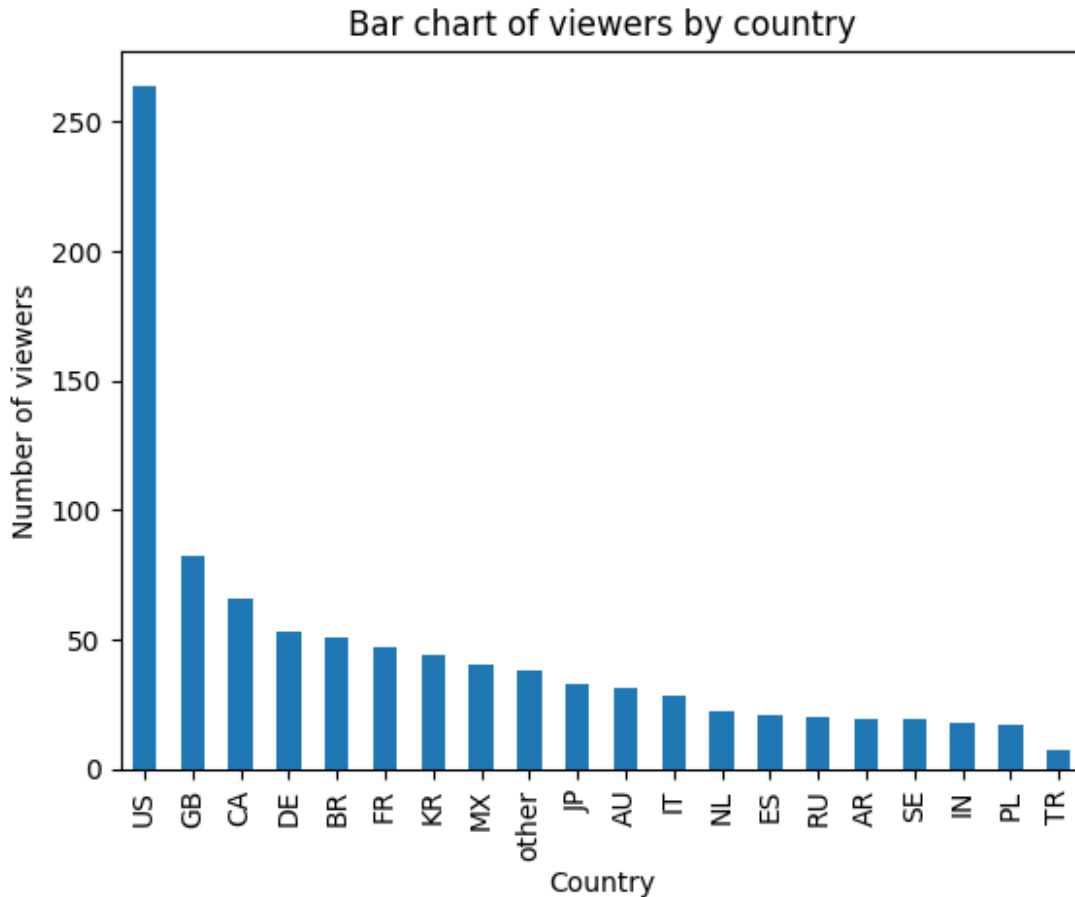
```
distribution_viewer_age =
df_viewers_all_fixed["age_group"].value_counts()
print("Given below is the distribtuion of viewers by age group")
display(distribution_viewer_age)
```

Given below is the distribtuion of viewers by age group

```
{"columns":[{"name":"age_group","rawType":"str","type":"string"},
{"name":"count","rawType":"int64","type":"integer"}],"ref":"402382c4-
e488-46f1-a110-959713976a18","rows":[["18-24","356"],["25-34","298"],
["35-44","140"],["45+","111"],["13-17","95"]],"shape":
{"columns":1,"rows":5}}
```

Bar chart and summary stats of distribution of viewers across countries

```
plt.figure()
df_viewers_all_fixed["country"].value_counts().plot(kind="bar")
plt.title("Bar chart of viewers by country")
plt.xlabel("Country")
plt.ylabel("Number of viewers")
plt.show()
```



```
distribution_viewer_countries =
df_viewers_all_fixed["country"].value_counts()
print("Given below is the distribution viewer countries")
display(distribution_viewer_countries)
```

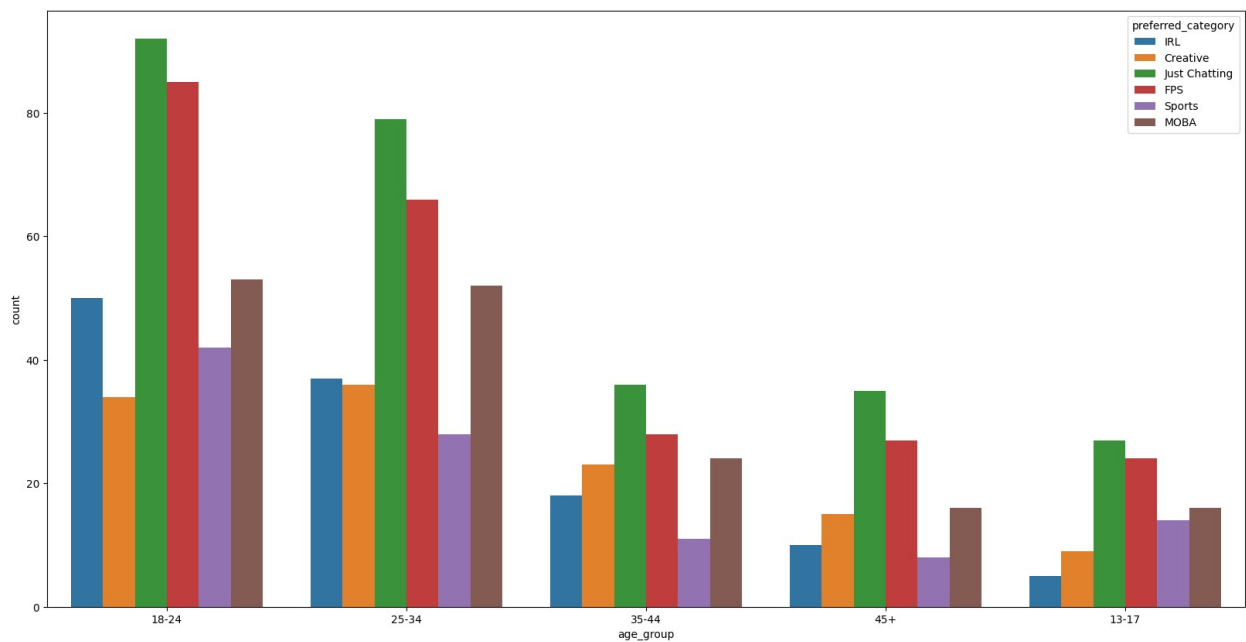
Given below is the distribution viewer countries

```
{
  "columns": [
    {
      "name": "country",
      "rawType": "str",
      "type": "string"
    },
    {
      "name": "count",
      "rawType": "int64",
      "type": "integer"
    }
  ],
  "ref": "4ccb1d4c-d887-4125-a808-07e436282a01",
  "rows": [
    ["US", "264"],
    ["GB", "82"],
    ["CA", "66"],
    ["DE", "53"],
    ["BR", "51"],
    ["FR", "47"],
    ["KR", "44"],
    ["MX", "40"],
    ["other", "38"],
    ["JP", "33"],
    ["AU", "31"],
    ["IT", "28"],
    ["NL", "22"],
    ["ES", "21"],
    ["RU", "20"],
    ["AR", "19"],
    ["SE", "19"],
    ["IN", "18"],
    ["PL", "17"],
    ["TR", "7"]
  ],
  "shape": {
    "columns": 1,
    "rows": 20
  }
}
```

Grouped/clustered barchart of viewers by age group and categories they prefer

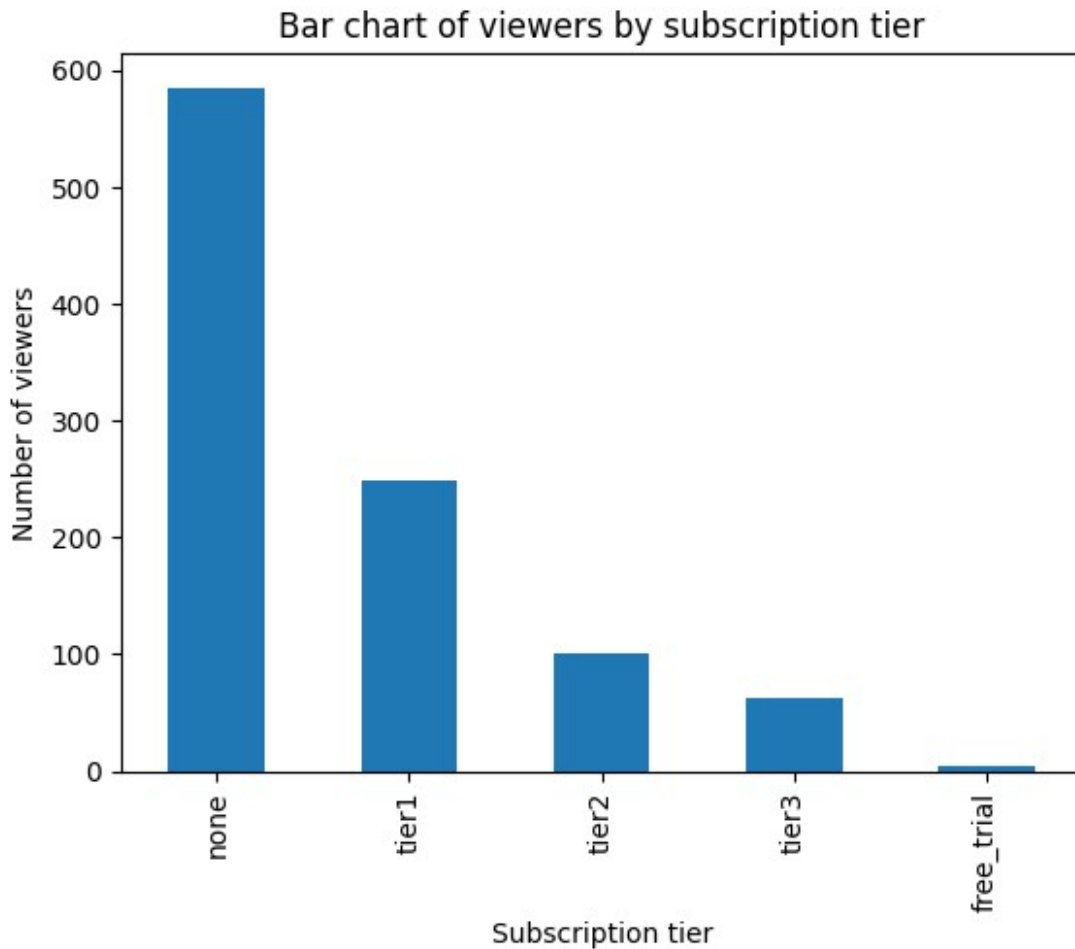
```
fig, ax = plt.subplots(figsize=(20,10))
sns.countplot(df_viewers_all_fixed, x="age_group",
```

```
hue="preferred_category")
plt.show()
```



Bar chart and summary stats of `viewers` across different subscription tiers

```
plt.figure()
df_viewers_all_fixed["subscription_tier"].value_counts().plot(kind="bar")
plt.title("Bar chart of viewers by subscription tier")
plt.xlabel("Subscription tier")
plt.ylabel("Number of viewers")
plt.show()
```



```
distribution_viewers_subscription =  
df_viewers_all_fixed["subscription_tier"].value_counts()  
print("Given below is the distribution of viewers by subscription  
tier")  
display(distribution_viewers_subscription)
```

Given below is the distribution of viewers by subscription tier

```
{"columns":  
[{"name": "subscription_tier", "rawType": "str", "type": "string"},  
{"name": "count", "rawType": "int64", "type": "integer"}], "ref": "f125f0f1-  
668d-4f25-ae47-3dca703473b9", "rows": [{"none", "585"}, {"tier1", "248"},  
["tier2", "100"], ["tier3", "63"], ["free_trial", "4"]], "shape":  
{"columns": 1, "rows": 5}}
```

Task 3 Summary report

Nothing much to report, as the images are more effective in telling the story from the data.

Task 4: Dealing with Outliers and Missing Values

- Are there any missing values or outliers in the original data? Perform the analysis and report your findings.
- Are there any outliers or missing values in the derived data? Perform the analysis and report your findings.
- If there are no outliers or missing values, introduce 3–5% of missing values in a selected column and 1–2% of outliers in another column.
- Experiment with two strategies for imputing missing values and two strategies for handling outliers. Show how each strategy affects the characteristics of the data.

Missing values in original data

`streamers.csv`

- 5 negative values in the `total_followers` column. I removed these rows.
- 8 missing values in the `languages` column. I removed these rows.

`streams.csv`

- 50 missing values in the `category` column. I removed these rows.
- 80 missing values in the `peak_viewers` column. I removed these rows.
- 40 negative values in `peak_viewers` column. I removed these rows.

`viewers.csv`

- 80 missing missing values in `country` column. And 38 entries with `other`. Though `other` isn't necessarily an issue, just pointing it out. I replaced `missing` with `other`.
- 5 entries with `subscription = TIER1` instead of `tier1`. Furthermore, the subscription tiers here don't match the tiers twitch has. This has been discussed on Canvas and Discord. I fixed this above by combining some of the fields.
- Repetitions and duplicates in `age_group`. Like `18 to 24` and `18-24`. It is described above and in more detail in Canvas and Discord discussions. I fixed them above by combining some of the age groups.

Outliers in original data

We can figure outliers using the $1.5 * IQR$ method

Outliers in stream duration in streams

```
q1 = df_streams["stream_duration_hrs"].quantile(0.25)
q3 = df_streams["stream_duration_hrs"].quantile(0.75)
iqr = q3 - q1
lower_bound = q1 - 1.5 * iqr
```

```
upper_bound = q3 + 1.5 * iqr
outliers = df_streams[(df_streams["stream_duration_hrs"] <
lower_bound) | (df_streams["stream_duration_hrs"] > upper_bound)]
display(outliers)
```

```
{
  "columns": [
    {"name": "index", "rawType": "int64", "type": "integer"},
    {"name": "stream_id", "rawType": "str", "type": "string"},
    {"name": "streamer_id", "rawType": "str", "type": "string"},
    {"name": "started_at", "rawType": "str", "type": "string"},
    {"name": "started_at_unix_time", "rawType": "int64", "type": "integer"},
    {"name": "ended_at", "rawType": "str", "type": "string"},
    {"name": "ended_at_unix_time", "rawType": "int64", "type": "integer"},
    {"name": "stream_duration_hrs", "rawType": "float64", "type": "float"},
    {"name": "category", "rawType": "str", "type": "string"},
    {"name": "peak_viewers", "rawType": "float64", "type": "float"},
    {"name": "title_has_hype_word", "rawType": "bool", "type": "boolean"},
    {"name": "was_raided", "rawType": "bool", "type": "boolean"}],
  "ref": "4e23d097-1033-47ea-a152-1e04db3fd4c2",
  "rows": [
    ["48", "SM00049", "STR0003", "2024-02-02 10:00:00", "1706868000", "2024-02-02 15:00:00", "1709908140", "72.0", "Just Chatting", "21516.0", "False", "False"],
    ["284", "SM00285", "STR0018", "2024-01-24 14:00:00", "1706104800", "2024-01-24 17:00:00", "1709504640", "72.0", "FPS", "69221.0", "False", "False"],
    ["332", "SM00333", "STR0021", "2024-02-12 22:00:00", "1707775200", "2024-02-13 02:00:00", "1706987520", "72.0", "FPS", "120277.0", "False", "False"],
    ["357", "SM00358", "STR0023", "2024-03-16 09:00:00", "1710579600", "2024-03-16 12:00:00", "1710690300", "48.0", "FPS", "6073.0", "False", "True"],
    ["364", "SM00365", "STR0024", "2024-02-28 22:00:00", "1709157600", "2024-02-29 02:00:00", "1708174680", "72.0", "FPS", "37056.0", "True", "False"],
    ["372", "SM00373", "STR0024", "2024-01-18 15:00:00", "1705590000", "2024-01-18 22:00:00", "1706177820", "48.0", "FPS", "16570.0", "True", "False"],
    ["417", "SM00418", "STR0026", "2024-03-15 21:00:00", "1710536400", "2024-03-16 02:00:00", "1706735820", "72.0", "FPS", "34674.0", "False", "False"],
    ["458", "SM00459", "STR0029", "2024-02-25 21:00:00", "1708894800", "2024-02-25 23:30:00", "1710117660", "48.0", "FPS", "42620.0", "False", "False"],
    ["546", "SM00547", "STR0034", "2024-02-25 12:00:00", "1708862400", "2024-02-25 16:00:00", "1708710480", "96.0", "MOBA", "4750.0", "False", "True"],
    ["676", "SM00677", "STR0043", "2024-01-15 13:00:00", "1705323600", "2024-01-15 20:00:00", "1709124960", "96.0", "IRL", "40556.0", "False", "False"],
    ["684", "SM00685", "STR0043", "2024-02-23 22:00:00", "1708725600", "2024-02-24 05:00:00", "1707567840", "96.0", "IRL", "42178.0", "False", "False"],
    ["886", "SM00887", "STR0056", "2024-01-30 20:00:00", "1706644800", "2024-01-31 01:00:00", "1705503180", "72.0", "Creative", "12098.0", "False", "False"],
    ["958", "SM00959", "STR0061", "2024-02-03 16:00:00", "1706976000", "2024-02-03 19:00:00", "1710960780", "72.0", "Sports", "209766.0", "True", "False"],
    ["988", "SM00989", "STR0063", "2024-01-05 19:00:00", "1704481200", "2024-01-06"]
  ]
}
```

```
00:00:00", "1704545280", "96.0", "Sports", "33418.0", "False", "False"]], "shape": {"columns": 11, "rows": 14}}
```

Outliers in streamer average concurrent viewers

```
q1 = df_streamers["avg_concurrent_viewers"].quantile(0.25)
q3 = df_streamers["avg_concurrent_viewers"].quantile(0.75)
iqr = q3 - q1
lower_bound = q1 - 1.5 * iqr
upper_bound = q3 + 1.5 * iqr
outliers = df_streamers[(df_streamers["avg_concurrent_viewers"] <
lower_bound) | (df_streamers["avg_concurrent_viewers"] > upper_bound)]
display(outliers)

{"columns": [{"name": "index", "rawType": "int64", "type": "integer"},
{"name": "streamer_id", "rawType": "str", "type": "string"},
{"name": "streamer_name", "rawType": "str", "type": "string"},
{"name": "category", "rawType": "str", "type": "string"},
{"name": "language", "rawType": "str", "type": "string"},
{"name": "partner_status", "rawType": "bool", "type": "boolean"},
{"name": "total_followers", "rawType": "int64", "type": "integer"},
{"name": "avg_concurrent_viewers", "rawType": "int64", "type": "integer"}],
"ref": "54bcd11e-c179-461c-b29a-dd138ae26ae7", "rows":
[["0", "STR0001", "xQc", "Just
Chatting", "English", "True", "11500000", "138706"],
["18", "STR0019", "Tfue", "FPS", "English", "True", "11100000", "141627"],
["60", "STR0061", "KaiCenat", "Sports", "English", "True", "12300000", "13147
7"],
["61", "STR0062", "IShowSpeed", "Sports", "English", "True", "18700000", "151
923"]], "shape": {"columns": 7, "rows": 4}}
```

Outliers in derived data

The derived data here are my tables with missing values fixed. Thus, there are no missing values in those. They still have outliers, as I didn't remove the outliers.

Experimenting with imputation

Missing value: Impute mean

```
df_streams_unaltered = pd.read_csv("data/streams.csv") # using this
since I want to see the missing values but I removed them earlier
df_streams_unaltered.drop(df_streams_unaltered[df_streams_unaltered["p
eak_viewers"] < 0].index, inplace=True) # remove negatives though
df_streams_unaltered["peak_viewers"] =
(df_streams_unaltered["peak_viewers"].fillna(df_streams_unaltered["pea
k_viewers"].mean()))
df_streams_unaltered["peak_viewers"].describe()
```

```
{
  "columns": [
    { "name": "index", "rawType": "str", "type": "string" },
    { "name": "peak_viewers", "rawType": "float64", "type": "float" }
  ],
  "ref": "a03a1d47-355d-4bfc-bf98-6b0f87184c5b",
  "rows": [
    [ "count", "1159.0" ],
    [ "mean", "39799.29935125115" ],
    [ "std", "48962.81346391884" ],
    [ "min", "57.0" ],
    [ "25%", "9685.5" ],
    [ "50%", "24338.0" ],
    [ "75%", "46156.5" ],
    [ "max", "370165.0" ]
  ],
  "shape": { "columns": 1, "rows": 8 }
}
```

Missing value: impute random

```
df_streams_unaltered = pd.read_csv("data/streams.csv")
df_streams_unaltered.drop(df_streams_unaltered[df_streams_unaltered["peak_viewers"] < 0].index, inplace=True) # remove negatives though
rng = np.random.default_rng()
df_streams_unaltered["peak_viewers"] =
(df_streams_unaltered["peak_viewers"].fillna(rng.integers(df_streams_unaltered["peak_viewers"].min(),
df_streams_unaltered["peak_viewers"].max()))
df_streams_unaltered["peak_viewers"].describe()
```

```
{
  "columns": [
    { "name": "index", "rawType": "str", "type": "string" },
    { "name": "peak_viewers", "rawType": "float64", "type": "float" }
  ],
  "ref": "22d29fb9-b045-46b9-b1f3-4ff600981bf3",
  "rows": [
    [ "count", "1159.0" ],
    [ "mean", "51136.638481449525" ],
    [ "std", "64284.31754796448" ],
    [ "min", "57.0" ],
    [ "25%", "9685.5" ],
    [ "50%", "24338.0" ],
    [ "75%", "60143.5" ],
    [ "max", "370165.0" ]
  ],
  "shape": { "columns": 1, "rows": 8 }
}
```

Between the two methods, it doesn't change the min, mean, max. Random increases the std. dev. due to randomness. Plus, if we use a range from min to max for range, the outliers can really skew it. Overall, imputing mean is a safer strategy.

Outliers: Replace with mean

```
df_streams = pd.read_csv("data/streams.csv")
q1 = df_streams["stream_duration_hrs"].quantile(0.25)
q3 = df_streams["stream_duration_hrs"].quantile(0.75)
iqr = q3 - q1
lower_bound = q1 - 1.5 * iqr
upper_bound = q3 + 1.5 * iqr
df_streams.loc[(df_streams["stream_duration_hrs"] < lower_bound) |
(df_streams["stream_duration_hrs"] > upper_bound),
"stream_duration_hrs"] = df_streams["stream_duration_hrs"].mean()
df_streams["stream_duration_hrs"].describe()
```

```
{
  "columns": [
    { "name": "index", "rawType": "str", "type": "string" },
    { "name": "stream_duration_hrs", "rawType": "float64", "type": "float" }
  ],
  "ref": "4bfd3d7d-b786-43f1-8eab-601bf70b5119",
  "rows": [
    [ "count", "1199.0" ],
    [ "mean", "3.7822156495439274" ],
    [ "std", "1.5893643619715345" ],
    [ "min", "0.0" ],
    [ "25%", "1.0" ],
    [ "50%", "2.0" ],
    [ "75%", "3.0" ],
    [ "max", "10.0" ]
  ],
  "shape": { "columns": 1, "rows": 8 }
}
```

```
[ "min", "1.5"], [ "25%", "2.5"], [ "50%", "3.0"], [ "75%", "5.0"],  
[ "max", "8.0"]], "shape": { "columns": 1, "rows": 8}}
```

Outliers: Winsorization

```
df_streams = pd.read_csv("data/streams.csv")  
q1 = df_streams["stream_duration_hrs"].quantile(0.25)  
q3 = df_streams["stream_duration_hrs"].quantile(0.75)  
iqr = q3 - q1  
lower_bound = q1 - 1.5 * iqr  
upper_bound = q3 + 1.5 * iqr  
df_streams.loc[df_streams["stream_duration_hrs"] < lower_bound,  
"stream_duration_hrs"] =  
df_streams["stream_duration_hrs"].quantile(0.05)  
df_streams.loc[df_streams["stream_duration_hrs"] > upper_bound,  
"stream_duration_hrs"] =  
df_streams["stream_duration_hrs"].quantile(0.95)  
df_streams["stream_duration_hrs"].describe()  
  
{ "columns": [ { "name": "index", "rawType": "str", "type": "string"},  
{ "name": "stream_duration_hrs", "rawType": "float64", "type": "float"}], "re  
f": "563b75de-9555-432b-a66b-8cb65f93f2ff", "rows": [ [ "count", "1199.0"],  
[ "mean", "3.811926605504587"], [ "std", "1.6266354168189687"],  
[ "min", "1.5"], [ "25%", "2.5"], [ "50%", "3.0"], [ "75%", "5.0"],  
[ "max", "8.0"]], "shape": { "columns": 1, "rows": 8}}
```

Between the two, it doesn't affect min and max. Std dev is higher in winsorization. So is mean. Overall, difference isn't much, but winsorization is reasonable in large datasets.

Task 5: Correlation and Behavioral Patterns

- Compute pairwise correlations between user features and the number of check-ins
- Visualize the results using a correlation matrix (use a coolwarm color scheme and proper scaling)
- Try Spearman rank correlation and explain when it is more appropriate than Pearson

For check-ins, I will use `bits_cheered` and `chat_messages_sent` in `sessions.csv`. I will create a new column in `df_sessions` called `total_checkins`. `total_checkins = bits_cheered + chat_messages_sent`

```
df_sessions["total_checkins"] = df_sessions["chat_messages_sent"] +  
df_sessions["bits_cheered"]  
display(df_sessions["total_checkins"].describe())  
display(df_sessions)
```

```
{
  "columns": [
    { "name": "index", "rawType": "str", "type": "string" },
    { "name": "total_checkins", "rawType": "float64", "type": "float" },
    { "name": "count", "rawType": "float64", "type": "float" },
    { "name": "std", "rawType": "float64", "type": "float" },
    { "name": "min", "rawType": "float64", "type": "float" },
    { "name": "25%", "rawType": "float64", "type": "float" },
    { "name": "50%", "rawType": "float64", "type": "float" },
    { "name": "75%", "rawType": "float64", "type": "float" },
    { "name": "max", "rawType": "float64", "type": "float" }
  ],
  "shape": { "columns": 1, "rows": 8 }
}
```

```
{
  "columns": [
    { "name": "index", "rawType": "int64", "type": "integer" },
    { "name": "session_id", "rawType": "str", "type": "string" },
    { "name": "viewer_id", "rawType": "str", "type": "string" },
    { "name": "streamer_id", "rawType": "str", "type": "string" },
    { "name": "stream_id", "rawType": "str", "type": "string" },
    { "name": "started_at", "rawType": "str", "type": "string" },
    { "name": "started_at_unix_time", "rawType": "int64", "type": "integer" },
    { "name": "ended_at", "rawType": "str", "type": "string" },
    { "name": "ended_at_unix_time", "rawType": "int64", "type": "integer" },
    { "name": "duration_mins", "rawType": "int64", "type": "integer" },
    { "name": "chat_messages_sent", "rawType": "int64", "type": "integer" },
    { "name": "bits_cheered", "rawType": "int64", "type": "integer" },
    { "name": "followed_during", "rawType": "bool", "type": "boolean" },
    { "name": "subscribed_during", "rawType": "bool", "type": "boolean" },
    { "name": "total_checkins", "rawType": "int64", "type": "integer" }
  ],
  "ref": "6c595f2e-51e8-4bd0-92f7-518ae693e633",
  "rows":
```

```
[
  [
    "0", "SES000001", "VWR00938", "STR0055", "SM00860", "2024-01-18",
    "19:16:00", "1705605360", "2024-01-18",
    "20:28:00", "1705609680", "72", "0", "0", "False", "False", "0"
  ],
  [
    "1", "SES000002", "VWR00804", "STR0034", "SM00542", "2024-01-04",
    "17:11:00", "1704388260", "2024-01-04",
    "19:41:00", "1704397260", "150", "11", "0", "False", "False", "11"
  ],
  [
    "2", "SES000003", "VWR00704", "STR0008", "SM00124", "2024-02-15",
    "15:11:00", "1708009860", "2024-02-15",
    "16:53:00", "1708015980", "102", "5", "0", "False", "False", "5"
  ],
  [
    "3", "SES000004", "VWR00926", "STR0024", "SM00373", "2024-01-18",
    "19:59:00", "1705607940", "2024-01-18",
    "20:56:00", "1705611360", "57", "3", "9663", "False", "False", "9666"
  ],
  [
    "4", "SES000005", "VWR00883", "STR0008", "SM00120", "2024-03-19",
    "20:08:00", "1710878880", "2024-03-19",
    "20:22:00", "1710879720", "14", "7", "0", "False", "False", "7"
  ],
  [
    "5", "SES000006", "VWR00860", "STR0056", "SM00877", "2024-01-07",
    "00:41:00", "1704588060", "2024-01-07",
    "02:13:00", "1704593580", "92", "13", "0", "False", "False", "13"
  ],
  [
    "6", "SES000007", "VWR00925", "STR0052", "SM00828", "2024-01-29",
    "21:41:00", "1706564460", "2024-01-30",
    "01:59:00", "1706579940", "258", "13", "0", "False", "False", "13"
  ],
  [
    "7", "SES000008", "VWR00519", "STR0047", "SM00730", "2024-03-13",
    "14:52:00", "1710341520", "2024-03-13",
    "14:59:00", "1710341940", "7", "0", "0", "False", "False", "0"
  ],
  [
    "8", "SES000009", "VWR00901", "STR0045", "SM00715", "2024-02-17",
    "18:42:00", "1708195320", "2024-02-17",
    "19:13:00", "1708197180", "31", "1", "0", "False", "False", "1"
  ],

```

["9", "SES000010", "VWR00869", "STR0016", "SM00238", "2024-02-03
18:07:00", "1706983620", "2024-02-03
18:54:00", "1706986440", "47", "5", "0", "False", "False", "5"],
["10", "SES000011", "VWR00812", "STR0033", "SM00530", "2024-03-17
11:00:00", "1710673200", "2024-03-17
11:33:00", "1710675180", "33", "1", "0", "False", "False", "1"],
["11", "SES000012", "VWR00424", "STR0006", "SM00096", "2024-03-27
12:29:00", "1711542540", "2024-03-27
14:37:00", "1711550220", "128", "1", "0", "False", "False", "1"],
["12", "SES000013", "VWR00839", "STR0042", "SM00676", "2024-01-24
13:53:00", "1706104380", "2024-01-24
14:05:00", "1706105100", "12", "0", "0", "False", "False", "0"],
["13", "SES000014", "VWR00633", "STR0024", "SM00374", "2024-03-01
21:30:00", "1709328600", "2024-03-01
22:57:00", "1709333820", "87", "3", "3873", "False", "False", "3876"],
["14", "SES000015", "VWR00925", "STR0030", "SM00487", "2024-01-07
19:38:00", "1704656280", "2024-01-07
20:20:00", "1704658800", "42", "0", "0", "False", "False", "0"],
["15", "SES000016", "VWR00601", "STR0012", "SM00182", "2024-03-12
12:37:00", "1710247020", "2024-03-12
13:03:00", "1710248580", "26", "2", "0", "False", "False", "2"],
["16", "SES000017", "VWR00928", "STR0020", "SM00313", "2024-03-20
02:48:00", "1710902880", "2024-03-20
03:43:00", "1710906180", "55", "7", "0", "False", "False", "7"],
["17", "SES000018", "VWR00067", "STR0061", "SM00950", "2024-01-05
00:02:00", "1704412920", "2024-01-05
02:29:00", "1704421740", "147", "8", "0", "True", "False", "8"],
["18", "SES000019", "VWR00992", "STR0057", "SM00894", "2024-01-11
21:31:00", "1705008660", "2024-01-11
23:46:00", "1705016760", "135", "6", "3985", "False", "False", "3991"],
["19", "SES000020", "VWR00726", "STR0074", "SM01179", "2024-01-31
13:02:00", "1706706120", "2024-01-31
14:07:00", "1706710020", "65", "4", "0", "False", "False", "4"],
["20", "SES000021", "VWR00994", "STR0051", "SM00792", "2024-02-19
12:10:00", "1708344600", "2024-02-19
12:46:00", "1708346760", "36", "5", "4638", "False", "False", "4643"],
["21", "SES000022", "VWR00846", "STR0075", "SM01182", "2024-03-20
18:00:00", "1710957600", "2024-03-20
18:51:00", "1710960660", "51", "7", "0", "False", "False", "7"],
["22", "SES000023", "VWR00722", "STR0064", "SM01010", "2024-03-09
12:11:00", "1709986260", "2024-03-09
13:17:00", "1709990220", "66", "4", "0", "False", "False", "4"],
["23", "SES000024", "VWR00979", "STR0070", "SM01108", "2024-02-28
20:44:00", "1709153040", "2024-02-28
20:49:00", "1709153340", "5", "1", "0", "False", "False", "1"],
["24", "SES000025", "VWR00616", "STR0024", "SM00367", "2024-03-30
16:15:00", "1711815300", "2024-03-30
16:29:00", "1711816140", "14", "1", "0", "False", "False", "1"],
["25", "SES000026", "VWR00387", "STR0065", "SM01021", "2024-03-11

16:06:00","1710173160","2024-03-11
18:50:00","1710183000","164","13","0","False","False","13"],
["26","SES000027","VWR00959","STR0060","SM00937","2024-01-07
20:20:00","1704658800","2024-01-07
21:57:00","1704664620","97","0","6251","False","False","6251"],
["27","SES000028","VWR00852","STR0066","SM01043","2024-03-25
16:41:00","1711384860","2024-03-25
16:49:00","1711385340","8","0","0","False","False","0"],
["28","SES000029","VWR00682","STR0005","SM00080","2024-03-21
22:10:00","1711059000","2024-03-21
22:27:00","1711060020","17","2","0","False","False","2"],
["29","SES000030","VWR00217","STR0021","SM00327","2024-01-31
14:06:00","1706709960","2024-01-31
14:56:00","1706712960","50","2","0","False","False","2"],
["30","SES000031","VWR00455","STR0054","SM00848","2024-01-23
18:17:00","1706033820","2024-01-23
21:33:00","1706045580","196","1","0","False","False","1"],
["31","SES000032","VWR00628","STR0067","SM01056","2024-02-08
09:05:00","1707383100","2024-02-08
11:35:00","1707392100","150","33","3589","False","False","3622"],
["32","SES000033","VWR00624","STR0006","SM00100","2024-01-08
21:50:00","1704750600","2024-01-08
22:13:00","1704751980","23","5","0","False","False","5"],
["33","SES000034","VWR00646","STR0071","SM01132","2024-02-18
23:42:00","1708299720","2024-02-19
00:58:00","1708304280","76","4","412","True","False","416"],
["34","SES000035","VWR00878","STR0023","SM00358","2024-03-16
11:47:00","1710589620","2024-03-16
11:55:00","1710590100","8","0","0","False","False","0"],
["35","SES000036","VWR00928","STR0055","SM00864","2024-03-10
08:33:00","1710059580","2024-03-10
08:46:00","1710060360","13","0","0","False","False","0"],
["36","SES000037","VWR00382","STR0018","SM00271","2024-01-02
21:05:00","1704229500","2024-01-02
23:08:00","1704236880","123","0","0","False","False","0"],
["37","SES000038","VWR00980","STR0060","SM00936","2024-02-28
09:47:00","1709113620","2024-02-28
10:52:00","1709117520","65","7","0","False","False","7"],
["38","SES000039","VWR00772","STR0026","SM00412","2024-01-31
14:11:00","1706710260","2024-01-31
15:06:00","1706713560","55","2","0","False","False","2"],
["39","SES000040","VWR00890","STR0072","SM01137","2024-02-20
18:21:00","1708453260","2024-02-20
20:01:00","1708459260","100","12","0","False","False","12"],
["40","SES000041","VWR00426","STR0049","SM00766","2024-03-07
14:54:00","1709823240","2024-03-07
15:31:00","1709825460","37","7","0","False","False","7"],
["41","SES000042","VWR00259","STR0024","SM00368","2024-02-19
13:44:00","1708350240","2024-02-19

```

16:10:00", "1708359000", "146", "3", "0", "False", "False", "3"],
["42", "SES000043", "VWR00927", "STR0003", "SM00047", "2024-02-17
12:49:00", "1708174140", "2024-02-17
13:27:00", "1708176420", "38", "0", "0", "False", "False", "0"],
["43", "SES000044", "VWR00969", "STR0006", "SM00098", "2024-03-14
20:36:00", "1710448560", "2024-03-14
20:46:00", "1710449160", "10", "0", "0", "False", "False", "0"],
["44", "SES000045", "VWR00978", "STR0049", "SM00759", "2024-01-05
23:16:00", "1704496560", "2024-01-06
03:12:00", "1704510720", "236", "47", "0", "False", "False", "47"],
["45", "SES000046", "VWR00665", "STR0004", "SM00069", "2024-03-07
08:37:00", "1709800620", "2024-03-07
10:09:00", "1709806140", "92", "15", "0", "False", "False", "15"],
["46", "SES000047", "VWR00765", "STR0055", "SM00859", "2024-03-23
12:27:00", "1711196820", "2024-03-23
13:05:00", "1711199100", "38", "1", "0", "False", "False", "1"],
["47", "SES000048", "VWR00865", "STR0044", "SM00695", "2024-02-09
16:56:00", "1707497760", "2024-02-09
17:26:00", "1707499560", "30", "1", "0", "False", "False", "1"],
["48", "SES000049", "VWR00872", "STR0044", "SM00701", "2024-03-08
12:59:00", "1709902740", "2024-03-08
14:29:00", "1709908140", "90", "9", "254", "False", "False", "263"],
["49", "SES000050", "VWR00949", "STR0030", "SM00475", "2024-01-11
15:37:00", "1704987420", "2024-01-11
19:24:00", "1705001040", "227", "7", "0", "False", "False", "7"]], "shape":
{"columns":14,"rows":22825}}

```

To make it easier to access the streamer category, I can do that with `pd.merge`

```

df_sessions_with_categories = pd.merge(df_sessions, df_streamers,
on="streamer_id", how="inner")
display(df_sessions_with_categories)

{"columns":[{"name":"index","rawType":"int64","type":"integer"},
{"name":"session_id","rawType":"str","type":"string"},
{"name":"viewer_id","rawType":"str","type":"string"},
{"name":"streamer_id","rawType":"str","type":"string"},
{"name":"stream_id","rawType":"str","type":"string"},
{"name":"started_at","rawType":"str","type":"string"},
{"name":"started_at_unix_time","rawType":"int64","type":"integer"},
{"name":"ended_at","rawType":"str","type":"string"},
{"name":"ended_at_unix_time","rawType":"int64","type":"integer"},
{"name":"duration_mins","rawType":"int64","type":"integer"},
{"name":"chat_messages_sent","rawType":"int64","type":"integer"},
{"name":"bits_cheered","rawType":"int64","type":"integer"},
{"name":"followed_during","rawType":"bool","type":"boolean"},
{"name":"subscribed_during","rawType":"bool","type":"boolean"},
{"name":"total_checkins","rawType":"int64","type":"integer"},
{"name":"streamer_name","rawType":"str","type":"string"},

```

```
{
  "name": "category", "rawType": "str", "type": "string",
  "name": "language", "rawType": "str", "type": "string",
  "name": "partner_status", "rawType": "bool", "type": "boolean",
  "name": "total_followers", "rawType": "int64", "type": "integer",
  "name": "avg_concurrent_viewers", "rawType": "int64", "type": "integer"
},
"ref": "1491ced8-f44c-4b30-8466-73ae7e678c74", "rows":
[
  [
    "0", "SES000001", "VWR00938", "STR0055", "SM00860", "2024-01-18",
    "19:16:00", "1705605360", "2024-01-18",
    "20:28:00", "1705609680", "72", "0", "0", "False", "False", "0", "PointCrow", "Creative", "English", "True", "890000", "9938"
  ],
  [
    "1", "SES000002", "VWR00804", "STR0034", "SM00542", "2024-01-04",
    "17:11:00", "1704388260", "2024-01-04",
    "19:41:00", "1704397260", "150", "11", "0", "False", "False", "11", "Caps", "MOBA", "English", "True", "980000", "6679"
  ],
  [
    "2", "SES000004", "VWR00926", "STR0024", "SM00373", "2024-01-18",
    "19:59:00", "1705607940", "2024-01-18",
    "20:56:00", "1705611360", "57", "3", "9663", "False", "False", "9666", "simple", "FPS", "English", "True", "3600000", "48998"
  ],
  [
    "3", "SES000006", "VWR00860", "STR0056", "SM00877", "2024-01-07",
    "00:41:00", "1704588060", "2024-01-07",
    "02:13:00", "1704593580", "92", "13", "0", "False", "False", "13", "Alpharad", "Creative", "English", "True", "760000", "9233"
  ],
  [
    "4", "SES000007", "VWR00925", "STR0052", "SM00828", "2024-01-29",
    "21:41:00", "1706564460", "2024-01-30",
    "01:59:00", "1706579940", "258", "13", "0", "False", "False", "13", "Northernlion", "Creative", "English", "True", "3100000", "6496"
  ],
  [
    "5", "SES000008", "VWR00519", "STR0047", "SM00730", "2024-03-13",
    "14:52:00", "1710341520", "2024-03-13",
    "14:59:00", "1710341940", "7", "0", "0", "False", "False", "0", "ConnorEatsPants", "IRL", "English", "False", "380000", "2674"
  ],
  [
    "6", "SES000009", "VWR00901", "STR0045", "SM00715", "2024-02-17",
    "18:42:00", "1708195320", "2024-02-17",
    "19:13:00", "1708197180", "31", "1", "0", "False", "False", "1", "Nmplol", "IRL", "English", "True", "1600000", "20511"
  ],
  [
    "7", "SES000010", "VWR00869", "STR0016", "SM00238", "2024-02-03",
    "18:07:00", "1706983620", "2024-02-03",
    "18:54:00", "1706986440", "47", "5", "0", "False", "False", "5", "Trainwreckstv", "Just Chatting", "English", "True", "2800000", "13871"
  ],
  [
    "8", "SES000011", "VWR00812", "STR0033", "SM00530", "2024-03-17",
    "11:00:00", "1710673200", "2024-03-17",
    "11:33:00", "1710675180", "33", "1", "0", "False", "False", "1", "Faker", "MOBA", "Korean", "True", "5800000", "36167"
  ],
  [
    "9", "SES000012", "VWR00424", "STR0006", "SM00096", "2024-03-27",
    "12:29:00", "1711542540", "2024-03-27",
    "14:37:00", "1711550220", "128", "1", "0", "False", "False", "1", "QTCinderella", "Just Chatting", "English", "True", "950000", "13948"
  ],
  [
    "10", "SES000014", "VWR00633", "STR0024", "SM00374", "2024-03-01",
    "21:30:00", "1709328600", "2024-03-01",
    "22:57:00", "1709333820", "87", "3", "3873", "False", "False", "3876", "simple"
  ]
]
```

, "FPS", "English", "True", "3600000", "48998"],
["11", "SES000015", "VWR00925", "STR0030", "SM00487", "2024-01-07
19:38:00", "1704656280", "2024-01-07
20:20:00", "1704658800", "42", "0", "0", "False", "False", "0", "Tyler1", "MOBA",
"English", "True", "5200000", "56574"],
["12", "SES000016", "VWR00601", "STR0012", "SM00182", "2024-03-12
12:37:00", "1710247020", "2024-03-12
13:03:00", "1710248580", "26", "2", "0", "False", "False", "2", "Hafu", "Just
Chatting", "English", "True", "1400000", "19667"],
["13", "SES000017", "VWR00928", "STR0020", "SM00313", "2024-03-20
02:48:00", "1710902880", "2024-03-20
03:43:00", "1710906180", "55", "7", "0", "False", "False", "7", "Nickmercs", "F
PS", "English", "True", "6800000", "69439"],
["14", "SES000018", "VWR00067", "STR0061", "SM00950", "2024-01-05
00:02:00", "1704412920", "2024-01-05
02:29:00", "1704421740", "147", "8", "0", "True", "False", "8", "KaiCenat", "Sp
orts", "English", "True", "12300000", "131477"],
["15", "SES000019", "VWR00992", "STR0057", "SM00894", "2024-01-11
21:31:00", "1705008660", "2024-01-11
23:46:00", "1705016760", "135", "6", "3985", "False", "False", "3991", "Lacari",
"Creative", "English", "False", "310000", "2469"],
["16", "SES000020", "VWR00726", "STR0074", "SM01179", "2024-01-31
13:02:00", "1706706120", "2024-01-31
14:07:00", "1706710020", "65", "4", "0", "False", "False", "4", "Pituquistream",
"Just Chatting", "Spanish", "True", "780000", "3932"],
["17", "SES000021", "VWR00994", "STR0051", "SM00792", "2024-02-19
12:10:00", "1708344600", "2024-02-19
12:46:00", "1708346760", "36", "5", "4638", "False", "False", "4643", "DougDou
g", "Creative", "English", "True", "1200000", "5518"],
["18", "SES000022", "VWR00846", "STR0075", "SM01182", "2024-03-20
18:00:00", "1710957600", "2024-03-20
18:51:00", "1710960660", "51", "7", "0", "False", "False", "7", "ProxyPipe", "M
OBA", "English", "False", "230000", "634"],
["19", "SES000024", "VWR00979", "STR0070", "SM01108", "2024-02-28
20:44:00", "1709153040", "2024-02-28
20:49:00", "1709153340", "5", "1", "0", "False", "False", "1", "TeamLiquid", "S
ports", "English", "True", "2200000", "16890"],
["20", "SES000025", "VWR00616", "STR0024", "SM00367", "2024-03-30
16:15:00", "1711815300", "2024-03-30
16:29:00", "1711816140", "14", "1", "0", "False", "False", "1", "s1mple", "FPS",
"English", "True", "3600000", "48998"],
["21", "SES000026", "VWR00387", "STR0065", "SM01021", "2024-03-11
16:06:00", "1710173160", "2024-03-11
18:50:00", "1710183000", "164", "13", "0", "False", "False", "13", "ESLCS", "Sp
orts", "English", "True", "2100000", "21527"],
["22", "SES000028", "VWR00852", "STR0066", "SM01043", "2024-03-25
16:41:00", "1711384860", "2024-03-25
16:49:00", "1711385340", "8", "0", "0", "False", "False", "0", "BeyondTheSummi
t", "Sports", "English", "True", "1300000", "11955"],

["23", "SES000029", "VWR00682", "STR0005", "SM00080", "2024-03-21
22:10:00", "1711059000", "2024-03-21
22:27:00", "1711060020", "17", "2", "0", "False", "False", "2", "Mizkif", "Just
Chatting", "English", "True", "2100000", "6771"],
["24", "SES000031", "VWR00455", "STR0054", "SM00848", "2024-01-23
18:17:00", "1706033820", "2024-01-23
21:33:00", "1706045580", "196", "1", "0", "False", "False", "1", "SmallAnt", "C
reative", "English", "True", "1100000", "11707"],
["25", "SES000032", "VWR00628", "STR0067", "SM01056", "2024-02-08
09:05:00", "1707383100", "2024-02-08
11:35:00", "1707392100", "150", "33", "3589", "False", "False", "3622", "Tarik
", "Sports", "English", "True", "3800000", "35224"],
["26", "SES000033", "VWR00624", "STR0006", "SM00100", "2024-01-08
21:50:00", "1704750600", "2024-01-08
22:13:00", "1704751980", "23", "5", "0", "False", "False", "5", "QTCinderella"
", "Just Chatting", "English", "True", "950000", "13948"],
["27", "SES000034", "VWR00646", "STR0071", "SM01132", "2024-02-18
23:42:00", "1708299720", "2024-02-19
00:58:00", "1708304280", "76", "4", "412", "True", "False", "416", "Ibai_espor
ts", "Sports", "Spanish", "True", "6500000", "31132"],
["28", "SES000035", "VWR00878", "STR0023", "SM00358", "2024-03-16
11:47:00", "1710589620", "2024-03-16
11:55:00", "1710590100", "8", "0", "0", "False", "False", "0", "Teep", "FPS", "E
nglish", "False", "340000", "4970"],
["29", "SES000036", "VWR00928", "STR0055", "SM00864", "2024-03-10
08:33:00", "1710059580", "2024-03-10
08:46:00", "1710060360", "13", "0", "0", "False", "False", "0", "PointCrow", "C
reative", "English", "True", "890000", "9938"],
["30", "SES000037", "VWR00382", "STR0018", "SM00271", "2024-01-02
21:05:00", "1704229500", "2024-01-02
23:08:00", "1704236880", "123", "0", "0", "False", "False", "0", "shroud", "FPS
", "English", "True", "10300000", "29145"],
["31", "SES000039", "VWR00772", "STR0026", "SM00412", "2024-01-31
14:11:00", "1706710260", "2024-01-31
15:06:00", "1706713560", "55", "2", "0", "False", "False", "2", "Symfuhny", "FP
S", "English", "True", "3200000", "14496"],
["32", "SES000040", "VWR00890", "STR0072", "SM01137", "2024-02-20
18:21:00", "1708453260", "2024-02-20
20:01:00", "1708459260", "100", "12", "0", "False", "False", "12", "BobRoss_Tw
itch", "Creative", "English", "True", "940000", "6872"],
["33", "SES000041", "VWR00426", "STR0049", "SM00766", "2024-03-07
14:54:00", "1709823240", "2024-03-07
15:31:00", "1709825460", "37", "7", "0", "False", "False", "7", "Jinny", "IRL",
", "English", "True", "820000", "8915"],
["34", "SES000042", "VWR00259", "STR0024", "SM00368", "2024-02-19
13:44:00", "1708350240", "2024-02-19
16:10:00", "1708359000", "146", "3", "0", "False", "False", "3", "s1mple", "FPS
", "English", "True", "3600000", "48998"],
["35", "SES000043", "VWR00927", "STR0003", "SM00047", "2024-02-17

12:49:00","1708174140","2024-02-17
13:27:00","1708176420","38","0","0","False","False","0","HasanAbi","Ju
st Chatting","English","True","2600000","34220"],
["36","SES000044","VWR00969","STR0006","SM00098","2024-03-14
20:36:00","1710448560","2024-03-14
20:46:00","1710449160","10","0","0","False","False","0","QTCinderella"
,"Just Chatting","English","True","950000","13948"],
["37","SES000045","VWR00978","STR0049","SM00759","2024-01-05
23:16:00","1704496560","2024-01-06
03:12:00","1704510720","236","47","0","False","False","47","Jinny","IR
L","English","True","820000","8915"],
["38","SES000046","VWR00665","STR0004","SM00069","2024-03-07
08:37:00","1709800620","2024-03-07
10:09:00","1709806140","92","15","0","False","False","15","Amouranth",
"Just Chatting","English","True","6200000","68607"],
["39","SES000047","VWR00765","STR0055","SM00859","2024-03-23
12:27:00","1711196820","2024-03-23
13:05:00","1711199100","38","1","0","False","False","1","PointCrow","C
reative","English","True","890000","9938"],
["40","SES000048","VWR00865","STR0044","SM00695","2024-02-09
16:56:00","1707497760","2024-02-09
17:26:00","1707499560","30","1","0","False","False","1","Sodapoppin",
"IRL","English","True","9100000","55152"],
["41","SES000049","VWR00872","STR0044","SM00701","2024-03-08
12:59:00","1709902740","2024-03-08
14:29:00","1709908140","90","9","254","False","False","263","Sodapoppi
n","IRL","English","True","9100000","55152"],
["42","SES000050","VWR00949","STR0030","SM00475","2024-01-11
15:37:00","1704987420","2024-01-11
19:24:00","1705001040","227","7","0","False","False","7","Tyler1","MOB
A","English","True","5200000","56574"],
["43","SES000051","VWR00288","STR0016","SM00254","2024-03-16
12:10:00","1710591000","2024-03-16
13:26:00","1710595560","76","6","0","False","False","6","Trainwreckstv
","Just Chatting","English","True","2800000","13871"],
["44","SES000053","VWR00914","STR0076","SM01192","2024-01-16
16:42:00","1705423320","2024-01-16
18:26:00","1705429560","104","1","0","False","False","1","BabyTate","F
PS","English","False","120000","678"],
["45","SES000054","VWR00894","STR0045","SM00712","2024-03-01
16:56:00","1709312160","2024-03-01
17:31:00","1709314260","35","0","0","False","False","0","Nmplol","IRL"
,"English","True","1600000","20511"],
["46","SES000055","VWR00731","STR0023","SM00364","2024-02-06
14:14:00","1707228840","2024-02-06
15:10:00","1707232200","56","8","0","False","False","8","Teep","FPS",
"English","False","340000","4970"],
["47","SES000056","VWR00933","STR0058","SM00901","2024-01-13
22:56:00","1705186560","2024-01-13

```

23:15:00", "1705187700", "19", "2", "0", "False", "False", "2", "Vinesauce", "C
reative", "English", "True", "1600000", "15029"],
["48", "SES000057", "VWR00553", "STR0037", "SM00601", "2024-03-05
14:26:00", "1709648760", "2024-03-05
15:23:00", "1709652180", "57", "1", "0", "False", "False", "1", "Jankos", "MOBA
", "English", "True", "780000", "2877"],
["49", "SES000058", "VWR00884", "STR0046", "SM00727", "2024-02-10
22:47:00", "1707605220", "2024-02-10
23:36:00", "1707608160", "49", "1", "0", "False", "False", "1", "Stanz", "IRL",
"English", "False", "410000", "5109"]], "shape":
{"columns":20, "rows":18753}}

```

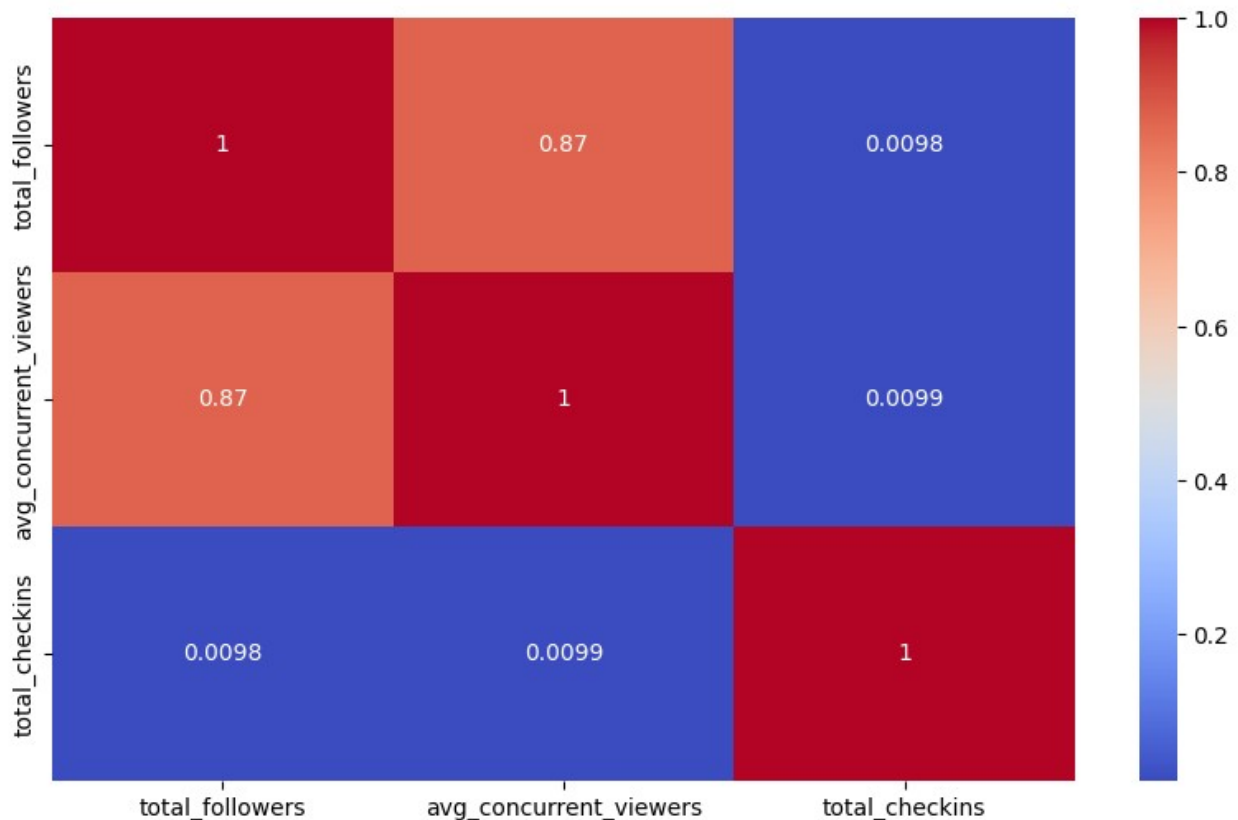
Correlation (pearson) of streamer features and number of check-ins

```

streamer_corr_columns = ["total_followers", "avg_concurrent_viewers",
"total_checkins"]
correlation_streamer =
df_sessions_with_categories[streamer_corr_columns].corr()
display(correlation_streamer)
fig, ax = plt.subplots(figsize=(10,6))
sns.heatmap(correlation_streamer, annot=True, cmap="coolwarm")
plt.show()

{"columns":[{"name":"index","rawType":"str","type":"string"},
{"name":"total_followers","rawType":"float64","type":"float"},
{"name":"avg_concurrent_viewers","rawType":"float64","type":"float"},
{"name":"total_checkins","rawType":"float64","type":"float"}], "ref":"3
d3bc9c0-e50a-49a8-bb6c-5c899ad3836e", "rows":
[["total_followers", "1.0", "0.8689446503939287", "0.009808162878273774"]
,
["avg_concurrent_viewers", "0.8689446503939287", "1.0", "0.00994003337144
6357"],
["total_checkins", "0.009808162878273774", "0.009940033371446357", "1.0"]
], "shape":{"columns":3, "rows":3}}

```

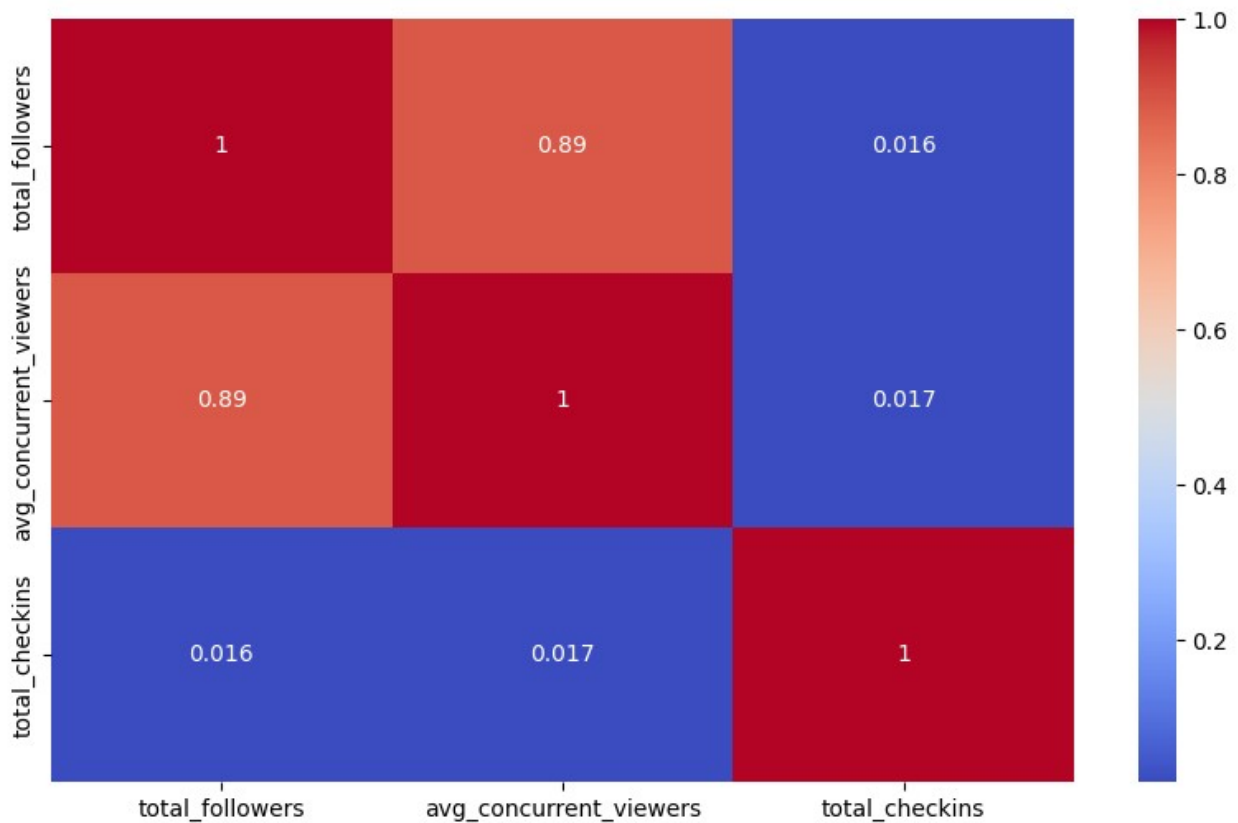


Correlation (spearman) of streamer features and number of check-ins

```
streamer_corr_columns = ["total_followers", "avg_concurrent_viewers",
"total_checkins"]
correlation_streamer_spearman =
df_sessions_with_categories[streamer_corr_columns].corr(method="spearman")
display(correlation_streamer_spearman)
fig, ax = plt.subplots(figsize=(10,6))
sns.heatmap(correlation_streamer_spearman, annot=True,
cmap="coolwarm")
plt.show()
```

```
{
  "columns": [
    {
      "name": "index",
      "rawType": "str",
      "type": "string"
    },
    {
      "name": "total_followers",
      "rawType": "float64",
      "type": "float"
    },
    {
      "name": "avg_concurrent_viewers",
      "rawType": "float64",
      "type": "float"
    },
    {
      "name": "total_checkins",
      "rawType": "float64",
      "type": "float"
    }
  ],
  "ref": "4550a3c9-c689-4ce9-a3e5-55444b259e6f",
  "rows": [
    [
      "total_followers",
      "1.0",
      "0.8894586879151173",
      "0.016423677581920107"
    ],
    [
      "avg_concurrent_viewers",
      "0.8894586879151173",
      "1.0",
      "0.016956081577795764"
    ],
    [
      "total_checkins",
      "0.016423677581920107",
      "0.016956081577795764",
      "1.0"
    ]
  ]
}
```

```
[ "total_checkins", "0.016423677581920107", "0.016956081577795764", "1.0" ]  
], "shape": {"columns": 3, "rows": 3}}
```



Spearman vs pearson

Pearson is to be used for primarily ordinal data and non linear relationships. As a result it is not really appropriate here. So we won't use it.

Task 5 Summary report

Task 6: Compare Behavior Across User Segments

- Segment users into 2–3 groups (e.g., by age bracket, account tier)
- Compare their viewing behavior over time using boxplots or dot plots
- Identify any outliers in group behavior and investigate them

Viewer behaviour based on age-group

For this, I would like to find the total number of sessions participated by the user. I can merge `df_sessions` with `df_viewers`. And add a column for total sessions by this viewer.

```
df_sessions_with_viewer_age = pd.merge(df_sessions,
df_viewers_all_fixed, on="viewer_id", how="inner")
df_sessions_with_viewer_age["sessions_by_viewer"] =
df_sessions_with_viewer_age.groupby("viewer_id")
["viewer_id"].transform("count")
display(df_sessions_with_viewer_age)

{"columns":[{"name":"index","rawType":"int64","type":"integer"},
{"name":"session_id","rawType":"str","type":"string"},
{"name":"viewer_id","rawType":"str","type":"string"},
{"name":"streamer_id","rawType":"str","type":"string"},
{"name":"stream_id","rawType":"str","type":"string"},
{"name":"started_at","rawType":"str","type":"string"},
{"name":"started_at_unix_time","rawType":"int64","type":"integer"},
{"name":"ended_at","rawType":"str","type":"string"},
{"name":"ended_at_unix_time","rawType":"int64","type":"integer"},
{"name":"duration_mins","rawType":"int64","type":"integer"},
{"name":"chat_messages_sent","rawType":"int64","type":"integer"},
{"name":"bits_cheered","rawType":"int64","type":"integer"},
{"name":"followed_during","rawType":"bool","type":"boolean"},
{"name":"subscribed_during","rawType":"bool","type":"boolean"},
{"name":"total_checkins","rawType":"int64","type":"integer"},
{"name":"age_group","rawType":"str","type":"string"},
{"name":"country","rawType":"str","type":"string"},
{"name":"account_age_days","rawType":"int64","type":"integer"},
{"name":"subscription_tier","rawType":"str","type":"string"},
{"name":"preferred_category","rawType":"str","type":"string"},
{"name":"sessions_by_viewer","rawType":"int64","type":"integer"}],"ref
":"ff73dfce-4c97-45d0-8ce8-96065e7c4285","rows":
[["0","SES000001","VWR00938","STR0055","SM00860","2024-01-18
19:16:00","1705605360","2024-01-18
20:28:00","1705609680","72","0","0","False","False","0","13-
17","IN","1875","none","Just Chatting","85"],
["1","SES000002","VWR00804","STR0034","SM00542","2024-01-04
17:11:00","1704388260","2024-01-04
19:41:00","1704397260","150","11","0","False","False","11","18-
24","RU","2085","tier1","FPS","32"],
["2","SES000003","VWR00704","STR0008","SM00124","2024-02-15
15:11:00","1708009860","2024-02-15
16:53:00","1708015980","102","5","0","False","False","5","18-
24","CA","1762","tier1","MOBA","40"],
["3","SES000004","VWR00926","STR0024","SM00373","2024-01-18
19:59:00","1705607940","2024-01-18
20:56:00","1705611360","57","3","9663","False","False","9666","18-
```

```
24","US","258","none","Just Chatting","89"],
["4","SES000005","VWR00883","STR0008","SM00120","2024-03-19
20:08:00","1710878880","2024-03-19
20:22:00","1710879720","14","7","0","False","False","7","45+","CA","27
65","tier2","Just Chatting","85"],
["5","SES000006","VWR00860","STR0056","SM00877","2024-01-07
00:41:00","1704588060","2024-01-07
02:13:00","1704593580","92","13","0","False","False","13","25-
34","US","839","none","FPS","98"],
["6","SES000007","VWR00925","STR0052","SM00828","2024-01-29
21:41:00","1706564460","2024-01-30
01:59:00","1706579940","258","13","0","False","False","13","18-
24","CA","2983","none","IRL","60"],
["7","SES000008","VWR00519","STR0047","SM00730","2024-03-13
14:52:00","1710341520","2024-03-13
14:59:00","1710341940","7","0","0","False","False","0","25-
34",null,"2824","tier1","Creative","9"],
["8","SES000009","VWR00901","STR0045","SM00715","2024-02-17
18:42:00","1708195320","2024-02-17
19:13:00","1708197180","31","1","0","False","False","1","18-
24","AR","1355","none","Creative","61"],
["9","SES000010","VWR00869","STR0016","SM00238","2024-02-03
18:07:00","1706983620","2024-02-03
18:54:00","1706986440","47","5","0","False","False","5","45+","DE","29
23","tier2","MOBA","85"],
["10","SES000011","VWR00812","STR0033","SM00530","2024-03-17
11:00:00","1710673200","2024-03-17
11:33:00","1710675180","33","1","0","False","False","1","18-
24","GB","2300","none","IRL","34"],
["11","SES000012","VWR00424","STR0006","SM00096","2024-03-27
12:29:00","1711542540","2024-03-27
14:37:00","1711550220","128","1","0","False","False","1","25-
34","KR","3413","none","IRL","14"],
["12","SES000013","VWR00839","STR0042","SM00676","2024-01-24
13:53:00","1706104380","2024-01-24
14:05:00","1706105100","12","0","0","False","False","0","25-
34","US","1587","none","IRL","40"],
["13","SES000014","VWR00633","STR0024","SM00374","2024-03-01
21:30:00","1709328600","2024-03-01
22:57:00","1709333820","87","3","3873","False","False","3876","25-
34","US","3156","none","Just Chatting","38"],
["14","SES000015","VWR00925","STR0030","SM00487","2024-01-07
19:38:00","1704656280","2024-01-07
20:20:00","1704658800","42","0","0","False","False","0","18-
24","CA","2983","none","IRL","60"],
["15","SES000016","VWR00601","STR0012","SM00182","2024-03-12
12:37:00","1710247020","2024-03-12
13:03:00","1710248580","26","2","0","False","False","2","25-
34","US","378","none","Just Chatting","38"],
```

["16", "SES000017", "VWR00928", "STR0020", "SM00313", "2024-03-20
02:48:00", "1710902880", "2024-03-20
03:43:00", "1710906180", "55", "7", "0", "False", "False", "7", "25-
34", "GB", "1323", "none", "IRL", "100"],
["17", "SES000018", "VWR00067", "STR0061", "SM00950", "2024-01-05
00:02:00", "1704412920", "2024-01-05
02:29:00", "1704421740", "147", "8", "0", "True", "False", "8", "25-
34", "SE", "732", "tier1", "Sports", "3"],
["18", "SES000019", "VWR00992", "STR0057", "SM00894", "2024-01-11
21:31:00", "1705008660", "2024-01-11
23:46:00", "1705016760", "135", "6", "3985", "False", "False", "3991", "25-
34", "US", "205", "tier1", "Just Chatting", "95"],
["19", "SES000020", "VWR00726", "STR0074", "SM01179", "2024-01-31
13:02:00", "1706706120", "2024-01-31
14:07:00", "1706710020", "65", "4", "0", "False", "False", "4", "45+", "BR", "16
50", "none", "MOBA", "34"],
["20", "SES000021", "VWR00994", "STR0051", "SM00792", "2024-02-19
12:10:00", "1708344600", "2024-02-19
12:46:00", "1708346760", "36", "5", "4638", "False", "False", "4643", "25-
34", "RU", "969", "none", "Creative", "91"],
["21", "SES000022", "VWR00846", "STR0075", "SM01182", "2024-03-20
18:00:00", "1710957600", "2024-03-20
18:51:00", "1710960660", "51", "7", "0", "False", "False", "7", "13-
17", "AU", "1556", "tier2", "IRL", "30"],
["22", "SES000023", "VWR00722", "STR0064", "SM01010", "2024-03-09
12:11:00", "1709986260", "2024-03-09
13:17:00", "1709990220", "66", "4", "0", "False", "False", "4", "13-
17", "RU", "1109", "none", "FPS", "38"],
["23", "SES000024", "VWR00979", "STR0070", "SM01108", "2024-02-28
20:44:00", "1709153040", "2024-02-28
20:49:00", "1709153340", "5", "1", "0", "False", "False", "1", "13-
17", "US", "3065", "tier3", "Just Chatting", "73"],
["24", "SES000025", "VWR00616", "STR0024", "SM00367", "2024-03-30
16:15:00", "1711815300", "2024-03-30
16:29:00", "1711816140", "14", "1", "0", "False", "False", "1", "18-
24", "other", "2358", "tier2", "Just Chatting", "22"],
["25", "SES000026", "VWR00387", "STR0065", "SM01021", "2024-03-11
16:06:00", "1710173160", "2024-03-11
18:50:00", "1710183000", "164", "13", "0", "False", "False", "13", "18-
24", null, "3310", "tier1", "FPS", "13"],
["26", "SES000027", "VWR00959", "STR0060", "SM00937", "2024-01-07
20:20:00", "1704658800", "2024-01-07
21:57:00", "1704664620", "97", "0", "6251", "False", "False", "6251", "13-
17", "IT", "466", "none", "Creative", "73"],
["27", "SES000028", "VWR00852", "STR0066", "SM01043", "2024-03-25
16:41:00", "1711384860", "2024-03-25
16:49:00", "1711385340", "8", "0", "0", "False", "False", "0", "18-
24", "US", "318", "none", "Just Chatting", "71"],
["28", "SES000029", "VWR00682", "STR0005", "SM00080", "2024-03-21

22:10:00","1711059000","2024-03-21
22:27:00","1711060020","17","2","0","False","False","2","35-
44","SE","2005","none","Just Chatting","40"],
["29","SES000030","VWR00217","STR0021","SM00327","2024-01-31
14:06:00","1706709960","2024-01-31
14:56:00","1706712960","50","2","0","False","False","2","35-
44","GB","1389","tier1","FPS","3"],
["30","SES000031","VWR00455","STR0054","SM00848","2024-01-23
18:17:00","1706033820","2024-01-23
21:33:00","1706045580","196","1","0","False","False","1","45+","US","2
725","none","Just Chatting","12"],
["31","SES000032","VWR00628","STR0067","SM01056","2024-02-08
09:05:00","1707383100","2024-02-08
11:35:00","1707392100","150","33","3589","False","False","3622","45+","
null","3507","tier2","FPS","32"],
["32","SES000033","VWR00624","STR0006","SM00100","2024-01-08
21:50:00","1704750600","2024-01-08
22:13:00","1704751980","23","5","0","False","False","5","25-
34","GB","3199","none","FPS","28"],
["33","SES000034","VWR00646","STR0071","SM01132","2024-02-18
23:42:00","1708299720","2024-02-19
00:58:00","1708304280","76","4","412","True","False","416","25-
34","RU","1645","tier1","FPS","38"],
["34","SES000035","VWR00878","STR0023","SM00358","2024-03-16
11:47:00","1710589620","2024-03-16
11:55:00","1710590100","8","0","0","False","False","0","35-
44","JP","677","none","Just Chatting","75"],
["35","SES000036","VWR00928","STR0055","SM00864","2024-03-10
08:33:00","1710059580","2024-03-10
08:46:00","1710060360","13","0","0","False","False","0","25-
34","GB","1323","none","IRL","100"],
["36","SES000037","VWR00382","STR0018","SM00271","2024-01-02
21:05:00","1704229500","2024-01-02
23:08:00","1704236880","123","0","0","False","False","0","35-
44","US","2816","tier2","IRL","13"],
["37","SES000038","VWR00980","STR0060","SM00936","2024-02-28
09:47:00","1709113620","2024-02-28
10:52:00","1709117520","65","7","0","False","False","7","18-
24","other","411","tier1","Sports","90"],
["38","SES000039","VWR00772","STR0026","SM00412","2024-01-31
14:11:00","1706710260","2024-01-31
15:06:00","1706713560","55","2","0","False","False","2","18-
24","IN","2571","tier1","FPS","38"],
["39","SES000040","VWR00890","STR0072","SM01137","2024-02-20
18:21:00","1708453260","2024-02-20
20:01:00","1708459260","100","12","0","False","False","12","18-
24","US","3311","none","IRL","87"],
["40","SES000041","VWR00426","STR0049","SM00766","2024-03-07
14:54:00","1709823240","2024-03-07

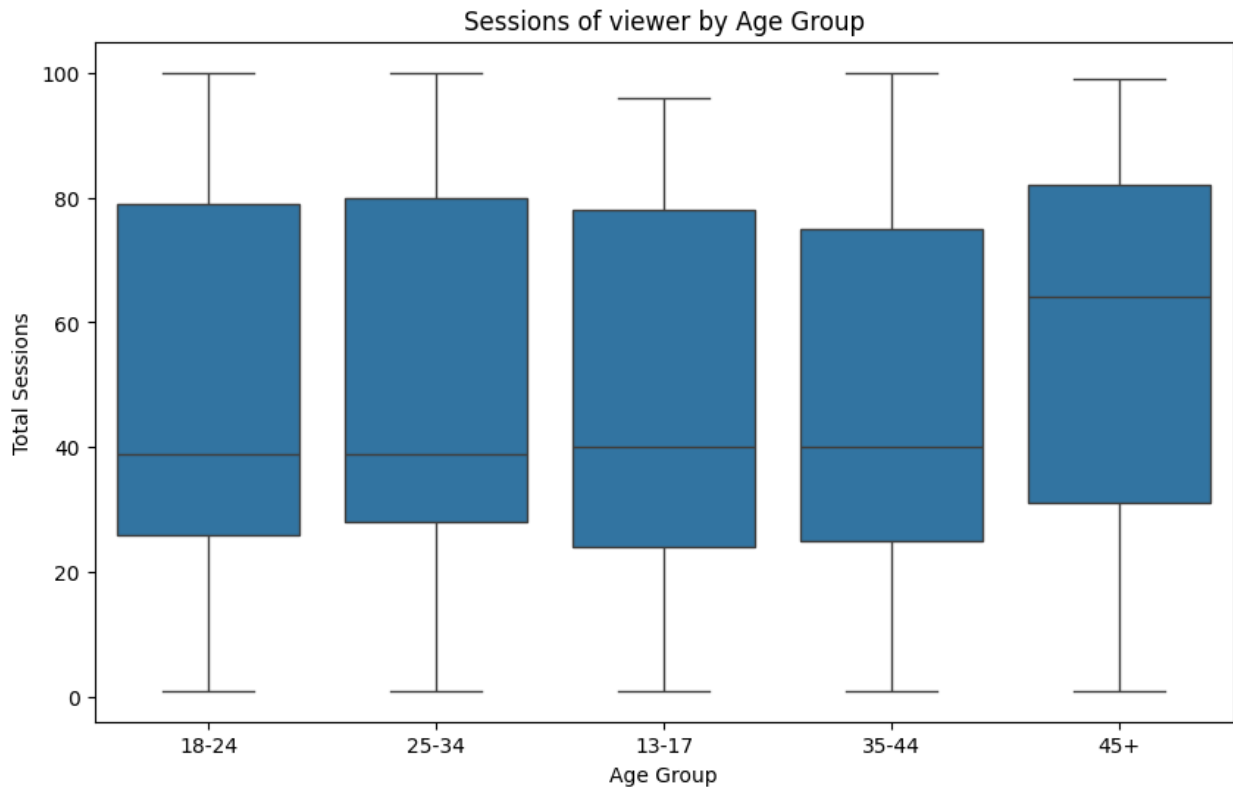
```

15:31:00","1709825460","37","7","0","False","False","7","25-
34",null,"1514","tier1","Just Chatting","15"],
["41","SES000042","VWR00259","STR0024","SM00368","2024-02-19
13:44:00","1708350240","2024-02-19
16:10:00","1708359000","146","3","0","False","False","3","25-
34","US","789","tier1","FPS","14"],
["42","SES000043","VWR00927","STR0003","SM00047","2024-02-17
12:49:00","1708174140","2024-02-17
13:27:00","1708176420","38","0","0","False","False","0","25-
34","BR","3147","tier1","Just Chatting","53"],
["43","SES000044","VWR00969","STR0006","SM00098","2024-03-14
20:36:00","1710448560","2024-03-14
20:46:00","1710449160","10","0","0","False","False","0","35-
44","RU","3464","none","Creative","50"],
["44","SES000045","VWR00978","STR0049","SM00759","2024-01-05
23:16:00","1704496560","2024-01-06
03:12:00","1704510720","236","47","0","False","False","47","18-
24","KR","494","none","FPS","100"],
["45","SES000046","VWR00665","STR0004","SM00069","2024-03-07
08:37:00","1709800620","2024-03-07
10:09:00","1709806140","92","15","0","False","False","15","18-
24","US","2620","none","FPS","34"],
["46","SES000047","VWR00765","STR0055","SM00859","2024-03-23
12:27:00","1711196820","2024-03-23
13:05:00","1711199100","38","1","0","False","False","1","18-
24","GB","2843","none","MOBA","36"],
["47","SES000048","VWR00865","STR0044","SM00695","2024-02-09
16:56:00","1707497760","2024-02-09
17:26:00","1707499560","30","1","0","False","False","1","18-
24","GB","1067","none","IRL","81"],
["48","SES000049","VWR00872","STR0044","SM00701","2024-03-08
12:59:00","1709902740","2024-03-08
14:29:00","1709908140","90","9","254","False","False","263","35-
44","IT","1436","tier1","Just Chatting","100"],
["49","SES000050","VWR00949","STR0030","SM00475","2024-01-11
15:37:00","1704987420","2024-01-11
19:24:00","1705001040","227","7","0","False","False","7","35-
44","CA","3133","none","Sports","75"]], "shape":
{"columns":20,"rows":22825}}

order = df_sessions_with_viewer_age.groupby("age_group")
["sessions_by_viewer"].median().sort_values().index
fig, ax = plt.subplots(figsize=(10,6))
sns.boxplot(data=df_sessions_with_viewer_age, y="sessions_by_viewer",
x="age_group", order=order)
plt.title("Sessions of viewer by Age Group")
plt.xlabel("Age Group")
plt.ylabel("Total Sessions")

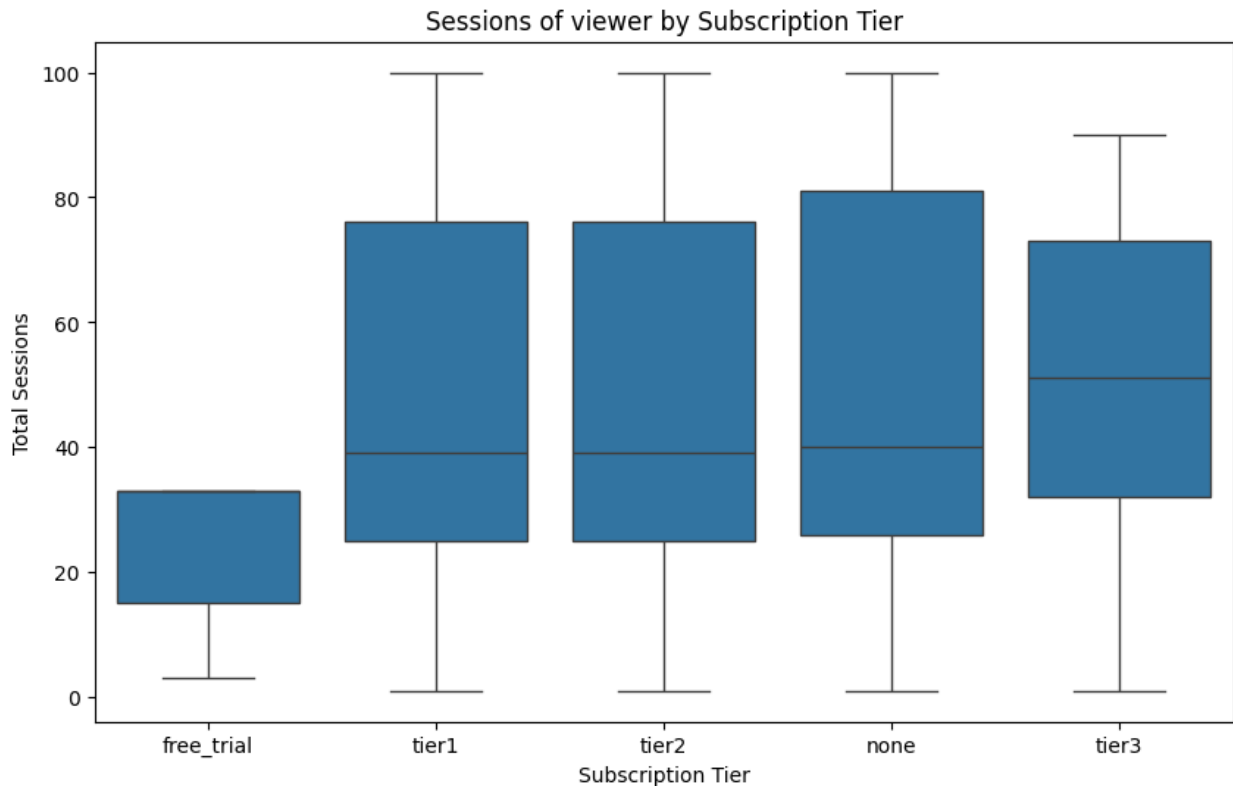
Text(0, 0.5, 'Total Sessions')

```



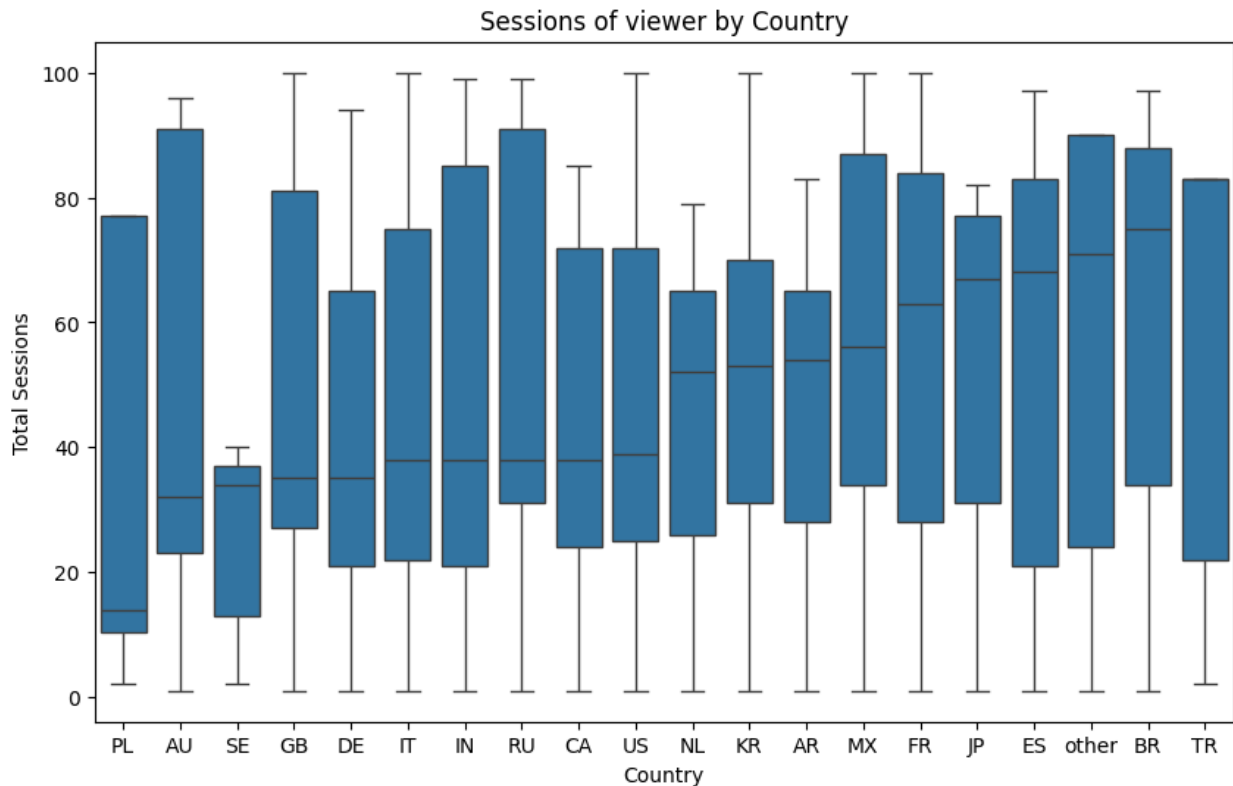
Viewer behaviour on subscription tier

```
order = df_sessions_with_viewer_age.groupby("subscription_tier")
["sessions_by_viewer"].median().sort_values().index
fig, ax = plt.subplots(figsize=(10,6))
sns.boxplot(data=df_sessions_with_viewer_age, y="sessions_by_viewer",
x="subscription_tier", order=order)
plt.title("Sessions of viewer by Subscription Tier")
plt.xlabel("Subscription Tier")
plt.ylabel("Total Sessions")
Text(0, 0.5, 'Total Sessions')
```



Viewer behaviour on country

```
order = df_sessions_with_viewer_age.groupby("country")
["sessions_by_viewer"].median().sort_values().index
fig, ax = plt.subplots(figsize=(10,6))
sns.boxplot(data=df_sessions_with_viewer_age, y="sessions_by_viewer",
x="country", order=order)
plt.title("Sessions of viewer by Country")
plt.xlabel("Country")
plt.ylabel("Total Sessions")
Text(0, 0.5, 'Total Sessions')
```



Task 6 Summary report

Task 7: Your Own EDA Questions

Formulate and explore two hypotheses of your own:

- Propose questions based on curiosity (e.g., "Does a user's variety of categories influence morning vs. evening or session duration?")
- Use summary statistics and visualizations to answer each question
- Briefly discuss what additional data would help you answer the questions more effectively

Does a viewer's age affect their preferred category? How many users in each age x category combination?

```
# task 7 code start here
counts_series = df_sessions_with_viewer_age.groupby(["age_group",
"preferred_category"]).size().reset_index(name="count")
display(counts_series)
heatmap = pd.crosstab(df_sessions_with_viewer_age["age_group"],
df_sessions_with_viewer_age["preferred_category"])
```

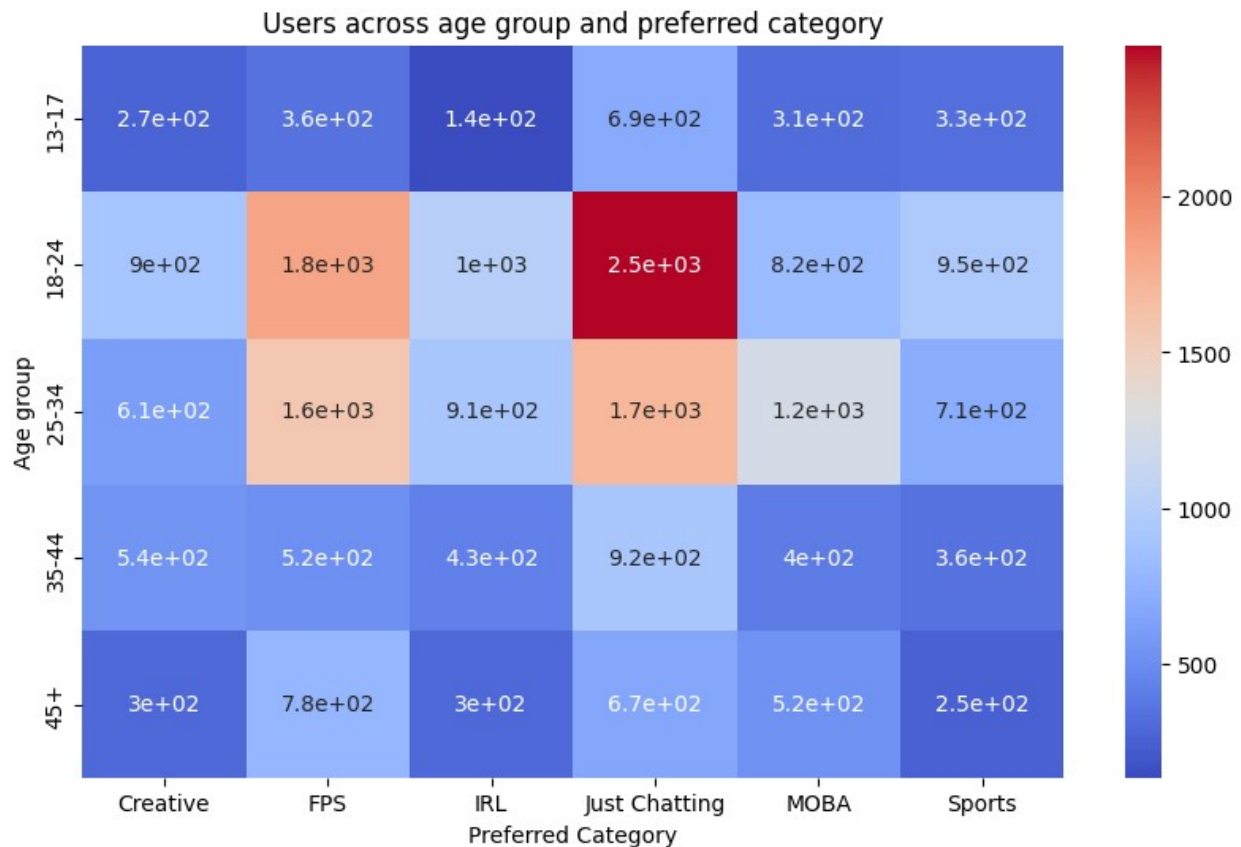
```

display(heatmap)
fig, ax = plt.subplots(figsize=(10,6))
sns.heatmap(heatmap, cmap="coolwarm", annot=True)
plt.title("Users across age group and preferred category")
plt.xlabel("Preferred Category")
plt.ylabel("Age group")
plt.show()

{"columns":[{"name":"index","rawType":"int64","type":"integer"},
{"name":"age_group","rawType":"str","type":"string"},
{"name":"preferred_category","rawType":"str","type":"string"},
{"name":"count","rawType":"int64","type":"integer"}],"ref":"80821089-
e0a7-4217-a441-0d2408552578","rows":[["0","13-17","Creative","269"],
["1","13-17","FPS","355"],["2","13-17","IRL","137"],["3","13-17","Just
Chatting","694"],["4","13-17","MOBA","312"],["5","13-
17","Sports","334"],["6","18-24","Creative","903"],["7","18-
24","FPS","1824"],["8","18-24","IRL","1017"],["9","18-24","Just
Chatting","2484"],["10","18-24","MOBA","820"],["11","18-
24","Sports","950"],["12","25-34","Creative","610"],["13","25-
34","FPS","1585"],["14","25-34","IRL","914"],["15","25-34","Just
Chatting","1701"],["16","25-34","MOBA","1231"],["17","25-
34","Sports","709"],["18","35-44","Creative","541"],["19","35-
44","FPS","518"],["20","35-44","IRL","426"],["21","35-44","Just
Chatting","916"],["22","35-44","MOBA","398"],["23","35-
44","Sports","356"],["24","45+","Creative","300"],
["25","45+","FPS","780"],["26","45+","IRL","302"],["27","45+","Just
Chatting","667"],["28","45+","MOBA","524"],
["29","45+","Sports","248"]],"shape":{"columns":3,"rows":30}}

{"columns":[{"name":"age_group","rawType":"str","type":"string"},
{"name":"Creative","rawType":"int64","type":"integer"},
{"name":"FPS","rawType":"int64","type":"integer"},
{"name":"IRL","rawType":"int64","type":"integer"},{"name":"Just
Chatting","rawType":"int64","type":"integer"},
{"name":"MOBA","rawType":"int64","type":"integer"},
{"name":"Sports","rawType":"int64","type":"integer"}],"ref":"41a2748d-
b26f-41e9-a125-4645cd8633fc","rows":[["13-
17","269","355","137","694","312","334"],["18-
24","903","1824","1017","2484","820","950"],["25-
34","610","1585","914","1701","1231","709"],["35-
44","541","518","426","916","398","356"],
["45+","300","780","302","667","524","248"]],"shape":
{"columns":6,"rows":5}}

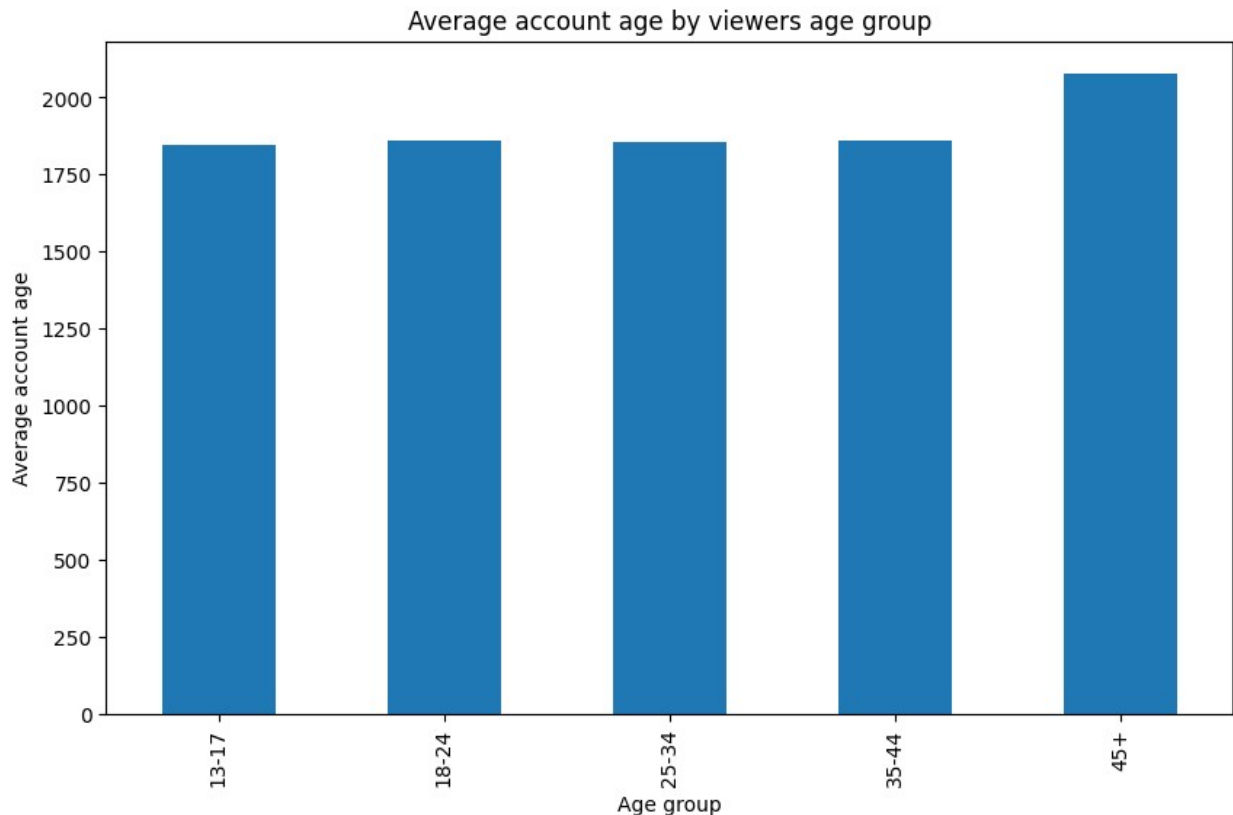
```



What are the mean age for each subscription tier?

```
viewers_by_tier = df_viewers_all_fixed.groupby("age_group")
["account_age_days"].mean()
display(viewers_by_tier)
plt.figure(figsize=(10,6))
viewers_by_tier.plot(kind="bar")
plt.title("Average account age by viewers age group")
plt.xlabel("Age group")
plt.ylabel("Average account age")
plt.show()

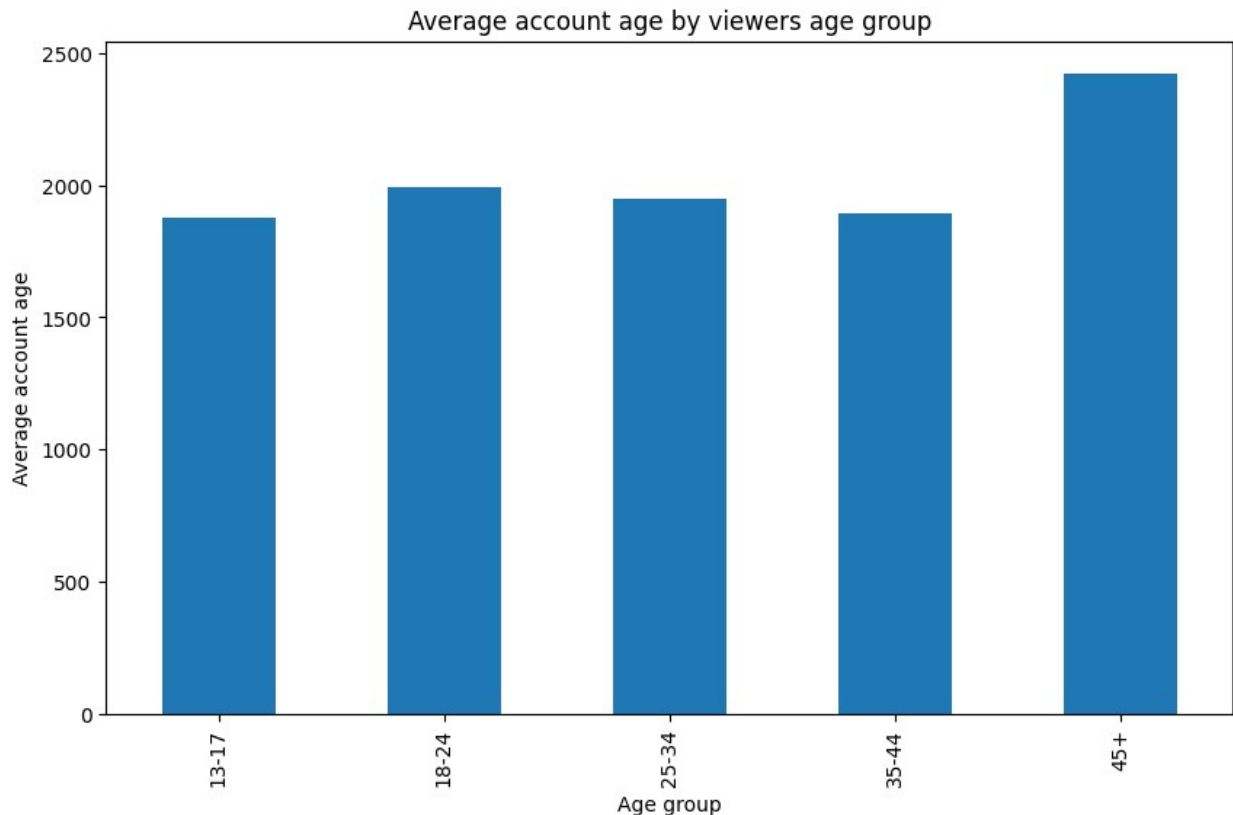
{"columns":[{"name":"age_group","rawType":"str","type":"string"},
{"name":"account_age_days","rawType":"float64","type":"float"}],"ref":
"6fb091c1-409b-47d4-afbfb-1a053ccaff55","rows":[{"13-
17","1845.4421052631578"},["18-24","1860.7247191011236"},["25-
34","1852.5805369127518"},["35-44","1861.1714285714286"],
["45+","2078.837837837838"]],"shape":{"columns":1,"rows":5}}
```



What are the median age for each subscription tier?

```
viewers_by_tier = df_viewers_all_fixed.groupby("age_group")
["account_age_days"].median()
display(viewers_by_tier)
plt.figure(figsize=(10,6))
viewers_by_tier.plot(kind="bar")
plt.title("Average account age by viewers age group")
plt.xlabel("Age group")
plt.ylabel("Average account age")
plt.show()

{"columns":[{"name":"age_group","rawType":"str","type":"string"},
{"name":"account_age_days","rawType":"float64","type":"float"}],"ref":
"0c88500a-f80b-4a75-89b1-47472176fd5b","rows":[["13-17","1875.0"],
["18-24","1990.0"],["25-34","1949.5"],["35-44","1895.0"],
["45+","2424.0"]],"shape":{"columns":1,"rows":5}}
```



Task 7 Summary report

Task 8: EDA Presentation (Insight Report)

Summarize your findings in a polished format:

- Create 3–4 slides or a single-page report highlighting your most important insights
- Include at least 2 charts and interpret them using clear, plain language
- Apply Tufte’s principles: minimize chartjunk, maximize data-to-ink ratio, and ensure clear labeling

Some data detective stuff

Overlapping streams by a single streamer

I want to see if there are any concurrent/overlapping streams by a certain streamer.

```

count_overlapping_streams = 0
df_streams_streamer = df_streams.groupby("streamer_id")
overlapping_streams = {"streamer_id":[], "stream_id":[], "start_time":
[], "end_time":[]}
df_overlapping_streams = pd.DataFrame(overlapping_streams)
for group, dataframe in df_streams_streamer:
    for row in dataframe.itertuples():
        start = row.started_at
        end = row.ended_at
        for row_to_compare in dataframe.itertuples():
            if (row.stream_id != row_to_compare.stream_id):
                start_compare = row_to_compare.started_at
                end_compare = row_to_compare.ended_at
                i1 = pd.Interval(pd.to_datetime(start),
pd.to_datetime(end))
                i2 = pd.Interval(pd.to_datetime(start_compare),
pd.to_datetime(end_compare))
                if (i1.overlaps(i2)):
                    # print(f"There is an overlapping stream, by
streamer {row.streamer_id} = {row_to_compare.streamer_id}. Stream ids
are: {row.stream_id} and {row_to_compare.stream_id}.")
                    count_overlapping_streams +=1
                    new_row = [row.streamer_id, row.stream_id,
row.started_at, row.ended_at]

df_overlapping_streams.loc[len(df_overlapping_streams)] = new_row

print(f"\nThere are total {count_overlapping_streams} overlapping
streams")
display(df_overlapping_streams)

overlapping_timelines = px.timeline(data_frame=df_overlapping_streams,
y=df_overlapping_streams["streamer_id"],
x_start=df_overlapping_streams["start_time"],
x_end=df_overlapping_streams["end_time"],
color=df_overlapping_streams["stream_id"], width = 1000, height = 600,
opacity=0.2)
overlapping_timelines.show()

```

There are total 56 overlapping streams

```

{"columns":[{"name":"index","rawType":"int64","type":"integer"},
{"name":"streamer_id","rawType":"str","type":"string"},
{"name":"stream_id","rawType":"str","type":"string"},
{"name":"start_time","rawType":"str","type":"string"},
{"name":"end_time","rawType":"str","type":"string"}],"ref":"831f6958-
6856-4889-9d5f-08bb881ee32a","rows":[["0","STR0001","SM00002","2024-
01-28 18:00:00","2024-01-28 21:00:00"],["1","STR0001","SM00009","2024-

```

01-28 17:00:00", "2024-01-28 21:00:00"], ["2", "STR0007", "SM00105", "2024-02-18 19:00:00", "2024-02-18 23:00:00"], ["3", "STR0007", "SM00107", "2024-02-18 18:00:00", "2024-02-18 22:00:00"], ["4", "STR0008", "SM00117", "2024-03-07 16:00:00", "2024-03-07 22:00:00"], ["5", "STR0008", "SM00118", "2024-03-07 15:00:00", "2024-03-07 20:00:00"], ["6", "STR0008", "SM00119", "2024-03-16 13:00:00", "2024-03-16 15:00:00"], ["7", "STR0008", "SM00130", "2024-03-16 13:00:00", "2024-03-16 15:00:00"], ["8", "STR0009", "SM00133", "2024-01-18 15:00:00", "2024-01-18 19:00:00"], ["9", "STR0009", "SM00142", "2024-01-18 11:00:00", "2024-01-18 17:00:00"], ["10", "STR0015", "SM00217", "2024-01-22 11:00:00", "2024-01-22 14:00:00"], ["11", "STR0015", "SM00228", "2024-01-22 10:00:00", "2024-01-22 15:00:00"], ["12", "STR0015", "SM00230", "2024-02-16 08:00:00", "2024-02-16 10:30:00"], ["13", "STR0015", "SM00237", "2024-02-16 10:00:00", "2024-02-16 12:30:00"], ["14", "STR0016", "SM00238", "2024-02-03 16:00:00", "2024-02-03 21:00:00"], ["15", "STR0016", "SM00239", "2024-02-03 20:00:00", "2024-02-04 01:00:00"], ["16", "STR0016", "SM00255", "2024-01-05 21:00:00", "2024-01-06 02:00:00"], ["17", "STR0016", "SM00256", "2024-01-05 19:00:00", "2024-01-06 00:00:00"], ["18", "STR0017", "SM00264", "2024-02-10 12:00:00", "2024-02-10 17:00:00"], ["19", "STR0017", "SM00265", "2024-02-10 15:00:00", "2024-02-10 19:00:00"], ["20", "STR0018", "SM00275", "2024-02-18 16:00:00", "2024-02-18 23:00:00"], ["21", "STR0018", "SM00278", "2024-02-18 14:00:00", "2024-02-18 20:00:00"], ["22", "STR0021", "SM00322", "2024-01-16 20:00:00", "2024-01-16 23:00:00"], ["23", "STR0021", "SM00334", "2024-01-16 20:00:00", "2024-01-16 23:00:00"], ["24", "STR0026", "SM00402", "2024-02-14 14:00:00", "2024-02-14 16:30:00"], ["25", "STR0026", "SM00420", "2024-02-14 14:00:00", "2024-02-14 18:00:00"], ["26", "STR0029", "SM00467", "2024-03-07 21:00:00", "2024-03-07 23:30:00"], ["27", "STR0029", "SM00470", "2024-03-07 21:00:00", "2024-03-08 00:00:00"], ["28", "STR0035", "SM00556", "2024-03-29 10:00:00", "2024-03-29 13:00:00"], ["29", "STR0035", "SM00559", "2024-03-29 11:00:00", "2024-03-29 15:00:00"], ["30", "STR0036", "SM00574", "2024-02-01 08:00:00", "2024-02-01 14:00:00"], ["31", "STR0036", "SM00579", "2024-03-09 16:00:00", "2024-03-09 22:00:00"], ["32", "STR0036", "SM00581", "2024-03-09 19:00:00", "2024-03-10 01:00:00"], ["33", "STR0036", "SM00586", "2024-02-01 09:00:00", "2024-02-01 11:30:00"], ["34", "STR0038", "SM00604", "2024-01-30 13:00:00", "2024-01-30 16:00:00"], ["35", "STR0038", "SM00612", "2024-01-30 13:00:00", "2024-01-30 16:00:00"], ["36", "STR0047", "SM00731", "2024-02-05 15:00:00", "2024-02-05 19:00:00"], ["37", "STR0047", "SM00732", "2024-02-05 15:00:00", "2024-02-05 19:00:00"], ["38", "STR0048", "SM00743", "2024-01-23 12:00:00", "2024-01-23 14:00:00"], ["39", "STR0048", "SM00746", "2024-01-23 13:00:00", "2024-01-23 16:00:00"], ["40", "STR0049", "SM00765", "2024-02-10 13:00:00", "2024-02-10 18:00:00"], ["41", "STR0049", "SM00769", "2024-02-10 16:00:00", "2024-02-10 18:00:00"], ["42", "STR0052", "SM00817", "2024-01-12 11:00:00", "2024-01-12 15:00:00"], ["43", "STR0052", "SM00824", "2024-01-12 11:00:00", "2024-01-12 18:00:00"], ["44", "STR0059", "SM00913", "2024-02-17 16:00:00", "2024-02-17 20:00:00"], ["45", "STR0059", "SM00920", "2024-02-17 16:00:00", "2024-02-17 18:30:00"], ["46", "STR0059", "SM00921", "2024-01-17 13:00:00", "2024-01-17 18:00:00"], ["47", "STR0059", "SM00927", "2024-01-17 13:00:00", "2024-01-17 17:00:00"], ["48", "STR0063", "SM00993", "2024-01-24 08:00:00", "2024-01-24 17:00:00"]

```
16:00:00"], ["49", "STR0063", "SM00995", "2024-01-24 09:00:00", "2024-01-24  
13:00:00"]], "shape": {"columns": 4, "rows": 56}}
```

```
{"config": {"plotlyServerURL": "https://plot.ly"}, "data": [{"base":  
["2024-01-  
28T18:00:00.000000"], "hovertemplate": "stream_id=SM00002<br>start_time=  
%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></  
extra>", "legendgroup": "SM00002", "marker":  
{"color": "#636efa", "opacity": 0.2, "pattern":  
{"shape": ""}, "name": "SM00002", "orientation": "h", "showlegend": true, "te  
xtposition": "auto", "type": "bar", "x":  
{"bdata": "gMukAA==", "dtype": "i4"}, "xaxis": "x", "y":  
["STR0001"], "yaxis": "y"}, {"base": ["2024-01-  
28T17:00:00.000000"], "hovertemplate": "stream_id=SM00009<br>start_time=  
%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></  
extra>", "legendgroup": "SM00009", "marker":  
{"color": "#EF553B", "opacity": 0.2, "pattern":  
{"shape": ""}, "name": "SM00009", "orientation": "h", "showlegend": true, "te  
xtposition": "auto", "type": "bar", "x":  
{"bdata": "ALrbAA==", "dtype": "i4"}, "xaxis": "x", "y":  
["STR0001"], "yaxis": "y"}, {"base": ["2024-02-  
18T19:00:00.000000"], "hovertemplate": "stream_id=SM00105<br>start_time=  
%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></  
extra>", "legendgroup": "SM00105", "marker":  
{"color": "#00cc96", "opacity": 0.2, "pattern":  
{"shape": ""}, "name": "SM00105", "orientation": "h", "showlegend": true, "te  
xtposition": "auto", "type": "bar", "x":  
{"bdata": "ALrbAA==", "dtype": "i4"}, "xaxis": "x", "y":  
["STR0007"], "yaxis": "y"}, {"base": ["2024-02-  
18T18:00:00.000000"], "hovertemplate": "stream_id=SM00107<br>start_time=  
%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></  
extra>", "legendgroup": "SM00107", "marker":  
{"color": "#ab63fa", "opacity": 0.2, "pattern":  
{"shape": ""}, "name": "SM00107", "orientation": "h", "showlegend": true, "te  
xtposition": "auto", "type": "bar", "x":  
{"bdata": "ALrbAA==", "dtype": "i4"}, "xaxis": "x", "y":  
["STR0007"], "yaxis": "y"}, {"base": ["2024-03-  
07T16:00:00.000000"], "hovertemplate": "stream_id=SM00117<br>start_time=  
%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></  
extra>", "legendgroup": "SM00117", "marker":  
{"color": "#FFA15A", "opacity": 0.2, "pattern":  
{"shape": ""}, "name": "SM00117", "orientation": "h", "showlegend": true, "te  
xtposition": "auto", "type": "bar", "x":  
{"bdata": "AJdJAQ==", "dtype": "i4"}, "xaxis": "x", "y":  
["STR0008"], "yaxis": "y"}, {"base": ["2024-03-  
07T15:00:00.000000"], "hovertemplate": "stream_id=SM00118<br>start_time=  
%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></  
extra>", "legendgroup": "SM00118", "marker":  
{"color": "#19d3f3", "opacity": 0.2, "pattern":  
{"shape": ""}, "name": "SM00118", "orientation": "h", "showlegend": true, "te
```

```
xtposition":"auto","type":"bar","x":
{"bdata":"gKgSAQ==","dtype":"i4"},"xaxis":"x","y":
["STR0008"],"yaxis":"y"},{"base":["2024-03-
16T13:00:00.000000"],"hovertemplate":"stream_id=SM00119<br>start_time=
%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></
extra>","legendgroup":"SM00119","marker":
{"color":"#FF6692","opacity":0.2,"pattern":
{"shape":""},"name":"SM00119","orientation":"h","showlegend":true,"te
xtposition":"auto","type":"bar","x":
{"bdata":"AN1tAA==","dtype":"i4"},"xaxis":"x","y":
["STR0008"],"yaxis":"y"},{"base":["2024-03-
16T13:00:00.000000"],"hovertemplate":"stream_id=SM00130<br>start_time=
%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></
extra>","legendgroup":"SM00130","marker":
{"color":"#B6E880","opacity":0.2,"pattern":
{"shape":""},"name":"SM00130","orientation":"h","showlegend":true,"te
xtposition":"auto","type":"bar","x":
{"bdata":"AN1tAA==","dtype":"i4"},"xaxis":"x","y":
["STR0008"],"yaxis":"y"},{"base":["2024-01-
18T15:00:00.000000"],"hovertemplate":"stream_id=SM00133<br>start_time=
%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></
extra>","legendgroup":"SM00133","marker":
{"color":"#FF97FF","opacity":0.2,"pattern":
{"shape":""},"name":"SM00133","orientation":"h","showlegend":true,"te
xtposition":"auto","type":"bar","x":
{"bdata":"ALrbAA==","dtype":"i4"},"xaxis":"x","y":
["STR0009"],"yaxis":"y"},{"base":["2024-01-
18T11:00:00.000000"],"hovertemplate":"stream_id=SM00142<br>start_time=
%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></
extra>","legendgroup":"SM00142","marker":
{"color":"#FECB52","opacity":0.2,"pattern":
{"shape":""},"name":"SM00142","orientation":"h","showlegend":true,"te
xtposition":"auto","type":"bar","x":
{"bdata":"AJdJAQ==","dtype":"i4"},"xaxis":"x","y":
["STR0009"],"yaxis":"y"},{"base":["2024-01-
22T11:00:00.000000"],"hovertemplate":"stream_id=SM00217<br>start_time=
%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></
extra>","legendgroup":"SM00217","marker":
{"color":"#636efa","opacity":0.2,"pattern":
{"shape":""},"name":"SM00217","orientation":"h","showlegend":true,"te
xtposition":"auto","type":"bar","x":
{"bdata":"gMukAA==","dtype":"i4"},"xaxis":"x","y":
["STR0015"],"yaxis":"y"},{"base":["2024-01-
22T10:00:00.000000"],"hovertemplate":"stream_id=SM00228<br>start_time=
%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></
extra>","legendgroup":"SM00228","marker":
{"color":"#EF553B","opacity":0.2,"pattern":
{"shape":""},"name":"SM00228","orientation":"h","showlegend":true,"te
xtposition":"auto","type":"bar","x":
{"bdata":"gKgSAQ==","dtype":"i4"},"xaxis":"x","y":
```

```

["STR0015"], "yaxis": "y"}, {"base": ["2024-02-16T08:00:00.000000"], "hovertemplate": "stream_id=SM00230<br>start_time=%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></extra>", "legendgroup": "SM00230", "marker": {"color": "#00cc96", "opacity": 0.2, "pattern": {"shape": ""}}, "name": "SM00230", "orientation": "h", "showlegend": true, "textposition": "auto", "type": "bar", "x": {"bdata": "QFSJAA==", "dtype": "i4"}, "xaxis": "x", "y": ["STR0015"], "yaxis": "y"}, {"base": ["2024-02-16T10:00:00.000000"], "hovertemplate": "stream_id=SM00237<br>start_time=%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></extra>", "legendgroup": "SM00237", "marker": {"color": "#ab63fa", "opacity": 0.2, "pattern": {"shape": ""}}, "name": "SM00237", "orientation": "h", "showlegend": true, "textposition": "auto", "type": "bar", "x": {"bdata": "QFSJAA==", "dtype": "i4"}, "xaxis": "x", "y": ["STR0015"], "yaxis": "y"}, {"base": ["2024-02-03T16:00:00.000000"], "hovertemplate": "stream_id=SM00238<br>start_time=%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></extra>", "legendgroup": "SM00238", "marker": {"color": "#FFA15A", "opacity": 0.2, "pattern": {"shape": ""}}, "name": "SM00238", "orientation": "h", "showlegend": true, "textposition": "auto", "type": "bar", "x": {"bdata": "gKgSAQ==", "dtype": "i4"}, "xaxis": "x", "y": ["STR0016"], "yaxis": "y"}, {"base": ["2024-02-03T20:00:00.000000"], "hovertemplate": "stream_id=SM00239<br>start_time=%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></extra>", "legendgroup": "SM00239", "marker": {"color": "#19d3f3", "opacity": 0.2, "pattern": {"shape": ""}}, "name": "SM00239", "orientation": "h", "showlegend": true, "textposition": "auto", "type": "bar", "x": {"bdata": "gKgSAQ==", "dtype": "i4"}, "xaxis": "x", "y": ["STR0016"], "yaxis": "y"}, {"base": ["2024-01-05T21:00:00.000000"], "hovertemplate": "stream_id=SM00255<br>start_time=%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></extra>", "legendgroup": "SM00255", "marker": {"color": "#FF6692", "opacity": 0.2, "pattern": {"shape": ""}}, "name": "SM00255", "orientation": "h", "showlegend": true, "textposition": "auto", "type": "bar", "x": {"bdata": "gKgSAQ==", "dtype": "i4"}, "xaxis": "x", "y": ["STR0016"], "yaxis": "y"}, {"base": ["2024-01-05T19:00:00.000000"], "hovertemplate": "stream_id=SM00256<br>start_time=%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></extra>", "legendgroup": "SM00256", "marker": {"color": "#B6E880", "opacity": 0.2, "pattern": {"shape": ""}}, "name": "SM00256", "orientation": "h", "showlegend": true, "textposition": "auto", "type": "bar", "x": {"bdata": "gKgSAQ==", "dtype": "i4"}, "xaxis": "x", "y": ["STR0016"], "yaxis": "y"}, {"base": ["2024-02-10T12:00:00.000000"], "hovertemplate": "stream_id=SM00264<br>start_time=

```

```

%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></
extra>", "legendgroup": "SM00264", "marker":
{"color": "#FF97FF", "opacity": 0.2, "pattern":
{"shape": ""}}, "name": "SM00264", "orientation": "h", "showlegend": true, "te
xtposition": "auto", "type": "bar", "x":
{"bdata": "gKgSAQ==", "dtype": "i4"}, "xaxis": "x", "y":
["STR0017"], "yaxis": "y"}, {"base": ["2024-02-
10T15:00:00.000000"], "hovertemplate": "stream_id=SM00265<br>start_time=
%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></
extra>", "legendgroup": "SM00265", "marker":
{"color": "#FECB52", "opacity": 0.2, "pattern":
{"shape": ""}}, "name": "SM00265", "orientation": "h", "showlegend": true, "te
xtposition": "auto", "type": "bar", "x":
{"bdata": "ALrbAA==", "dtype": "i4"}, "xaxis": "x", "y":
["STR0017"], "yaxis": "y"}, {"base": ["2024-02-
18T16:00:00.000000"], "hovertemplate": "stream_id=SM00275<br>start_time=
%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></
extra>", "legendgroup": "SM00275", "marker":
{"color": "#636efa", "opacity": 0.2, "pattern":
{"shape": ""}}, "name": "SM00275", "orientation": "h", "showlegend": true, "te
xtposition": "auto", "type": "bar", "x":
{"bdata": "gIWAAQ==", "dtype": "i4"}, "xaxis": "x", "y":
["STR0018"], "yaxis": "y"}, {"base": ["2024-02-
18T14:00:00.000000"], "hovertemplate": "stream_id=SM00278<br>start_time=
%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></
extra>", "legendgroup": "SM00278", "marker":
{"color": "#EF553B", "opacity": 0.2, "pattern":
{"shape": ""}}, "name": "SM00278", "orientation": "h", "showlegend": true, "te
xtposition": "auto", "type": "bar", "x":
{"bdata": "AJdJAQ==", "dtype": "i4"}, "xaxis": "x", "y":
["STR0018"], "yaxis": "y"}, {"base": ["2024-01-
16T20:00:00.000000"], "hovertemplate": "stream_id=SM00322<br>start_time=
%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></
extra>", "legendgroup": "SM00322", "marker":
{"color": "#00cc96", "opacity": 0.2, "pattern":
{"shape": ""}}, "name": "SM00322", "orientation": "h", "showlegend": true, "te
xtposition": "auto", "type": "bar", "x":
{"bdata": "gMukAA==", "dtype": "i4"}, "xaxis": "x", "y":
["STR0021"], "yaxis": "y"}, {"base": ["2024-01-
16T20:00:00.000000"], "hovertemplate": "stream_id=SM00334<br>start_time=
%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></
extra>", "legendgroup": "SM00334", "marker":
{"color": "#ab63fa", "opacity": 0.2, "pattern":
{"shape": ""}}, "name": "SM00334", "orientation": "h", "showlegend": true, "te
xtposition": "auto", "type": "bar", "x":
{"bdata": "gMukAA==", "dtype": "i4"}, "xaxis": "x", "y":
["STR0021"], "yaxis": "y"}, {"base": ["2024-02-
14T14:00:00.000000"], "hovertemplate": "stream_id=SM00402<br>start_time=
%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></
extra>", "legendgroup": "SM00402", "marker":

```

```

{"color": "#FFA15A", "opacity": 0.2, "pattern":
{"shape": ""}, "name": "SM00402", "orientation": "h", "showlegend": true, "te
xtposition": "auto", "type": "bar", "x":
{"bdata": "QFSJAA==", "dtype": "i4"}, "xaxis": "x", "y":
["STR0026"], "yaxis": "y"}, {"base": ["2024-02-
14T14:00:00.000000"], "hovertemplate": "stream_id=SM00420<br>start_time=
%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></
extra>", "legendgroup": "SM00420", "marker":
{"color": "#19d3f3", "opacity": 0.2, "pattern":
{"shape": ""}, "name": "SM00420", "orientation": "h", "showlegend": true, "te
xtposition": "auto", "type": "bar", "x":
{"bdata": "ALrbAA==", "dtype": "i4"}, "xaxis": "x", "y":
["STR0026"], "yaxis": "y"}, {"base": ["2024-03-
07T21:00:00.000000"], "hovertemplate": "stream_id=SM00467<br>start_time=
%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></
extra>", "legendgroup": "SM00467", "marker":
{"color": "#FF6692", "opacity": 0.2, "pattern":
{"shape": ""}, "name": "SM00467", "orientation": "h", "showlegend": true, "te
xtposition": "auto", "type": "bar", "x":
{"bdata": "QFSJAA==", "dtype": "i4"}, "xaxis": "x", "y":
["STR0029"], "yaxis": "y"}, {"base": ["2024-03-
07T21:00:00.000000"], "hovertemplate": "stream_id=SM00470<br>start_time=
%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></
extra>", "legendgroup": "SM00470", "marker":
{"color": "#B6E880", "opacity": 0.2, "pattern":
{"shape": ""}, "name": "SM00470", "orientation": "h", "showlegend": true, "te
xtposition": "auto", "type": "bar", "x":
{"bdata": "gMukAA==", "dtype": "i4"}, "xaxis": "x", "y":
["STR0029"], "yaxis": "y"}, {"base": ["2024-03-
29T10:00:00.000000"], "hovertemplate": "stream_id=SM00556<br>start_time=
%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></
extra>", "legendgroup": "SM00556", "marker":
{"color": "#FF97FF", "opacity": 0.2, "pattern":
{"shape": ""}, "name": "SM00556", "orientation": "h", "showlegend": true, "te
xtposition": "auto", "type": "bar", "x":
{"bdata": "gMukAA==", "dtype": "i4"}, "xaxis": "x", "y":
["STR0035"], "yaxis": "y"}, {"base": ["2024-03-
29T11:00:00.000000"], "hovertemplate": "stream_id=SM00559<br>start_time=
%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></
extra>", "legendgroup": "SM00559", "marker":
{"color": "#FECB52", "opacity": 0.2, "pattern":
{"shape": ""}, "name": "SM00559", "orientation": "h", "showlegend": true, "te
xtposition": "auto", "type": "bar", "x":
{"bdata": "ALrbAA==", "dtype": "i4"}, "xaxis": "x", "y":
["STR0035"], "yaxis": "y"}, {"base": ["2024-02-
01T08:00:00.000000"], "hovertemplate": "stream_id=SM00574<br>start_time=
%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></
extra>", "legendgroup": "SM00574", "marker":
{"color": "#636efa", "opacity": 0.2, "pattern":
{"shape": ""}, "name": "SM00574", "orientation": "h", "showlegend": true, "te

```

```

xtposition":"auto","type":"bar","x":
{"bdata":"AJdJAQ==","dtype":"i4"},"xaxis":"x","y":
["STR0036"],"yaxis":"y"},{"base":["2024-03-
09T16:00:00.000000"],"hovertemplate":"stream_id=SM00579<br>start_time=
%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></
extra>","legendgroup":"SM00579","marker":
{"color":"#EF553B","opacity":0.2,"pattern":
{"shape":""},"name":"SM00579","orientation":"h","showlegend":true,"te
xtposition":"auto","type":"bar","x":
{"bdata":"AJdJAQ==","dtype":"i4"},"xaxis":"x","y":
["STR0036"],"yaxis":"y"},{"base":["2024-03-
09T19:00:00.000000"],"hovertemplate":"stream_id=SM00581<br>start_time=
%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></
extra>","legendgroup":"SM00581","marker":
{"color":"#00cc96","opacity":0.2,"pattern":
{"shape":""},"name":"SM00581","orientation":"h","showlegend":true,"te
xtposition":"auto","type":"bar","x":
{"bdata":"AJdJAQ==","dtype":"i4"},"xaxis":"x","y":
["STR0036"],"yaxis":"y"},{"base":["2024-02-
01T09:00:00.000000"],"hovertemplate":"stream_id=SM00586<br>start_time=
%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></
extra>","legendgroup":"SM00586","marker":
{"color":"#ab63fa","opacity":0.2,"pattern":
{"shape":""},"name":"SM00586","orientation":"h","showlegend":true,"te
xtposition":"auto","type":"bar","x":
{"bdata":"QFSJAA==","dtype":"i4"},"xaxis":"x","y":
["STR0036"],"yaxis":"y"},{"base":["2024-01-
30T13:00:00.000000"],"hovertemplate":"stream_id=SM00604<br>start_time=
%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></
extra>","legendgroup":"SM00604","marker":
{"color":"#FFA15A","opacity":0.2,"pattern":
{"shape":""},"name":"SM00604","orientation":"h","showlegend":true,"te
xtposition":"auto","type":"bar","x":
{"bdata":"gMukAA==","dtype":"i4"},"xaxis":"x","y":
["STR0038"],"yaxis":"y"},{"base":["2024-01-
30T13:00:00.000000"],"hovertemplate":"stream_id=SM00612<br>start_time=
%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></
extra>","legendgroup":"SM00612","marker":
{"color":"#19d3f3","opacity":0.2,"pattern":
{"shape":""},"name":"SM00612","orientation":"h","showlegend":true,"te
xtposition":"auto","type":"bar","x":
{"bdata":"gMukAA==","dtype":"i4"},"xaxis":"x","y":
["STR0038"],"yaxis":"y"},{"base":["2024-02-
05T15:00:00.000000"],"hovertemplate":"stream_id=SM00731<br>start_time=
%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></
extra>","legendgroup":"SM00731","marker":
{"color":"#FF6692","opacity":0.2,"pattern":
{"shape":""},"name":"SM00731","orientation":"h","showlegend":true,"te
xtposition":"auto","type":"bar","x":
{"bdata":"ALrbAA==","dtype":"i4"},"xaxis":"x","y":

```

```
[ "STR0047" ], "yaxis": "y" }, { "base": [ "2024-02-05T15:00:00.000000" ], "hovertemplate": "stream_id=SM00732<br>start_time=%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></extra>", "legendgroup": "SM00732", "marker": { "color": "#B6E880", "opacity": 0.2, "pattern": { "shape": "" }, "name": "SM00732", "orientation": "h", "showlegend": true, "textposition": "auto", "type": "bar", "x": { "bdata": "ALrbAA==", "dtype": "i4" }, "xaxis": "x", "y": [ "STR0047" ], "yaxis": "y" }, { "base": [ "2024-01-23T12:00:00.000000" ], "hovertemplate": "stream_id=SM00743<br>start_time=%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></extra>", "legendgroup": "SM00743", "marker": { "color": "#FF97FF", "opacity": 0.2, "pattern": { "shape": "" }, "name": "SM00743", "orientation": "h", "showlegend": true, "textposition": "auto", "type": "bar", "x": { "bdata": "ANltAA==", "dtype": "i4" }, "xaxis": "x", "y": [ "STR0048" ], "yaxis": "y" }, { "base": [ "2024-01-23T13:00:00.000000" ], "hovertemplate": "stream_id=SM00746<br>start_time=%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></extra>", "legendgroup": "SM00746", "marker": { "color": "#FECB52", "opacity": 0.2, "pattern": { "shape": "" }, "name": "SM00746", "orientation": "h", "showlegend": true, "textposition": "auto", "type": "bar", "x": { "bdata": "gMukAA==", "dtype": "i4" }, "xaxis": "x", "y": [ "STR0048" ], "yaxis": "y" }, { "base": [ "2024-02-10T13:00:00.000000" ], "hovertemplate": "stream_id=SM00765<br>start_time=%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></extra>", "legendgroup": "SM00765", "marker": { "color": "#636efa", "opacity": 0.2, "pattern": { "shape": "" }, "name": "SM00765", "orientation": "h", "showlegend": true, "textposition": "auto", "type": "bar", "x": { "bdata": "gKgSAQ==", "dtype": "i4" }, "xaxis": "x", "y": [ "STR0049" ], "yaxis": "y" }, { "base": [ "2024-02-10T16:00:00.000000" ], "hovertemplate": "stream_id=SM00769<br>start_time=%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></extra>", "legendgroup": "SM00769", "marker": { "color": "#EF553B", "opacity": 0.2, "pattern": { "shape": "" }, "name": "SM00769", "orientation": "h", "showlegend": true, "textposition": "auto", "type": "bar", "x": { "bdata": "ANltAA==", "dtype": "i4" }, "xaxis": "x", "y": [ "STR0049" ], "yaxis": "y" }, { "base": [ "2024-01-12T11:00:00.000000" ], "hovertemplate": "stream_id=SM00817<br>start_time=%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></extra>", "legendgroup": "SM00817", "marker": { "color": "#00cc96", "opacity": 0.2, "pattern": { "shape": "" }, "name": "SM00817", "orientation": "h", "showlegend": true, "textposition": "auto", "type": "bar", "x": { "bdata": "ALrbAA==", "dtype": "i4" }, "xaxis": "x", "y": [ "STR0052" ], "yaxis": "y" }, { "base": [ "2024-01-12T11:00:00.000000" ], "hovertemplate": "stream_id=SM00824<br>start_time=
```

```

%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></
extra>", "legendgroup": "SM00824", "marker":
{"color": "#ab63fa", "opacity": 0.2, "pattern":
{"shape": ""}}, "name": "SM00824", "orientation": "h", "showlegend": true, "te
xtposition": "auto", "type": "bar", "x":
{"bdata": "gIWAAQ==", "dtype": "i4"}, "xaxis": "x", "y":
["STR0052"], "yaxis": "y"}, {"base": ["2024-02-
17T16:00:00.000000"], "hovertemplate": "stream_id=SM00913<br>start_time=
%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></
extra>", "legendgroup": "SM00913", "marker":
{"color": "#FFA15A", "opacity": 0.2, "pattern":
{"shape": ""}}, "name": "SM00913", "orientation": "h", "showlegend": true, "te
xtposition": "auto", "type": "bar", "x":
{"bdata": "ALrbAA==", "dtype": "i4"}, "xaxis": "x", "y":
["STR0059"], "yaxis": "y"}, {"base": ["2024-02-
17T16:00:00.000000"], "hovertemplate": "stream_id=SM00920<br>start_time=
%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></
extra>", "legendgroup": "SM00920", "marker":
{"color": "#19d3f3", "opacity": 0.2, "pattern":
{"shape": ""}}, "name": "SM00920", "orientation": "h", "showlegend": true, "te
xtposition": "auto", "type": "bar", "x":
{"bdata": "QFSJAA==", "dtype": "i4"}, "xaxis": "x", "y":
["STR0059"], "yaxis": "y"}, {"base": ["2024-01-
17T13:00:00.000000"], "hovertemplate": "stream_id=SM00921<br>start_time=
%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></
extra>", "legendgroup": "SM00921", "marker":
{"color": "#FF6692", "opacity": 0.2, "pattern":
{"shape": ""}}, "name": "SM00921", "orientation": "h", "showlegend": true, "te
xtposition": "auto", "type": "bar", "x":
{"bdata": "gKgSAQ==", "dtype": "i4"}, "xaxis": "x", "y":
["STR0059"], "yaxis": "y"}, {"base": ["2024-01-
17T13:00:00.000000"], "hovertemplate": "stream_id=SM00927<br>start_time=
%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></
extra>", "legendgroup": "SM00927", "marker":
{"color": "#B6E880", "opacity": 0.2, "pattern":
{"shape": ""}}, "name": "SM00927", "orientation": "h", "showlegend": true, "te
xtposition": "auto", "type": "bar", "x":
{"bdata": "ALrbAA==", "dtype": "i4"}, "xaxis": "x", "y":
["STR0059"], "yaxis": "y"}, {"base": ["2024-01-
24T08:00:00.000000"], "hovertemplate": "stream_id=SM00993<br>start_time=
%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></
extra>", "legendgroup": "SM00993", "marker":
{"color": "#FF97FF", "opacity": 0.2, "pattern":
{"shape": ""}}, "name": "SM00993", "orientation": "h", "showlegend": true, "te
xtposition": "auto", "type": "bar", "x":
{"bdata": "AHS3AQ==", "dtype": "i4"}, "xaxis": "x", "y":
["STR0063"], "yaxis": "y"}, {"base": ["2024-01-
24T09:00:00.000000"], "hovertemplate": "stream_id=SM00995<br>start_time=
%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></
extra>", "legendgroup": "SM00995", "marker":

```

```

{"color": "#FECB52", "opacity": 0.2, "pattern":
{"shape": ""}, "name": "SM00995", "orientation": "h", "showlegend": true, "te
xtposition": "auto", "type": "bar", "x":
{"bdata": "ALrbAA==", "dtype": "i4"}, "xaxis": "x", "y":
["STR0063"], "yaxis": "y"}, {"base": ["2024-03-
19T08:00:00.000000"], "hovertemplate": "stream_id=SM01017<br>start_time=
%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></
extra>", "legendgroup": "SM01017", "marker":
{"color": "#636efa", "opacity": 0.2, "pattern":
{"shape": ""}, "name": "SM01017", "orientation": "h", "showlegend": true, "te
xtposition": "auto", "type": "bar", "x":
{"bdata": "gKgSAQ==", "dtype": "i4"}, "xaxis": "x", "y":
["STR0065"], "yaxis": "y"}, {"base": ["2024-03-
19T10:00:00.000000"], "hovertemplate": "stream_id=SM01026<br>start_time=
%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></
extra>", "legendgroup": "SM01026", "marker":
{"color": "#EF553B", "opacity": 0.2, "pattern":
{"shape": ""}, "name": "SM01026", "orientation": "h", "showlegend": true, "te
xtposition": "auto", "type": "bar", "x":
{"bdata": "AHS3AQ==", "dtype": "i4"}, "xaxis": "x", "y":
["STR0065"], "yaxis": "y"}, {"base": ["2024-02-
01T12:00:00.000000"], "hovertemplate": "stream_id=SM01086<br>start_time=
%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></
extra>", "legendgroup": "SM01086", "marker":
{"color": "#00cc96", "opacity": 0.2, "pattern":
{"shape": ""}, "name": "SM01086", "orientation": "h", "showlegend": true, "te
xtposition": "auto", "type": "bar", "x":
{"bdata": "QFSJAA==", "dtype": "i4"}, "xaxis": "x", "y":
["STR0069"], "yaxis": "y"}, {"base": ["2024-02-
01T13:00:00.000000"], "hovertemplate": "stream_id=SM01098<br>start_time=
%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></
extra>", "legendgroup": "SM01098", "marker":
{"color": "#ab63fa", "opacity": 0.2, "pattern":
{"shape": ""}, "name": "SM01098", "orientation": "h", "showlegend": true, "te
xtposition": "auto", "type": "bar", "x":
{"bdata": "gMukAA==", "dtype": "i4"}, "xaxis": "x", "y":
["STR0069"], "yaxis": "y"}, {"base": ["2024-01-
24T18:00:00.000000"], "hovertemplate": "stream_id=SM01170<br>start_time=
%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></
extra>", "legendgroup": "SM01170", "marker":
{"color": "#FFA15A", "opacity": 0.2, "pattern":
{"shape": ""}, "name": "SM01170", "orientation": "h", "showlegend": true, "te
xtposition": "auto", "type": "bar", "x":
{"bdata": "QFSJAA==", "dtype": "i4"}, "xaxis": "x", "y":
["STR0074"], "yaxis": "y"}, {"base": ["2024-01-
24T19:00:00.000000"], "hovertemplate": "stream_id=SM01173<br>start_time=
%{base}<br>end_time=%{x}<br>streamer_id=%{y}<extra></
extra>", "legendgroup": "SM01173", "marker":
{"color": "#19d3f3", "opacity": 0.2, "pattern":
{"shape": ""}, "name": "SM01173", "orientation": "h", "showlegend": true, "te

```

```

xtposition":"auto","type":"bar","x":
{"bdata":"wGVSA==","dtype":"i4"},"xaxis":"x","y":
["STR0074"],"yaxis":"y"},"layout":
{"barmode":"overlay","height":600,"legend":{"title":
{"text":"stream_id"},"tracegroupgap":0},"margin":{"t":60},"template":
{"data":{"bar":[{"error_x":{"color":"#2a3f5f"},"error_y":
{"color":"#2a3f5f"},"marker":{"line":
{"color":"#E5ECF6","width":0.5},"pattern":
{"fillmode":"overlay","size":10,"solidity":0.2}},"type":"bar"}],"barpo
lar":[{"marker":{"line":{"color":"#E5ECF6","width":0.5},"pattern":
{"fillmode":"overlay","size":10,"solidity":0.2}},"type":"barpolar"}],"
carpet":[{"aaxis":
{"endlinecolor":"#2a3f5f","gridcolor":"white","linecolor":"white","min
orgridcolor":"white","startlinecolor":"#2a3f5f"},"baxis":
{"endlinecolor":"#2a3f5f","gridcolor":"white","linecolor":"white","min
orgridcolor":"white","startlinecolor":"#2a3f5f"},"type":"carpet"}],"ch
oropleth":[{"colorbar":
{"linewidth":0,"ticks":"","type":"choropleth"}],"contour":
{"colorbar":{"linewidth":0,"ticks":"","colorscale":
[[0,"#0d0887"],[0.1111111111111111,"#46039f"],
[0.2222222222222222,"#7201a8"],[0.3333333333333333,"#9c179e"],
[0.4444444444444444,"#bd3786"],[0.5555555555555556,"#d8576b"],
[0.6666666666666666,"#ed7953"],[0.7777777777777778,"#fb9f3a"],
[0.8888888888888888,"#fdca26"],
[1,"#f0f921"]],"type":"contour"}],"contourcarpet":[{"colorbar":
{"linewidth":0,"ticks":"","type":"contourcarpet"}],"heatmap":
{"colorbar":{"linewidth":0,"ticks":"","colorscale":
[[0,"#0d0887"],[0.1111111111111111,"#46039f"],
[0.2222222222222222,"#7201a8"],[0.3333333333333333,"#9c179e"],
[0.4444444444444444,"#bd3786"],[0.5555555555555556,"#d8576b"],
[0.6666666666666666,"#ed7953"],[0.7777777777777778,"#fb9f3a"],
[0.8888888888888888,"#fdca26"],
[1,"#f0f921"]],"type":"heatmap"}],"histogram":[{"marker":{"pattern":
{"fillmode":"overlay","size":10,"solidity":0.2}},"type":"histogram"}],
"histogram2d":[{"colorbar":{"linewidth":0,"ticks":"","colorscale":
[[0,"#0d0887"],[0.1111111111111111,"#46039f"],
[0.2222222222222222,"#7201a8"],[0.3333333333333333,"#9c179e"],
[0.4444444444444444,"#bd3786"],[0.5555555555555556,"#d8576b"],
[0.6666666666666666,"#ed7953"],[0.7777777777777778,"#fb9f3a"],
[0.8888888888888888,"#fdca26"],
[1,"#f0f921"]],"type":"histogram2d"}],"histogram2dcontour":
{"colorbar":{"linewidth":0,"ticks":"","colorscale":
[[0,"#0d0887"],[0.1111111111111111,"#46039f"],
[0.2222222222222222,"#7201a8"],[0.3333333333333333,"#9c179e"],
[0.4444444444444444,"#bd3786"],[0.5555555555555556,"#d8576b"],
[0.6666666666666666,"#ed7953"],[0.7777777777777778,"#fb9f3a"],
[0.8888888888888888,"#fdca26"],
[1,"#f0f921"]],"type":"histogram2dcontour"}],"mesh3d":[{"colorbar":
{"linewidth":0,"ticks":"","type":"mesh3d"}],"parcoords":[{"line":
{"colorbar":{"linewidth":0,"ticks":"","type":"parcoords"}],"pie":

```

```

[{"automargin":true,"type":"pie"},"scatter":[{"fillpattern":
{"fillmode":"overlay","size":10,"solidity":0.2},"type":"scatter"},"scatter3d":[{"line":{"colorbar":{"outlinewidth":0,"ticks":""},"marker":{"colorbar":{"outlinewidth":0,"ticks":""},"type":"scatter3d"},"scattercarpet":[{"marker":{"colorbar":{"outlinewidth":0,"ticks":""},"type":"scattercarpet"},"scattergeo":[{"marker":{"colorbar":{"outlinewidth":0,"ticks":""},"type":"scattergeo"},"scattergl":[{"marker":{"colorbar":{"outlinewidth":0,"ticks":""},"type":"scattergl"},"scattermap":[{"marker":{"colorbar":{"outlinewidth":0,"ticks":""},"type":"scattermap"},"scattermapbox":[{"marker":{"colorbar":{"outlinewidth":0,"ticks":""},"type":"scattermapbox"},"scatterpolar":[{"marker":{"colorbar":{"outlinewidth":0,"ticks":""},"type":"scatterpolar"},"scatterpolargl":[{"marker":{"colorbar":{"outlinewidth":0,"ticks":""},"type":"scatterpolargl"},"scatterternary":[{"marker":{"colorbar":{"outlinewidth":0,"ticks":""},"type":"scatterternary"},"surface":[{"colorbar":{"outlinewidth":0,"ticks":"","colorscale":[[0,"#0d0887"],[0.1111111111111111,"#46039f"],[0.2222222222222222,"#7201a8"],[0.3333333333333333,"#9c179e"],[0.4444444444444444,"#bd3786"],[0.5555555555555556,"#d8576b"],[0.6666666666666666,"#ed7953"],[0.7777777777777778,"#fb9f3a"],[0.8888888888888888,"#fdca26"],[1,"#f0f921"]},"type":"surface"},"table":[{"cells":{"fill":{"color":"#EBF0F8"},"line":{"color":"white"},"header":{"fill":{"color":"#C8D4E3"},"line":{"color":"white"},"type":"table"}}},"layout":{"annotationdefaults":{"arrowcolor":"#2a3f5f","arrowhead":0,"arrowwidth":1},"autotypenumbers":"strict","coloraxis":{"colorbar":{"outlinewidth":0,"ticks":"","colorscale":{"diverging":[[0,"#8e0152"],[0.1,"#c51b7d"],[0.2,"#de77ae"],[0.3,"#f1b6da"],[0.4,"#fde0ef"],[0.5,"#f7f7f7"],[0.6,"#e6f5d0"],[0.7,"#b8e186"],[0.8,"#7fbc41"],[0.9,"#4d9221"],[1,"#276419"]},"sequential":[[0,"#0d0887"],[0.1111111111111111,"#46039f"],[0.2222222222222222,"#7201a8"],[0.3333333333333333,"#9c179e"],[0.4444444444444444,"#bd3786"],[0.5555555555555556,"#d8576b"],[0.6666666666666666,"#ed7953"],[0.7777777777777778,"#fb9f3a"],[0.8888888888888888,"#fdca26"],[1,"#f0f921"]},"sequentialminus":[[0,"#0d0887"],[0.1111111111111111,"#46039f"],[0.2222222222222222,"#7201a8"],[0.3333333333333333,"#9c179e"],[0.4444444444444444,"#bd3786"],[0.5555555555555556,"#d8576b"],[0.6666666666666666,"#ed7953"],[0.7777777777777778,"#fb9f3a"],[0.8888888888888888,"#fdca26"],[1,"#f0f921"]}}},"colorway":["#636efa","#EF553B","#00cc96","#ab63fa","#FFA15A","#19d3f3","#FF6692","#B6E880","#FF97FF","#FECB52"],"font":{"color":"#2a3f5f"},"geo":{"bgcolor":"white","lakecolor":"white","landcolor":"#E5ECF6","showlake

```

```
s":true,"showland":true,"subunitcolor":"white"},"hoverlabel":
{"align":"left"},"hovermode":"closest","mapbox":
{"style":"light"},"paper_bgcolor":"white","plot_bgcolor":"#E5ECF6","po
lar":{"angularaxis":
{"gridcolor":"white","linecolor":"white","ticks":""},"bgcolor":"#E5ECF
6","radialaxis":
{"gridcolor":"white","linecolor":"white","ticks":""}},"scene":
{"xaxis":
{"backgroundcolor":"#E5ECF6","gridcolor":"white","gridwidth":2,"lineco
lor":"white","showbackground":true,"ticks":"","zerolinecolor":"white"}
,"yaxis":
{"backgroundcolor":"#E5ECF6","gridcolor":"white","gridwidth":2,"lineco
lor":"white","showbackground":true,"ticks":"","zerolinecolor":"white"}
,"zaxis":
{"backgroundcolor":"#E5ECF6","gridcolor":"white","gridwidth":2,"lineco
lor":"white","showbackground":true,"ticks":"","zerolinecolor":"white"}
},"shapedefaults":{"line":{"color":"#2a3f5f"}},ternary":{"aaxis":
{"gridcolor":"white","linecolor":"white","ticks":""},"baxis":
{"gridcolor":"white","linecolor":"white","ticks":""},"bgcolor":"#E5ECF
6","caxis":
{"gridcolor":"white","linecolor":"white","ticks":""}},"title":
{"x":5.0e-2},"xaxis":
{"automargin":true,"gridcolor":"white","linecolor":"white","ticks":"","
"title":
{"standoff":15},"zerolinecolor":"white","zerolinewidth":2},"yaxis":
{"automargin":true,"gridcolor":"white","linecolor":"white","ticks":"","
"title":
{"standoff":15},"zerolinecolor":"white","zerolinewidth":2}}},"width":1
000,"xaxis":{"anchor":"y","domain":[0,1],"type":"date"},"yaxis":
{"anchor":"x","domain":[0,1],"title":{"text":"streamer_id"}}}}
```

As we can see above, there are 56 instances of streamers doing more than one concurrent stream.

Young users with long account age

There are some users in age group 13-17 with account age > 3000 days. That doesn't seem good given minimum age requirements.

```
implausible_age =
df_viewers_all_fixed.loc[(df_viewers_all_fixed["age_group"] == "13-
17") & (df_viewers_all_fixed["account_age_days"] > 3000)]
display(implausible_age)

{"columns":[{"name":"index","rawType":"int64","type":"integer"},
{"name":"viewer_id","rawType":"str","type":"string"},
{"name":"age_group","rawType":"str","type":"string"},
{"name":"country","rawType":"str","type":"string"},
{"name":"account_age_days","rawType":"int64","type":"integer"},
```

```
{"name":"subscription_tier","rawType":"str","type":"string"},
{"name":"preferred_category","rawType":"str","type":"string"}],"ref":
"ef9a8ac7-9855-49a5-90e9-320d1abaffe4","rows":[[{"63","VWR00064","13-
17","US","3177","tier2","FPS"},{"107","VWR00108","13-
17","AU","3437","none","Just Chatting"},{"179","VWR00180","13-
17","US","3209","none","Just Chatting"},{"219","VWR00220","13-
17","AR","3003","none","MOBA"},{"223","VWR00224","13-
17","US","3013","none","Sports"},{"229","VWR00230","13-
17","NL","3595","tier3","FPS"},{"261","VWR00262","13-
17","AU","3462","none","MOBA"},{"355","VWR00356","13-
17","MX","3518","none","FPS"},{"376","VWR00377","13-
17","GB","3247","tier1","FPS"},{"383","VWR00384","13-
17","KR","3636","none","FPS"},{"400","VWR00401","13-
17","FR","3477","none","FPS"},{"426","VWR00427","13-
17","KR","3536","tier1","MOBA"},{"463","VWR00464","13-
17","BR","3057","none","MOBA"},{"541","VWR00542","13-
17","ES","3211","none","Just Chatting"},{"835","VWR00836","13-
17","BR","3043","tier2","Sports"},{"917","VWR00918","13-
17","MX","3572","tier3","Just Chatting"},{"978","VWR00979","13-
17","US","3065","tier3","Just Chatting"},{"998","VWR00999","13-
17","US","3628","none","Creative"}]], "shape":{"columns":6,"rows":18}}
```